UNISYS | Securing Your Tomorrow®

ClearPath OS 2200
Symbolic Stream Generator (SSG)

Programming Reference Manual

ClearPath OS 2200 Release 20.0

March 2023 7831 1784–010

NO WARRANTIES OF ANY NATURE ARE EXTENDED BY THIS DOCUMENT. Any product or related information described herein is only furnished pursuant and subject to the terms and conditions of a duly executed agreement to purchase or lease equipment or to license software. The only warranties made by Unisys, if any, with respect to the products described in this document are set forth in such agreement. Unisys cannot accept any financial or other responsibility that may be the result of your use of the information in this document or software material, including direct, special, or consequential damages.

You should be very careful to ensure that the use of this information and/or software material complies with the laws, rules, and regulations of the jurisdictions with respect to which it is used.

The information contained herein is subject to change without notice. Revisions may be issued to advise of such changes and/or additions.

Notice to U.S. Government End Users: This software and any accompanying documentation are commercial items which have been developed entirely at private expense. They are delivered and licensed as commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Use, reproduction, or disclosure by the Government is subject to the terms of Unisys' standard commercial license for the products, and where applicable, the restricted/limited rights provisions of the contract data rights clauses.

Unisys and other Unisys product and service names mentioned herein, as well as their respective logos, are trademarks or registered trademarks of Unisys Corporation. All other trademarks referenced herein are the property of their respective owners.

Contents

Section 1. General Description

1.1. Documentation Updates 1–1
1.2. Notation Conventions 1–1
1.3. Functions 1–2
1.4. Applications 1–3
1.5. SSG Features 1–3
1.5.1. Program File Support 1–3
1.5.2. Use of Additional Memory 1–3
1.6. New Features for This Release 1–4

Section 2. SSG Input and Output

2.1. @SSG Control Statement 2–1
2.1.1. Options 2–1
2.1.2. Parameters 2–3
2.2. Input from the Runstream 2–5
2.3. SSG Input 2–11
2.3.1. Skeleton (SKEL) 2–12
2.3.2. Skeleton Corrections (SKEL COR) 2–12
2.4. SSG Output 2–13
2.5. SSG Margins and Headings 2–13

Section 3. Referencing SSG Values

3.1. Stream Generation Statements 3–1
3.1.1. SGS Input Formats 3–1
3.1.2. SGS References 3–7
3.1.2.1. Standard SGS References 3–7
3.1.2.2. Extended SGS Subfield References 3–7
3.1.2.3. SGS File Name References 3–13
3.1.2.4. Composing SGS References 3–16
3.1.3. SGS Label Access 3–17
3.1.4. Automatically Defined SGSs 3–19
3.1.4.1. DATIME$ 3–19
3.1.4.2. MARGIN$ 3–20
3.1.4.3. CSFSTA$ 3–20
3.1.4.4. INFO$ 3–21
3.1.4.5. SYSTEM$ 3–22
3.1.4.6. SOURCE$ 3–24
3.1.4.7. TIME$ 3–24
3.1.4.8. SUPS$ 3–25
3.1.4.9. DATE$ 3–26

3.1.5. SGS Examples....3–26
3.2. SSG Variables....3–29
3.2.1. Creating Local and Global Variables....3–29
3.2.2. Referencing the Value of a Variable....3–29
3.2.2.1. Referencing Variables Using Bracketed References (Method 1)....3–30
3.2.2.2. Referencing Variables Using Nonbracketed References (Method 2)....3–32
3.2.3. Using Reserved Global Variables....3–33
3.2.3.1. String Descriptor Variables....3–39
3.2.3.2. PRTOFF$....3–40
3.2.3.3. SOPT$....3–40
3.2.3.4. UNS_CONTROL$ and UNS_INPUT$....3–41
3.2.3.5. FEATURE_LEVEL$....3–43
3.3. Process Parameter References....3–44
3.4. Bracketed Reference Restrictions....3–47

Section 4. Skeleton and Symstream

4.1. Introduction....4–1
4.2. Element and Version Name Restrictions....4–3
4.3. Nondirective Images....4–4
4.4. Directive Images....4–5
4.5. Defining Skeleton Image Sequences (Closed Subroutines)....4–7

Section 5. SSG Directives

5.1. Logical Directive Groups....5–1
5.1.1. Skeleton Creation....5–1
5.1.2. Skeleton Listing Control....5–1
5.1.3. Skeleton Logic Control....5–2
5.1.4. Variable Modification....5–2
5.1.5. Skeleton Communication....5–2
5.1.6. Skeleton Output Control....5–3
5.1.7. SGS Modification....5–3
5.1.8. PCF/TCF Modification....5–3
5.1.9. Miscellaneous....5–3
5.2. Directive Descriptions....5–4
5.2.1. *ABORT....5–4
5.2.2. *ACCEPT....5–4
5.2.3. *ADD....5–6
5.2.4. *BRKPT....5–7
5.2.5. *CLEAR....5–8
5.2.6. *COPY....5–9
5.2.7. *CORRECT....5–10
5.2.8. *CREATE....5–14
5.2.9. *CSF....5–16
5.2.10. *DEFINE and *ENDDEFINE....5–18
5.2.11. *DISPLAY....5–19
5.2.12. *DIVIDE....5–20

5.2.13. *DLOC 5–21
5.2.14. *DO and *ENDDO 5–22
5.2.15. *DUMP 5–22
5.2.16. *EDIT 5–23
5.2.17. *EJECT 5–27
5.2.18. *ELSE 5–28
5.2.19. *ELSEIF 5–28
5.2.20. *ENDCORRECT 5–28
5.2.21. *ENDDEFINE (,E) 5–28
5.2.22. *ENDDO (,E) 5–28
5.2.23. *ENDIF (,E) 5–28
5.2.24. *EXIT 5–28
5.2.25. *IF, *ELSEIF, *ELSE, and *ENDIF 5–29
5.2.25.1. Description of Conditions 5–32
5.2.25.2. Test for Variable Value Conditional 5–42
5.2.25.3. Test for Existence Conditional 5–43
5.2.25.4. Test for Zero Conditional 5–43
5.2.25.5. Test for Relation Conditional 5–44
5.2.25.6. Row, Column, or Keyword Search Conditional 5–46
5.2.25.7. Test for Element Entry Conditional 5–50
5.2.26. *INCREMENT and *LOOP 5–51
5.2.27. *ITERATE 5–54
5.2.28. *LIST 5–54
5.2.29. *LOOP(,E) 5–54
5.2.30. *MERGE 5–55
5.2.31. *MESSAGE 5–60
5.2.32. *MULTIPLY 5–62
5.2.33. *PCONTROL 5–62
5.2.34. *PROCESS 5–63
5.2.35. *REMOVE 5–65
5.2.36. *RESTORE 5–67
5.2.37. *RETURN 5–68
5.2.38. *SAVE 5–68
5.2.39. *SET 5–69
5.2.40. *SORT 5–72
5.2.41. *UNLIST 5–74
5.2.42. *WAIT 5–74

Section 6. Permanent and Temporary Streams

6.1. Introduction 6–1
6.2. PCF and TCF Sets 6–1
6.3. Element Entries 6–2
6.4. Permanent Streams (PCF) 6–3
6.5. Temporary Streams (TCF) 6–4
6.6. PCF and TCF Set Access 6–5
6.7. Set References 6–5
6.8. Line–Change Format 6–8
6.9. Change Control Characters 6–11
6.10. Revised Temporary Correction (RTC) Stream 6–12

Appendix A. Diagnostic, Error, and Warning Messages

A.1. Diagnostic Messages A–1
A.2. Error Messages A–2
A.3. Warning Messages A–17

Appendix B. Maintaining a Permanent TCF

B.1. Introduction B–1
B.2. Building a Software Product B–1
B.3. Correction Files (Definitions and Relationships) B–2
B.4. Local Use of a Permanent TCF B–3
B.5. A Sample Skeleton B–4

Appendix C. Examples

C.1. Introduction C–1
C.2. Tic Tac Toe Example C–2
C.3. File Lister Example C–12
C.4. A Correcting Skeleton (CORRECTSKEL) C–14
C.5. UTS Calculator Skeleton C–16
C.6. Define Packet Examples C–18
C.6.1. To Find a Day (Find_the_Day) C–18
C.6.2. A Debug Aid (Echo_Pars) C–20
C.6.3. A Recursive Define Packet (Count_Nested_Calls) C–21
C.6.4. Processing Unsolicited Input C–22

Index 1

Tables

2–1. @SSG Control Statement Options 2–1
2–2. File Identification Statements 2–6
2–3. File Identification Statement Options 2–8
2–4. OPT Statement Options 2–9

3–1. SGS References 3–7
3–2. String Descriptor References 3–8
3–3. SGS File Name Reference 3–14
3–4. File Types and Subtypes 3–15
3–5. SGS Field References 3–16
3–6. Valid $SGS References 3–17
3–7. Sample $SGS References 3–18
3–8. References to Mixed Uppercase and Lowercase SGS Labels 3–27
3–9. SSG Reserved Global Variables 3–34
3–10. Read-Only Variables 3–39
3–11. FEATURE_LEVEL$ Features 3–43
3–12. Process Parameter References 3–45
3–13. Nested Bracketed Reference Restrictions 3–49

4–1. Symstream Definitions 4–1
4–2. Symstream Definition Examples 4–3

5–1. DUMP Directive 5–23
5–2. Results of Boolean Operations Using AND, OR, XOR, and NOT 5–34
5–3. Boolean Operator Priorities 5–35
5–4. *IF Relations 5–44
5–5. Row/Columns Search Table 5–46
5–6. IF PCF TCF Search 5–50
5–7. Input to MERGESKEL 5–59
5–8. MESSAGE Options 5–60

6–1. PCF and TCF Set References 6–6

A–1. File-Id and Descriptions A–4

Section 1
General Description

The Symbolic Stream Generator (SSG) is a general purpose interpreter that manipulates input and output symbolic streams. SSG is used to generate, maintain, and update the proper system configuration for OS 2200 Exec, processors, and libraries. This section describes SSG and its applications.

This manual can be used as a reference document for descriptions of SSG messages, input syntax, and error messages. The intended audience of the manual is primarily programmers responsible for the generation and maintenance of software products and programmers who need a flexible method for creating and maintaining runstreams or other symbolic files or elements.

The manual provides information on how to use SSG to manipulate one or more symbolic input streams and to create one or more output streams. To use SSG, you must have a basic knowledge of OS2200 Executive Control Language (ECL), file usage and administrative concepts. Further information on these can be found in the manuals listed in the Related Product Information appendix of this document.

1.1. Documentation Updates

This document contains all the information that was available at the time of publication. Changes identified after release of this document are included in problem list entry (PLE) 19333442. To obtain a copy of the PLE, contact your Unisys representative or access the current PLE from the Unisys Product Support website:

http://www.support.unisys.com/all/ple/19333442

Note: *If you are not logged into the Product Support site, you will be asked to do so.*

1.2. Notation Conventions

The examples in this manual show, as nearly as possible, how the SSG output and your input appear during a demand session.

Note: *The $ character is used in reserved names. To avoid duplication and confusing results, do not use this character in names you define.*

The following conventions are used in this manual to describe the symbolic input to SSG

- Ellipsis (...)—indicates that you can give any number of additional parameters of the same format as the last shown.
- Brackets [ ] —delimit five types of SSG references. The brackets must be ordered as shown.
- Parentheses ( )—contain optional items in syntax descriptions. Also used in text to identify superscripted characters (5th).
- Oversized braces—designate a required choice of terms. You must select one of the choices listed vertically within the braces. In the following example, you must select either the LOCAL or GLOBAL keyword for the *DUMP directive:

$$\text{*DUMP} \quad \left\{ \begin{matrix} \text{LOCAL} \\ \\ \text{GLOBAL} \end{matrix} \right\}$$

Oversized brackets—designate an optional choice of terms. You can select one or none of the choices listed vertically within these brackets. In the following example, you can select either one or none of the three optional file formats specified for the *EDIT directive:

$$\text{*EDIT}\ (, options) \quad \text{ON} \left[\begin{matrix} edtsym \\ edtsym.\ comment \\ .comment \end{matrix} \right]$$

Uppercase text (TEXT)—represents input entered as is. Unless otherwise indicated, this text can be coded in upper- or lowercase.

> ***Note:*** *SSG directive names can be entered in either uppercase or lowercase alphabetic characters.*

- Lowercase, italicized text (*text*)—represents user-supplied parameters. Unless otherwise indicated, you must supply a string, variable reference, SGS reference, set reference, or process parameter reference in its place.

1.3. Functions

SSG enables you to create and control a symbolic environment. SSG directs and manipulates one or more symbolic input streams and creates one or more symbolic output streams. The SSG processor is a general purpose interpreter that manipulates symbolic streams. Directions to build stream images are given to SSG through a skeleton (program) written in Symstream. You can place the resulting symbolic output streams in files or elements and then list or dynamically add (@ADD) them before you terminate SSG. When SSG terminates, the @ADD control statement, which you can suppress, makes the symbolic images part of the runstream.

SSG also helps you maintain special symbolic streams that you can list, correct, and update to use later. The primary purpose of these special symbolic streams is to maintain corrections to be applied to symbolic elements in program files. These symbolic streams are the permanent correction file (PCF) and the temporary correction file (TCF). Using SSG, you can merge correction streams with other correction streams, list and place them in the generated output stream, and insert them into their original files.

1.4. Applications

SSG is used to generate OS 2200 Exec, processors, and libraries. SSG maintains and updates permanent corrections (PCF streams) by applying input derived from temporary corrections (TCF streams), and skeleton-generated corrections. It creates the appropriate runstream images to generate the proper system configuration under the control of one or more skeletons.

You can use SSG to create symbolic streams such as data images, OS 2200 Exec control statements, correction images, and COBOL or FORTRAN source language statements.

1.5. SSG Features

The SSG features described in this section are of special note.

1.5.1. Program File Support

All references to program files within this manual apply to both standard program files (PF) and large program files (LPF).

SSG does not support large element program files (LEPF) in most contexts. However, Table of Contents (TOC) system generation statements (SGSs) can be created for LEPF elements by specifying the '*filename./label*' format on the SSG call line (see 2.1.2), on the SGS file identification statement (see 2.2), and on the *ADD directive (see 5.2.3).

Neither program files nor SDF files can be word-addressable files. SSG does not support word-addressable files.

1.5.2. Use of Additional Memory

SSG operates with up to 100 data banks, each with the capacity of approximately octal 0675000 or decimal 228,000 words.

Data banks are dynamically acquired as needed during processing. The number of banks that can be acquired is fixed by the variable MAXBNKS in element SSG$DEF. If more memory is needed, the value of MAXBNKS can be increased from its initial 100 to a maximum of 503. The SSG termination line contains the number of data banks acquired during processing. For more information about the SSG termination line, refer to A.1.

1.6. New Features for This Release

The following new features are included in the SSG level 24R4:

- The global reserved read-only variable FEATURE_LEVEL$ identifies SSG features supported for the currently executing SSG. The variable value has been incremented from 1 to 2 for the 24R4 release. For more information, refer to 3.2.3.5, Table 3-9, and Table 3-11.
- The new SGS option U has been implemented to specify that the long format TOC SGS date and time information should be based on UTC time instead of on local time. The option can be used with the SGS file identification statement (refer to 2.2) and with the *ADD directive (refer to 5.2.3). Also, refer to 3.1.1 and Table 2-3.
- The long format TOC SGSs have been enhanced with additional date and time information and formats. Refer to 3.1.1.

Section 2
SSG Input and Output

This section describes the options and parameters used on the @SSG control statement, the length of images in the skeleton streams, and margins and headings for generated output.

2.1. @SSG Control Statement

The @SSG control statement has the following format:

```
@label:SSG,options  param-1,param-2,param-3,param-4,param-5,param-6,...
```

Options and parameters are discussed in the following subsections.

2.1.1. Options

Use discretion when you select *options* because several can produce a lengthy printed listing.

The *options* for the @SSG control statement are listed and described in Table 2–1.

Table 2–1. @SSG Control Statement Options

Option	Description
A	Continue SSG execution regardless of "no find" conditions.
B	Do not dynamically add (@ADD) the last part of the generated output stream. Without the B option, and in the absence of errors, the last part of the output stream is automatically added by SSG (see the L option description; also see 2.4).
C	Double space all listings.
D	In combination with the E option, eliminate indention.
E	List the revised skeleton stream in a structured, indented form. The D option can be included to eliminate indentation.
F	List input permanent streams (all PCF sets).
G	List input temporary streams (all TCF sets).

Table 2–1. @SSG Control Statement Options

Option	Description
H	List the revised temporary correction (RTC) stream. See also RTC$ (Table 3–9) and *CORRECT (5.2.7).
I	List SGSs as they are input. See IOPT$ (Table 3–9) and Table 2–3.
J	List revised PCF*xxxx* element entries for which there are corresponding (TCF*xxxx*) element entries at the conclusion of skeleton processing. The page is ejected for each element listed. Only those PCF sets with a program file attached are listed (see 6.4). The R option should not be used in conjunction with the J option.
K	List part 1 of the generated output stream.
L	Use @ADD,L (instead of @ADD) to dynamically add the last part of the generated output stream.
M	Suppress printing of the SSG sign-on line, copyright lines, sign-off line, and echo of file specification images.
N	Suppress headings generated by SSG.
O	Order PCF*xxxx* element entries in ascending order. Any duplicate element entries, within a given PCF set, will be noted and the second entry discarded. The sort is according to the ASCII collating sequence (see the X option).
P	PCF*xxxx* element entries are to be composed of the symbolic element entries from a program file specified after *param-6* on the @SSG call (see 6.4).
Q	Order TCF*xxxx* element entries in ascending order. Any duplicate element entries within a given TCF set are noted and the second entry discarded. The sort is according to the ASCII collating sequence (see also the X option).
R	List revised PCF*xxxx* element entries at the conclusion of skeleton processing. PCF sets with a program file attached are listed (see 6.4). The J option should not be used in conjunction with the R option.
S	Order SGSs with the same label in ascending order, according to the first subfield SGS. The sort is according to the ASCII collating sequence (see also the X option). See SOPT$ (Table 3–9) and 3.2.3.3 and the *SORT directive (5.2.40).
T	TCF*xxxx* element entries are composed of the symbolic element entries from a program specified after *param-6* on the @SSG call (see 6.5).
U	List revised TCF*xxxx* element entries at the conclusion of skeleton processing. TCF sets with a program file attached are listed (see 6.5).
V	Print both input and updated line numbers with the correction lines.
W	List correction lines at the top of the listing.

Table 2–1. @SSG Control Statement Options

Option	Description
X	Used in combination with any of the O, Q, or S options. Nonnumeric strings are left justified and space-filled; numeric substrings are right justified and zero-filled. This is the recommended option for sorting library elements. The following example illustrates how the X option works with two SGSs: X AA000000000123xyz22!! X AA123xyz022! @SSG,SX sorts the example placing the second SGS first. The numeric substrings are equivalent, but one exclamation mark (!) is less than two exclamation marks (!!), so the sorted order is reversed. When sorted without the X option, the order of the elements is the same as the example because the character 0 is less than the character 1. The numeric substrings given have no size limit since they are not evaluated.
Y	Cause the skeleton to be read in and a revised skeleton to be produced, but the revised skeleton is not interpreted. Also, an A option is assumed for *COPY images. This option is only useful for validating matching skeleton directives and applying skeleton corrections to obtain a revised skeleton for later interpretation.
Z	Reserved for site or user use (see ZOPT$, Table 3–9). If the site elects to use the Z option, the user cannot.

2.1.2. Parameters

All call line parameters are optional. Input and output streams specified as parameters on the @SSG call can be either system data format (SDF) files or symbolic elements in program files. The only exception is for *filename./label* stream generation statement (SGS) input, where *filename* must be a program file (or empty).

Use standard *filename.eltname* notation. For more information, refer to the *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949).

SSG accepts mixed ASCII and Fieldata input. Though SSG input is not required, you can use SSG to validate and list any combination of input streams.

Parameters

param-1

specifies the source of the skeleton stream (see 2.3.1).

param-2

specifies the source of SGSs. Two input formats are allowed:

Format 1

An SDF file or a symbolic element containing SGSs.

Format 2

```
filename./label
```

where the *filename* specified must be a program file (or empty). The *filename* can be a standard program file (PF), a large program file (LPF), or a large element program file (LEPF). SSG takes the element names from the program file and attaches the specified *label* to each, creating one SGS for each nondeleted element present. See 3.1.1 for the format of these SGSs created by SSG. An SGS label supplied in this format is a maximum of 12 characters in length and can include the characters A through Z, 0 through 9, the dollar sign ($), or hyphen (-).

Both types of SGS input can also be used after *param-6* on the @SSG call (see Section 3).

param-3

specifies the destination for part 1 of the generated output stream.

param-4

specifies the destination of the revised temporary correction (RTC) stream (see 6.10).

param-5

specifies the destination of the revised skeleton stream.

param-6

specifies the source of the corrections to be applied to the skeleton stream. The default control character for images in the skeleton correction stream is the minus sign (-), but it can be changed (see 2.3.2).

param-7

The first of a number of fields specifying input streams. The formats are as follows (must always start at *param-7*):

Permanent corrections

```
PCFxxxx/number,name-1,...,name-n
```

Temporary corrections

```
TCFxxxx/number,name-1,...,name-n
```

Stream generation statements

```
SGS/number,name-1,...,name-n
```

Additional SSG options

```
OPT/extra-options
```

where:

number

specifies the number of input *name-i* to be supplied for the specified type of input stream. If *number* is omitted, 1 is assumed, in which case the slash (/) preceding *number* should be omitted.

name-1,...,name-n

specifies SDF files or symbolic elements that are the source for the specified type of input stream. For SGS input, either format described for *param-2* can be used in any sequence.

extra-options

specifies additional SSG options as described in Table 2-4.

You can repeat this format any number of times by placing a comma (,) between *name-n* and the next type of input stream.

Note: *If the P or T options are present for PCF and TCF input,* number *must always be 1 (or nothing) and* name-1 *must be a program file (see 6.4 and 6.5). This form tends to make the SSG call line hard to read.*

Example

```
PCF2/1,FILE
```

No more than one file is associated with a P or T option PCF or TCF set name.

2.2. Input from the Runstream

The recommended way to input to SSG is from the runstream where file identification (file-id) statements define the type of input stream. The file-id statements, which are shown in Table 2–2, are echoed by SSG if

- This is a batch run.
- You perform an @ADD on the images.
- You breakpoint PRINT$.

Table 2–2. File Identification Statements

File-id Statement	Type of Input
SKEL (*specification*)	Skeleton stream
SKEL COR (*specification*)	Skeleton correction stream
SGS(,*options*) (*specification*)	Stream generation statements
PCF*xxxx*(,*options*) (*specification*)	Permanent corrections
TCF*xxxx*(,*options*) (*specification*)	Temporary corrections
OPT,*extra-options*	Additional SSG options

where:

options

can be supplied by a string (see Table 2–3).

extra-options

can be supplied by a string (see Table 2–4).

specification

is the name of an SDF file or symbolic element that is the source for the specified type of input stream. Standard filename.eltname notation is used to interpret *specification*. The *specification* is not necessary and, if omitted, input from the runstream following the file-id statement is assumed.

Four *specification* formats are available for the SGS file-id statement:

Format 1

$$\left\{\begin{array}{l} filename. \\ filename.element \end{array}\right\}$$

is the standard format for the source of SGSs, either an SDF file, a standard program file (PF) symbolic element, or a large program file (LPF) symbolic element.

Format 2

filename./label

is used to create table of content (TOC) SGSs with the specified *label* for each nondeleted element in a program file. The *filename* can be a standard program file (PF), a large program file (LPF), or a large element program file (LEPF). The C and U options can be used with this format. The format is described in 2.1.2 for param-2, Format 2.

Format 3

filename.element label

is used to create TOC SGSs with the specified *label* for the specified nondeleted elements. The *filename* can be a standard program file (PF) or a large program file (LPF). The *filename* cannot be a large element program file (LEPF). This format requires the E option. The A, C, O, R, S, and U options shown in Table 2–3 can be used with this format.

Format 4

$$\left\{ \begin{matrix} filename. \\ \\ filename.element \end{matrix} \right\} label$$

is used to create SGSs with the specified *label* for each image in an SDF file, a standard program file (PF) symbolic element, or a large program file (LPF) symbolic element. This format requires the L option. The B option shown in Table 2–3 can be used with this format.

If the P option is specified for the PCF*xxxx* file-id statement, *specification* must be present and must be a program file or an empty file.

If the T option is specified for the TCF*xxxx* file-id statement, *specification* must be present and must be a program file or an empty file.

Table 2–3. File Identification Statement Options

Option	Description
SGS File Identification Statement	
A	Format 3 only. Create a TOC SGS for the absolute element specified. An error occurs if the absolute element specified is not in the file. Requires the E option.
B	Format 4 only. Do not create an SGS for blank images. Requires the L option.
C	Formats 2 and 3 only. Create the short form of the TOC SGSs. The short form is shown in 3.1.1.
E	Format 3 only. Create a TOC SGS for each non-deleted element specified. The A, O, R, and S options can be included to select specific element types. If none are included, a TOC SGS is created for each element type present in the file. An error occurs if the element specified is not in the file. The C option can be included to create the short form of the TOC SGS. The forms of the TOC SGS are shown in 3.1.1.
I	All formats. List SGSs as they are input. The I option overrides the absence of the I option on the @SSG control statement.
L	Format 4 only. Create an SGS with the specified *label* for each image in the SDF file or symbolic element. The B option can be included.
N	All formats. Do not list SGSs. The N option overrides the I option on this SGS file-id statement and overrides the I option on the @SSG control statement
O	Format 3 only. Create a TOC SGS for the omnibus element specified. An error occurs if the omnibus element specified is not in the file. Requires the E option.
R	Format 3 only. Create a TOC SGS for the relocatable element specified. An error occurs if the relocatable element specified is not in the file. Requires the E option.
S	Format 3 only. Create a TOC SGS for the symbolic element specified. An error occurs if the symbolic element specified is not in the file. Requires the E option.
U	Formats 2 and 3 only. Use to specify that date and time information on the TOC SGSs should be expressed in Coordinated Universal Time (UTC) instead of the default, local time. Ignored when the C option is specified.
PCF*xxxx* File Identification Statement	
P	PCF*xxxx* element entries are to be composed of the symbolic element entries from the program file *specification*, which is required.
TCF*xxxx* File Identification Statement	
T	TCF*xxxx* element entries are to be composed of the symbolic element entries from the program file *specification*, which is required.

Table 2–4. OPT Statement Options

Extra Option	Description
C	Use to suppress printing of *COPY images (*COPY,N).
D	Combine with the SSG B and K options to list the source line numbers of images output by the skeleton. The number is part of the image and thus the image is unusable. Also prints *PROCESS images as they are interpreted.
E	Use to cause SSG to take an ER EXIT$ exit, rather than an ER ERR$ exit.
M	Use to debug. Lists skeleton images as they are interpreted and lists symbolic images as they are generated (see MOPT$, Table 3–9). Skeleton images consisting of only a comment (*) are not listed (see 4.5).
N	Use to debug. Lists value of variables as they are modified, *IF directive information, and the source of skeleton images (see NEW_BOOLEAN$ and NOPT$ in Table 3–9).
U	Use to specify that date/time information on the automatically defined DATE$, DATIME$, and TIME$ SSGs should be based on Coordinate Universal Time (UTC) instead of the default, local time.
V	Use to suppress printing of vertical scores for indented revised skeleton listings.

Description

Any number of file-id statements are allowed in any order. All file-id statements follow the optional MARGIN statement (see 2.5).

If *specification* is present on the file-id statement, the images for the specified type of input stream are taken from that file or element.

If *specification* is omitted, SSG reads all images from the runstream following the file-id statement until a control statement is encountered and then associates those images with the input stream. A matching @EOF control statement is required if input from the runstream is followed by another file-id statement. Any other control statement inhibits SSG from reading further runstream images.

Since standard filename.eltname notation is used to interpret *specification*, a space-period-space (.) on a file-id statement terminates the scan of that file-id statement.

When formats 2, 3, and 4 are used for the SGS file-id statement, an SGS *label* must be specified. In all cases, the *label* is from 1 to 12 characters from the set of alphabetics (A through Z and a through z), numerics (0 through 9), dollar sign ($), and minus sign (-).

When formats 2, 3, and 4 are used for the SGS file-id statement and when the minus sign (-) is used as part of the label, the following special formatting considerations apply:

- For the *filename.element label* specification (formats 3 and 4), the *label* is a separate syntactic unit. In this context, a minus sign in the *label* is a "break" character (see Table 4–1), which indicates the end of the *label*. For this entire *label* to be processed correctly, it must be enclosed in a pair of single quotation marks.

- For the *filename./label* specification (format 2 only) and the *filename. label* specification (format 4 only), the *label* is syntactically processed with the *filename*. In this context, a minus sign is considered a valid element or version name character. The entire *label* is processed correctly without special formatting. Do not enclose the *label* in a pair of single quotation marks.

When multiple OPT file-id statements are included, the specific *extra-options* are additive. That is, the specified *extra-options* are added to all of the *extra-options* specified on the previous OPT file-id statements.

For example, the three OPT file-id statements

```
OPT,CV
OPT,E
OPT,U
```

are equivalent to the OPT file-id statement

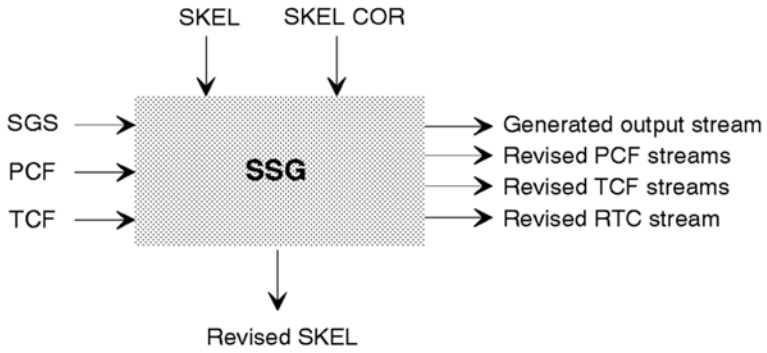
```
OPT,CVEU
```

Examples

```
SGS,E UPDATES.DAILY ''NEW-TODAY''
SGS,LB EXCEPTION*RECORDS.NOFIND  ''CUST-NO-FIND''
SGS,C LEVEL1R1*SOURCE./BASE-ELTS
SGS,L SUMMARY.  UP-TO-DATE
```

2.3. SSG Input

The following diagram shows how SGS, PCF, TCF, SKEL, and SKEL COR input streams and files can be interpreted by SSG to produce different forms of output streams and files.



SSG accepts a maximum of 132 characters per image. When the size of an input image exceeds the maximum number of characters, SSG truncates the image, if possible, and issues a diagnostic message.

Some input images can be continued on the next sequential image by placing a semicolon (;) at the end of the image to be continued.

The rules for continuing input images depend on the type and usage of the input image. Additional information on continuing specific images is available as follows:

- SGS input streams – see 3.1.1.
- SKEL and SKEL COR input streams.
  - SGS references – see 3.1.2.4 .
  - Process parameter references – see 3.1.2.4 and 3.3.
  - PCF and TCF set references – see 3.1.2.4 and 6.7.
  - Bracketed variable references – see 3.2.2.1.
  - Directive images – see 4.4.

2.3.1. Skeleton (SKEL)

The skeleton stream is a collection of Symstream statements interpreted by SSG to generate one or more symbolic output streams (see Section 4). It can be supplied by *param-1* on the @SSG call, by the *specification* on a SKEL file-id statement, or as one or more images following a SKEL file-id statement. Skeleton corrections can be used to update the skeleton. The skeleton can make use of SGSs and correction streams to create one or more symbolic output streams.

You can supply multiple skeleton streams; however, only the contents of the last skeleton stream you supplied are retained.

For a more detailed description of the skeleton, refer to Section 4.

2.3.2. Skeleton Corrections (SKEL COR)

The skeleton correction stream is a collection of correction images that is applied to the skeleton stream before SSG interprets it. The resulting symbolic stream is referred to as the revised skeleton stream, or the skeleton (SKEL). You can supply skeleton corrections by *param-6* on the @SSG call, by the *specification* on a SKEL COR file-id statement, or images following a SKEL COR file-id statement.

When you start a skeleton correction, the control character is the minus sign (-). You can change it by placing *-=new change characters* as the first image of the skeleton correction stream. You cannot make partial line corrections. You can use constructions that use the last image indicator (!).

You can supply multiple skeleton correction streams. However, only the content of the last skeleton correction stream is retained. No effort is made by SSG to merge multiple skeleton correction streams. In addition, a change of control character image in an earlier supplied skeleton correction stream is not brought forth to the last supplied skeleton correction stream. The control character always reverts to the minus sign (-) character.

You can save and list the revised skeleton stream. For more information, refer to *param-5* in 2.1.2 and the E and D options in Table 2–1.

2.4. SSG Output

All output is optional.

The revised skeleton stream is the result of applying the input skeleton correction stream to the input skeleton stream. (If no skeleton correction stream exists, the revised skeleton stream is exactly like the input skeleton stream.)

The generated output stream consists of symbolic images output when you execute the revised skeleton stream. You can use the skeleton to direct symbolic input images and created symbolic images to the generated output stream. Unless you use the B option on the @SSG call, the last part of the generated output stream (*BRKPT) is dynamically added by SSG (unless errors were encountered) with an @ADD or @ADD,L control statement before termination. If you specify *param-3* on the @SSG call or you supply a *name* on the last *BRKPT directive, the last file or element is added (only one stream is added). If you do not specify a file or element, SSG automatically adds an internally created stream. You can use nested @SSG calls (one generated output stream that contains an @SSG call that, in turn, adds its generated output stream) to virtually any level.

The revised PCF*xxxx* stream consists of those symbolic elements in the program file attached to PCF*xxxx*. See 6.4 for attaching a program file to a PCF set. See 5.2.7 for modifying the program file.

The revised TCF*xxxx* stream consists of those symbolic elements in the program file attached to TCF*xxxx*. See 6.5 for attaching a program file to a TCF set. See 5.2.30 for modifying the program file.

The revised temporary correction (RTC) stream is discussed in 6.10.

If SSG encounters errors during processing, bit 8 of the run condition word is set upon termination. This occurs if the value of the SSG variable ERRCNT$ is nonzero, an abort directive is encountered, or processing is abnormally terminated.

2.5. SSG Margins and Headings

Use the following format to change margins:

```
MARGIN(,options) (l,t,b,(lpi))
```

where:

options

must be supplied by a string:

P Prints this MARGIN statement.

R Leaves the user-specified margins in effect after exit from SSG.

l

is the number of lines per page. Can be supplied by a numeric string of 1, 2, or 3 characters.

t

is the number of blank lines at the top of page. Can be supplied by a numeric string of 1, 2, or 3 characters.

b

is the number of blank lines at the bottom of the page. Can be supplied by a numeric string of 1, 2, or 3 characters.

lpi

is the number of lines per inch. Can be supplied by a numeric string of 1, 2, or 3 characters.

The MARGIN statement, if present, immediately follows the @SSG call. SSG then performs an ER APRTCN$.

- If you do not supply a margin image, the margins in effect when you execute SSG remain in effect until SSG terminates.
- If you do supply a MARGIN, SSG uses these margins, and restores the margins to system default (*,*,*) when it terminates.
- If you supply the R option on the MARGIN statement, the margins remain in effect after SSG terminates.
- If you supply an invalid MARGIN statement, the system default values are used.

For a skeleton to determine which margin values are specified, SSG automatically defines an SGS (see 3.1.4) in the following format:

```
MARGIN$  *,*,* (l,t,b(,lpi))
```

where *,*,* reflect run-time margin values (in lieu of a MARGIN statement) and are provided for information only.

- If you supply a MARGIN statement, field 2, (including *l* ,*t* ,*b,* and *lpi* if specified) is included.
- If you do not supply a MARGIN statement to SSG, field 1 always exists.

Note: *The values *,*,* are not used currently. (If future Exec enhancements allow the system default margins to be obtained, then those values will be substituted for the *,*,* values.)*

Section 3
Referencing SSG Values

This section discusses stream generation statements, variables, process parameter references, and bracketed reference restrictions.

3.1. Stream Generation Statements

You can use stream generation statements (SGS) to provide tables or lists of data for skeletons.

3.1.1. SGS Input Formats

It is helpful to think of an SGS (or group of SGSs) as a matrix. SGSs can be

- Automatically defined by SSG
- Created from the *param-2* and *param-7* parameters of the @SSG call
- Created from an SGS file-id statement
- Created during skeleton processing by the *ACCEPT, *ADD, or *CREATE directives

Format

```
label ( subfield-11,…,subfield-1n … subfield-m1,…,subfield-mn )
```

Parameters

label

Can be supplied by a string. The string can begin with a semicolon (;); it cannot contain a space, period, or comma. The label is truncated at 32 characters (see 3.1.3).

subfield

Is a string that does not begin with a semicolon (;) and does not contain a space, period, or comma.

Note: *A subfield may contain up to 128 characters, the maximum string length (see Table 4–1).*

Three methods allow a subfield to contain any character.

Method 1

Delimit the subfield with single quotation marks (‘…’). The single quotation marks are part of the subfield and are counted in the subfield length. This is referred to as a quoted subfield (see Table 3–2).

Method 2

Delimit the subfield with pairs of single quotation marks (‘‘…’’). The pairs of single quotation marks are not part of the subfield and are not counted in the subfield length. This is referred to a double quoted subfield (see Table 3–2). A double quoted subfield may not contain pairs of single quotation marks (nesting is neither allowed nor necessary).

Method 3

Enclose the characters with pairs of single quotation marks (…‘‘…’’…). The pairs of single quotation marks are not part of the subfield and are not counted in the subfield length. However, this is not a double quoted subfield unless the pairs of single quotation marks are the first characters and the last characters in the subfield.

Note: *Do not confuse pairs of single quotation marks (two consecutive single quote characters) with the double quotation mark. Pairs of single quotation marks(‘’) are used to enclose special characters, such as a semicolon, space, period, or comma. Double quotation marks (“) have no special significance to SSG and are treated as normal characters.*

SGSs can be continued on more than one line by using a trailing semicolon. A trailing semicolon is a semicolon preceded by a space (;) or a semicolon preceded by a comma (,;) that occurs on an SGS image. Any data that comes after the trailing semicolon on an SGS image is ignored.

An embedded semicolon is a semicolon that is not preceded by a space or a comma. It is treated as a normal character in an SGS subfield and does not trigger continuation.

Note: *SSG ignores the last input SGS from a file or file-id statement if the last line contains a continuation character (;) preceded by a space.*

Example 1

```
LABEL1 This SGS will be continued ;
to the next line because of the trailing semicolon
```

Example 2

```
LABEL2 This SGS will be continued,;
to the next line because of the trailing semicolon
```

Example 3

```
LABEL3 This SGS will not be continued; because the semicolon is
embedded
```

Example 4

```
LABEL4 Trailing    ,spaces in an SGS act as field separators
```

Field 2, subfield1 is null, and field 2, subfield 2 contains “spaces”.

Example 5

```
LABEL5 Leading,     spaces in a subfield are ignored.
```

“spaces” is field 1, subfield 2 and has a length of 6.

Example 6

```
LABEL6 These,,are,'''',three ,examples of null subfields
             ^      ^        ^
```

The ^ points to field 1, subfield 2, field 1, subfield 4, field 2, subfield 1, all of which are null subfields.

Example 7

```
LABEL7
```

LABEL7 has no fields or subfields explicitly defined, but there is an implied null field 1, subfield 1.

Description

- SGS labels, fields, and subfields
  - An SGS has one label.
  - An SGS has any number of fields, which are separated by spaces.
  - Each field can have from one to any number of subfields, which are separated by commas.
  - An SGS that consists of a label and no fields is treated as an SGS statement with one field containing one null subfield. A null subfield is a subfield that contains no characters.
  - Any number of SGS images with the same label or different labels can exist.

- Scanning SGSs
  - The SGS is free form; the label need not start in column 1.
  - A period in any position on the SGS terminates the scan of that image unless the period is enclosed in single quotation marks or pairs of single quotation marks.
  - An asterisk followed by a period (*.) is treated as an SGS with label * and one null subfield. It is not treated as a comment when the SGS is scanned.
  - The scan of an SGS ignores leading spaces. It interprets the first trailing space as a field separator.
  - The scan of an SGS interprets the first trailing comma as a subfield separator.
- Alphabetic characters in SGS labels

  For input and referencing, alphabetic characters in an SGS label are converted to uppercase characters. Therefore, the SGS labels LAB, lab, Lab, and laB are all treated as being identical. The subfields on an SGS are maintained in uppercase and lowercase as you input them.

- Constraints on SGS label usage

  Some label names should not be used for SGSs that you define. These include the following:

  - Names that are reserved for special uses, such as # (see 3.3 and 5.2.34), $PCF (see 6.6), $SGS (see 3.1.3), $TCF (see 6.6), P (see 6.7), and T (see 6.7)
  - SGS label names that are automatically defined (described in 3.1.4)
  - SGS label names that begin or end with $, which could be incompatible with future releases
- Creating TOC SGSs

  SSG can create table of contents (TOC) SGSs from the nondeleted elements in a standard program file (PF), a large program file (LPF), or a large element program file (LEPF). You can create these SGSs by specifying *filename./label* SGS input, where *label* is the SGS label for all TOC SGSs created from nondeleted elements in *filename*. The label supplied in this format can be a maximum of 12 characters and can include the characters A through Z, 0 through 9, the dollar sign ($), or hyphen (-).

- The *filename./label* specification for TOC SGS input

  You can use this specification in the following formats:

  - As *param-2* on the SSG call line (see 2.1.2)
  - As *param-7* on the SSG call line (see 2.1.2)
  - On the SGS file-id statement (see 2.2)
  - On the *ADD directive (see 5.2.3)

- TOC SGS formats

  There are short and long versions of this SGS. The long version is the default and appears in this format:

```
label element-name version-name ;
      numeric-element-type(,numeric-element-subtype) ;
     mnemonic-element-type(,mnemonic-element-subtype(,latest-element-cycle)) ;
      yymmdd,hhmmss,yyyymmdd,yyyy-mm-dd,hh:mm:ss(,zzzz) ;
      element/version qual*file sector-address,length-in-sectors
```

  You can create the short TOC SGS by using the SGS file-id or the *ADD directive C option. You cannot obtain this version by supplying *filename./label* as *param-2* or *param-7* on the SSG call line. The short version has the following format:

```
label element-name version-name ;
       numeric-element-type(,numeric-element-subtype)
```

When the I option is specified the TOC SGSs are printed. For the long format the first four lines (*label* and fields 1 to 5) appear on the first line of the SGS and the last line (fields 6 to 8) appear on the continuation of the SGS. For the short format both lines (*label* and fields 1 to 3) appear on a single line with no continuation.

- TOC SGS subfields
  - If the version name of an element is blank, that field exists but returns zero characters on reference or tests (null subfield).
  - If there is no element subtype, field 3, subfield 2 is omitted. However, *element-name* is always in field 1, subfield 1; *version-name* is always in field 2, subfield 1; and *numeric-element-type* is always in field 3, subfield 1.
  - The *numeric-element-type* and *numeric-element-subtype* are numeric subfields.
  - For *mnemonic-element-subtype* definitions, refer to the *SYSLIB Programming Reference Manual* (7833 1733).
  - For LEPFs, the *length-in-sectors* subfield has a value of -1 if the element has an illegal text length granularity (not sectors, tracks or positions) or if the text length cannot be converted to a single-precision, positive decimal integer.

- Long format TOC SGS date and time information
  - The date and time information is in field 5 and is based on the Reversed-TDATE$ or Modified-SWTIME timestamp stored in the program file element for this instance of the TOC SGS.
  - The optional sixth subfield "zzzz" ([*label*,*n*,5,6]) is the one- to four-character time zone mnemonic. If the timestamp is in Reversed-TDATE$ format, then this subfield is absent and the number of subfields in field 5 ([*label*,*n*,5]) is 5. If the timestamp is in Modified-SWTIME format, then this subfield is present and the number of subfields in field 5 ([*label*,*n*,5]) is 6.

    Note: *The "zzzz" ([*label,n,*5,6]) subfield is filled with four hyphens "----" when the site uses zeroes or ASCII spaces for the mnemonic name in their system's time zone configuration .*
  - If the SGS U option is specified (see Table 2-3), then the date and time information is expressed in UTC time. If the SGS U is not specified (the default), then the information is expressed in local time. The U option is ignored for short format TOC SGSs.
  - The presence or absence of the U option affects both the date information in subfields 1, 3, and 4 and the time information in subfields 2, 5, and 6.
  - The U option can be specified for TOC SGSs on an SGS file identification statement (see 2.2) and on an *ADD directive (see. 5.2.3).
  - The U option cannot be specified for TOC SGSs on SSG call line parameters param-2 and param-7 (see 2.1.2).
  - When the U option is specified and the timestamp is in Reversed-TDATE$ format, SSG assumes that a seasonal offset was not in effect when converting the local time value to UTC time.
  - When the U option is specified and the timestamp is in Reversed-TDATE$ format, the time zone offset used to convert it to UTC time is the offset configured for your system. The resulting UTC time may or may not be the actual UTC time when the program file element was created.
  - When the U option is specified and the timestamp is in Modified-SWTIME format, the time zone mnemonic in subfield 6 is "UTC".
  - When the U option is not specified and the timestamp is in Modified-SWTIME format, the times in subfields 2 and 5 and the time zone mnemonic in subfield 6 are the local times and the standard or seasonal mnemonic configured for your system. These may or may not be the local times and time zone mnemonic for the system where the program file element was created.
  - The setting of the reserved global variable UTC$ does not affect the time base used for TOC SGS date and time information.
- TOC SGS file name references

  If you create an SGS of the form *filename./label*, file information is saved and becomes associated with the SGS label. The file information can then be referenced, as described in 3.1.2.3.

3.1.2. SGS References

This subsection discusses the three SGS reference types and how to compose them.

3.1.2.1. Standard SGS References

You can reference an SGS in the skeleton according to label, statement, field, and subfield. After you make an SGS reference, the value returned, SYMBOLIC or NUMERIC, is substituted for the reference. With a label *l*, a statement number *n*, a field number *f*, and a subfield number *s*, you can make the SGS references shown in Table 3–1.

Table 3–1. SGS References

SGS Reference	Value Returned	Type
[*l*]	Number of SGSs with the label *l*. If none, a value of 0 is returned.	NUMERIC
[*l*,*n*]	Number of fields on the *n th* SGS with the label *l*. If the referenced statement does not exist, a NO FIND condition exists.	NUMERIC
[*l*,*n*,*f*]	Number of subfields in the *f th* field on the *n th* SGS with the label *l*. If the referenced statement or field does not exist, a NO FIND condition exists.	NUMERIC
[*l*,*n*,*f*,*s*]	Contents of the *s th* subfield in the *f th* field on the *n th* SGS with the label *l*. If the referenced statement, field, or subfield does not exist, a NO FIND condition exists.	SYMBOLIC

3.1.2.2. Extended SGS Subfield References

In addition to the references in Table 3–1, information can be obtained about a particular subfield using the 19 string descriptor references. All references are for the *s th* subfield in the *f th* field on the *n th* SGS with the label *l*. If the subfield or character referenced (only SUBSTR$) does not exist, a NO FIND condition exists.

The string descriptor references in Table 3–2 are shown with the SSG global variable names that correspond to string descriptor numbers 0 through 18 (see Table 3–10). If desired, the string descriptor numbers can be used in place of the SSG global variable names.

When SGSs are the output from the SGSSTR$ function, continuation occurs on a subfield boundary. If the value of the variable IMGLEN$ (see Table 3–9) is less than the number of characters in an output subfield, the subfield will always be split in two. The maximum size of an SGS subfield is 128 characters. The value of IMGLEN$ can be set to 132 to get around the problem.

Table 3–2. String Descriptor References

SGS Reference	Value Returned	Type
[*l*,*n*,*f*,*s*,SUBSTR$,*sc*,*nc*]	Substring of length *nc* starting at character *sc*. Characters are numbered from left to right, starting at 1. *nc* must be ≥ 0. If *sc* and/or *nc* are omitted, they are assumed to be 1. If a subfield is a null subfield, *sc* can only evaluate to 1, and *nc* can only evaluate to 0. If L is the number of characters in a non-null subfield and *nc* ≥ 1, then *nc*, *sc*, *nc*, and L must satisfy the following constraint: 1 ≤ *sc* ≤ *sc*+*nc*-1 ≤ *L*; if *nc* equals 0, 1 ≤ sc ≤ *L*.	SYMBOLIC
[*l*,*n*,*f*,*s*,ALEN$]	Number of characters in the subfield if every character is alphabetic; otherwise, 0.	NUMERIC
[*l*,*n*,*f*,*s*,QLEN$]	If the subfield is delimited by single quotation marks ('…'), the number of characters in the subfield, including the single quotation marks; 1 if the subfield is a single quotation mark; otherwise, 0.	NUMERIC
[*l*,*n*,*f*,*s*,DQLEN$]	If the subfield is delimited by pairs of single quotation marks (''…''), the number of characters in the subfield, not including the pairs of single quotation marks; otherwise, 0. A subfield consisting of four single quotation marks is a zero-length, double-quoted subfield. Such a subfield is undetectable unless it is the last subfield in a field. Such an observation is important when echoing SGSs in some output stream.	NUMERIC
[*l*,*n*,*f*,*s*,NLEN$]	Number of characters in the subfield if every character is numeric; otherwise, 0.	NUMERIC
[*l*,*n*,*f*,*s*,LEN$]	Number of characters in the subfield.	NUMERIC

Table 3–2. String Descriptor References

SGS Reference	Value Returned	Type
[*l*,*n*,*f*,*s*,SGSSTR$,*ef*,*es*]	Returns a string that starts at field *f*, subfield *s* of the specified SGS and ends at field *ef*, subfield *es* of the same SGS. This allows groups of fields and subfields to be returned with one reference, which eliminates the need to loop through each field and subfield. If the ending field or subfield does not exist, no error occurs. The string contains everything up to and including the last field and/or subfield. If an SGS subfield was double-quoted, it is double-quoted on the returned string. If the SGS reference is part of an output image and continuation must occur, the continuation occurs on a subfield boundary, unless the number of characters in a subfield is greater than IMGLEN$. ***Note:*** *The $SGS label cannot be used with this reference (see 3.1.3).*	SYMBOLIC
	The following example illustrates the SGSSTR$ reference: *CREATE SGS: CHECK This is an SGS ; of the form,A23 The SGS reference [CHECK,1,4,1,SGSSTR$,7,2] or [CHECK,1,4,1,6,7,2] returns: SGS of the form,A23 Using this SGS reference with automatically defined SGSs may exceed the text limit in skeleton communication directives such as *DISPLAY and *MESSAGE.	
[*l*,*n*,*f*,*s*,LCLEN$]	Returns the number of characters if all characters are lowercase; otherwise 0.	NUMERIC

Table 3–2. String Descriptor References

SGS Reference	Value Returned	Type
l,*n*,*f*,*s*,UCLEN$]	Returns the number of characters if all characters are uppercase; otherwise 0.	NUMERIC
[*l*,*n*,*f*,*s*,ANLEN$]	Returns the number of characters if all characters are alphanumeric; otherwise 0.	NUMERIC
[*l*,*n*,*f*,*s*,SCLEN$]	Returns the number of characters if all characters are alphanumeric, $, or -; otherwise 0.	NUMERIC
[*l*,*n*,*f*,*s*,LCSTR$]	Returns the normal character string with all lowercase characters if the CASE$ variable is set to 0.	SYMBOLIC
[*l*,*n*,*f*,*s*,UCSTR$]	Returns the normal character string with all uppercase characters if the CASE$ variable is set to 0.	SYMBOLIC
[*l*,*n*,*f*,*s*,BSTR$]	Returns the string that precedes, but does not include, the character set in STRINGCHAR$. If STRINGCHAR$ contains an alphabetic character, CASECHAR$ determines if the comparison is case sensitive or not case sensitive. If the STRINGCHAR$ character is not found, the entire string is returned.	SYMBOLIC
[*l*,*n*,*f*,*s*,ASTR$]	Returns the string that follows, but does not include the character set in STRINGCHAR$. If STRINGCHAR$ contains an alphabetic character, CASECHAR$ determines if the comparison is case sensitive or not case sensitive. If the STRINGCHAR$ character is not found, a null string is returned.	SYMBOLIC

Table 3–2. String Descriptor References

SGS Reference	Value Returned	Type
[*l*,*n*,*f*,*s*,MSTR$]	Returns the string between, but not including, the character set in STARTCHAR$ and STOPCHAR$, respectively. If STARTCHAR$ or STOPCHAR$ contains an alphabetic character, CASECHAR$ determines if the comparison is case sensitive or not case sensitive. If the STARTCHAR$ character is not found, a null string is returned. If the STOPCHAR$ character is not found, the entire string following the STARTCHAR$ character is returned.	SYMBOLIC
[*l*,*n*,*f*,*s*,LSTR$,*nc*]	Returns a string composed of the left *nc* characters. If the string is less than *nc* characters long, the returned string is space-filled to *nc* characters. A negative number can be specified for the *nc* value and it returns all characters in the string except the left *nc* characters.	SYMBOLIC
[*l*,*n*,*f*,*s*,RSTR$,*nc*]	Returns a string composed of the right *nc* characters. If the string is less than *nc* characters long, the returned string is space filled to *nc* characters. A negative number can be specified for the *nc* value and it returns all of the characters in the string except the right *nc* character.	SYMBOLIC

Table 3–2. String Descriptor References

SGS Reference	Value Returned	Type
[*l*,*n*,*f*,*s*,PSGSTR$,*ef*,*es*]	Returns a string that starts at field *f*, subfield *s* of the specified SGS and ends at field *ef*, subfield *es* of the same SGS. This allows groups of fields and subfields to be returned with one reference, which eliminates the need to loop through each field and subfield. If the ending field or subfield does not exist, no error will occur. The string will contain everything up to and including the last field and/or subfield. If an SGS subfield was double-quoted, it will be double-quoted on the returned string. If the SGS reference is part of a non-directive image, the start of each field except the first is output to a new image, indented by 3 spaces. If the SGS reference is part of a non-directive image and continuation must occur, the continuation will occur on a subfield boundary, unless the number of characters in a subfield is greater than IMGLEN$. The continued subfield is indented by 6 spaces on the new image. ***Note:*** *The $SGS label cannot be used with this reference (see 3.1.3).* The following example illustrates the PSGSTR$ reference: *CREATE SGS: CHECK This is an SGS of ; the form,A23 The SGS reference [CHECK,1,4,1,PSGSTR$,7,2] or [CHECK,1,4,1,18,7,2] returns SGS ; of ; the ; form,A23	SYMBOLIC

3.1.2.3. SGS File Name References

A file can become associated with an SGS label in several ways:

- A specification of '*filename./label*' for *param-2* or *param-7* on the @SSG call line. See 2.1.2.
- An 'SGS *filename./label*' file identification statement. See 2.2.
- Interpretation of an '*ADD SGS *filename./label*' directive. See 5.2.3.
- Interpretation of a '*CREATE SGS *filename./label*' directive. See 5.2.8.

The file name references can also be used when a program file becomes associated with a P option PCF set or a T option TCF set. In these cases, the set name PCF*xxxx* or TCF*xxxx* replaces the SGS label in the file name references. See 6.2 and 6.7.

SSG saves information about the file and makes it available with the file name references shown in Table 3–3.

- If more than one program file is associated with *label*, the file name references will return the information for the most recently processed file.
- If a file is not associated with *label*, the file name references will result in a NO FIND condition.

Table 3–3. SGS File Name Reference

SGS Reference	Value Returned	Type
[/,0,D]	Directory-id associated with label */* as determined by ER FITEM$.	SYMBOLIC
[/,0,Q]	Qualifier associated with label */* as determined by ER FITEM$.	SYMBOLIC
[/,0,F]	File name associated with label */* as determined by ER FITEM$.	SYMBOLIC
[/,0,C]	F-cycle associated with the label */* as determined by ER FITEM$. If necessary, a leading + or – is attached to the F-cycle indicating a relative, rather than absolute, F-cycle.	SYMBOLIC
[/,0,R]	Read key associated with the label */* as determined by ER MSCON$.	SYMBOLIC
[/,0,W]	Write key associated with the label */* as determined by ER MSCON$.	SYMBOLIC
[/,0,T]	File type as determined from the first sector of the file. See Table 3–4.	SYMBOLIC
[/,0,S]	File subtype as determined from the first sector of the file. See Table 3–4.	SYMBOLIC

The extended SGS subfield references described in 3.1.2.2, except the SGSSTR$ and PSGSTR$ string descriptors, can be used with the file name references. For example, [*label*,0,F,LEN$] could be used to obtain the number of characters in the file name of the file associated with */label*.

Table 3–4 lists the file type and file subtype symbolic values that can be returned for various types of files.

Table 3–4. File Types and Subtypes

File Type	Type – [*label*,0,T]	Subtype – [*label*,0,S]
Standard program file (PF)	PF	PF
Large program file (LPF)	LPF	LPF
Standard large element program file (LEPF) [1]	FILE	LEPF
Large large element program file (LEPF) [1]	FILE	LEPF
System Data Format (SDF) file [2]	FILE	SDF
Empty file	FILE	EMPTY
None of the above	FILE	OTHER [3]

Notes:

1. *As indicated in the 'Type' column, LEPFs are not completely supported as program files by SSG. They are supported only in the SGS contexts described at the beginning of this section.*
2. *SSG only recognizes the SDF file types documented in the* Data Structures Programming Reference Manual (7833 3481). *Non-standard SDF files may not be recognized by SSG.*
3. *Subsequent SSG features might uniquely identify files currently identified with a subtype of 'OTHER'.*

3.1.2.4. Composing SGS References

The fields of an SGS reference can be composed as shown in Table 3–5.

Table 3–5. SGS Field References

SGS Reference	Value Returned
l	Can be a string as described under *label* in 3.1.1 or an SGS reference, set reference, variable reference, or a process parameter reference that returns such a string. If a string is used, the first character should not be *, #, or [.
n, f, s, sc, nc, ef, es	Can be an integer expression or numeric expression.
string descriptor number	Can be an integer expression or numeric expression.
D,Q,F,C,R,W,T,S (on 3rd parameter of file name reference)	Can be a string, variable reference, or process parameter reference whose first character returns uppercase or lowercase D, Q, F, C, R, W, T, or S.

Note: *SGS references or set references (see 6.7) used to supply the label, statement number, field number, subfield number, subfield descriptor number, starting character, or number of characters cannot contain another SGS reference or set reference. SGS references and set references can be nested to only one level.*

Any comma in the reference may be followed by one or more spaces.

Any comma or space in the reference may be followed by a semicolon to indicate that the reference is continued on the next sequential image.

Any characters after the semicolon are ignored. Leading spaces on the next image are ignored.

Note: *All of the SGS reference composition rules described in this section also apply to method 2 process parameter references (see 3.3) and to PCF and TCF set references (see 6.7).*

The following examples are logically identical:

Examples

```
[LABEL,INST,FLD,SUB,SUBSTR$,START,NUM]

[LABEL,INST,;

            FLD, SUB, ;

                      SUBSTR$,  START,  NUM]
```

3.1.3. SGS Label Access

SSG has a facility that allows access to SGS labels without requiring knowledge of the actual SGS labels. The reserved label $SGS can be used to return the ith defined SGS label, to determine whether an ith SGS label was defined, and to return the total number of SGS labels defined. Table 3–6 illustrates the valid bracketed $SGS references and what values they return.

Table 3–6. Valid $SGS References

$SGS Reference	Value Returned
[$SGS]	Returns the number of defined SGSs.
[$SGS,*n*]	Returns 1 if there are at least *n* SGSs defined; otherwise a NO FIND error is returned.
[$SGS,*n*,*f*]	Always returns a NO FIND.
[$SGS,*n*,1,1]	Returns the nth defined SGS label. If it does not exist, then a NO FIND error is returned. The [$SGS,*n*,*f*,*s*] reference will always return a NO FIND error if either *f* or *s* is not 1.
[$SGS,*n*,1,1,*sd*]	String descriptor references (see Table 3–2) can be used with the $SGS reserved label (except for SGSSTR$ and PSGSTR$). The [$SGS,*n*,*f*,*s*,*sd*] string descriptor reference will always return a NO FIND error if either *f* or *s* is not 1.

Examples

```
LAB1  This,is the first user-defined,SGS
LAB1  This,is the second SGS defined with the LAB1 label
LAB2  A new,SGS label
LAB3  The last,SGS defined
```

Table 3–7 shows references that use the SGSs listed in these examples (in addition to the automatically defined SGSs described in 3.1.4).

Table 3–7. Sample $SGS References

$SGS Reference	Value Returned	Explanation
[$SGS]	12	There are 12 distinct SGS labels defined (9 automatically defined, see 3.1.4, and 3 user-defined).
[$SGS,10,1,1]	LAB1	LAB1 if the 10 th defined SGS label (the first 9 are automatically defined).
[$SGS,11,1,1]	LAB2	Even though there are 2 SGSs with the label LAB1, LAB2 is the 11 th defined SGS label.
[$SGS,1]	1	The first SGS label is defined (the automatically defined DATIME$ SGS).
[$SGS,13]	NO FIND	The 13 th SGS label is not defined.
[$SGS,1,2]	NO FIND	This type of reference [$SGS,*n*,*f*] always returns a NO FIND.
[$SGS,1,1,2]	NO FIND	In this [$SGS,*n*,*f*,*s*] reference, *s* is not 1, resulting in a NO FIND.
[$SGS,10,1,1,LEN$]	4	The substring descriptor reference LEN$ returns the number of characters in the 10th defined SGS label (LAB1).

The SGS label with #, created if NEW_PROCESS$ is nonzero, cannot be referenced using the $SGS reserved label.

Lowercase alphabetic characters appearing in an SGS label are converted to uppercase before the label is placed in the SGS label chain.

Note: *If* filename./label *is used to define SGSs, and* filename *is empty or contains zero elements, no SGSs are created, and* label *does not appear in the SGS label chain.*

3.1.4. Automatically Defined SGSs

When SSG is initialized, there are nine defined SGSs present. They are described in the following subsections in the order in which they appear in the SGS label chain. To distinguish which SGS labels you can define when using $SGS, a reserved global variable SSGSGS$ is defined. SSGSGS$ is set to 9 and corresponds to the number of automatically defined SGS labels. To avoid possible confusion, it is recommended that you do not use these nine labels for SGS labels that you define.

Note: *Any automatically defined SGS label can be removed (*REMOVE). Removing the label does not change the value of SSGSGS$, but it does change the value of SGSCNT$ (even though these SGSs are not originally counted in SGSCNT$).*

3.1.4.1. DATIME$

The DATIME$ SGS returns the date and time when SSG is invoked. If the U extra option (see 2.2 and Table 2-4) is present, then the date and time information is based on UTC time. Otherwise, the date and time information is based on local time.

Format

```
DATIME$ mm/dd/yy hh:mm:ss,fffffffff,zzzz yymmdd hhmmss ;
        yyyy mm/dd/yyyy yyyymmdd dow,ndow yyyy-mm-dd ;
        time-zone-offset,seasonal-offset,time-zone-mnemonic
```

where:

fffffffff

is the nine-digit fractional seconds; nanoseconds past *hh:mm:ss*.

zzzz

is the time zone mnemonic for the time, as selected by the U extra option.

dow

is the three-character abbreviation for the day of the week (for example, Sun, Mon, or Tue). The *dow* subfield is derived from the SYSLIB SFDTBL$ table. This allows for localization, since the SFDTBL$ table can be modified at a site. For more information, refer to the *SYSLIB Programming Reference Manual* (7833 1733).

ndow

is the numeric day of the week (for example, 1 for Sunday, 2 for Monday, and 3 for Tuesday).

time-zone-offset

is the signed offset from UTC time in seconds for the system. A minus sign precedes negative value. No sign character is used for a positive value.

seasonal-offset

is the seasonal time offset in seconds for the system.

time-zone-mnemonic

is the system time zone mnemonic for the system.

The *time-zone-offset*, *seasonal-offset*, and *time-zone-mnemonic* subfields are based on the current values for the system, and are not affected by the presence of the U extra option.

The date and time values are based on when SSG is invoked and do not change throughout skeleton execution. The time base used for the SGS subfield values is based on the presence or absence of the U extra option when SSG is involved. Skeleton changes to the value of the UTC$ reserved global variable do not change the format of the DATIME$ SGS.

DATIME$ is the first SGS label defined ([$SGS,1,1,1]).

3.1.4.2. MARGIN$

The MARGIN$ SGS returns run-time margin values configured by SSG in field 1, sub-field 1, and user-specified (through the MARGIN statement) margin values in field 2.

Format

```
MARGIN$ *,*,* (l,t,b(,lpi))
```

If a MARGIN statement is not supplied, field 2 is omitted. (Field 1 is not currently in use. It always contains the value * * *.)

MARGIN$ is the second SGS label defined ([$SGS,2,1,1]).

3.1.4.3. CSFSTA$

The CSFSTA$ SGS returns the status of the most recently interpreted *CSF directive (see 5.2.9) in field 1, subfield 1.

Format

```
CSFSTA$ 12-octal-digit-status-code
```

The fields always contain 12 characters, right justified, and are initially filled with zeros. See also CSFSTA$ in Table 3–9.

CSFSTA$ is the third SGS label defined ([$SGS,3,1,1]).

3.1.4.4. INFO$

The INFO$ SGS supplies information about you when SSG is executed.

Format

```
INFO$     run-id,grun-id acct-num proj,implied-qual,default-qual ;
          user-id,privileged-state run-mode,brkpt-mode run-cond ;
          initial-dir-id,implied-dir-id,default-dir-id
```

where:

run-id

is the run-id of the user.

grun-id

is the generated run-id.

acct-num

is the account number of the run.

proj

is the project-id.

implied-qual

is the implied qualifier.

default-qual

is the default qualifier.

user-id

is the user-id.

privileged-state

is the initial privileged state of the run. It can have a value of ON or OFF. Refer to 5.2.13 for information on how this subfield can be used with the DLOC directive.

run-mode

has a value of either DEMAND, BATCH, or DEADLINE.

brkpt-mode

has a value of BRKPT or NOBRKPT.

run-cond

is a 12-digit octal number representing the value of the *run-condition* word, which is retrieved at the start of SSG execution with the COND$ Executive request.

initial-dir-id

is the initial directory id. It has a value of STD, SHARED, or BOTH.

implied-dir-id

is the implied directory-id, which can be changed with @QUAL dir-id#. It has a value of STD, SHARED, or BOTH.

default-dir-id

is the default directory-id, which can be changed with @QUAL,D dir-id#. It has a value of STD, SHARED, or BOTH.

INFO$ is the fourth SGS label defined ([$SGS,4,1,1]).

3.1.4.5. SYSTEM$

The SYSTEM$ SGS supplies information about the program environment that you are running.

Format

```
SYSTEM$     exec-lev,sys-site-id  ;
            input-device,terminal-type  ;
            system-type,system-mcn,system-architecture  ;
            mhfs-status,mhfs-host-id
```

where:

exec-lev

is the release level of the Exec currently executing.

sys-site-id

is the system's configured site-id.

input-device

is the site-id of the demand terminal.

terminal-type

is a twelve-digit octal number containing terminal type information.

If this is a normal demand run, the SSG reference [SYSTEM$,1,2,2,SUBSTR$,1,3] contains the value 001, the SSG reference [SYSTEM$,1,2,2,SUBSTR$,7,3] contains the terminal device family, and the SSG reference [SYSTEM$,1,2,2,SUBSTR$,10,3] contains the terminal device type. For a description of device family and device type values, refer to the *System Interface for Legacy Application Systems (SILAS) Configuration and Operations Guide* (7851 5475).

A batch run will have a terminal type value of twelve zeroes.

Alternate communications interfaces (such as, TAS, cpFTP, DDP-FJT) can each have unique terminal type information.

system-type

is the system type, such as 2200/8030 (includes Dorado 700/800 systems), and 2200/8060 (includes Dorado 4200 systems).

system-mcn

is the Manufacturing Control Number (MCN) for the system.

system-architecture

is the system technical architecture for the system. It has a value of H for a CMOS system and a value of E for a SAIL-based system.

mhfs-status

is the status of Multi-Host File Sharing (MHFS) for the system. Values are

NC	MHFS is not configured.
UP	MHFS is configured, up, and available.
DN	MHFS is configured but down.
RV	MHFS is configured but reserved.

mhfs-host-id

is the single, upper-case, alphabetic character A to D that identifies this system in the MHFS environment. If *mhfs-status* is NC, then this subfield is null.

SYSTEM$ is the fifth SGS label defined ([$SGS,5,1,1]).

3.1.4.6. SOURCE$

The SOURCE$ SGS supplies the file and element name of the skeleton source. It also supplies the name of the file containing the SSG absolute that is executing.

Format

```
SOURCE$  ''file.element'' file,element absolute-file
```

Both file and element contain the absolute or relative cycle number as part of the name. All three fields always exist but can be null (zero characters). In the case of the runstream supplying the skeleton, the first two fields are null, and the third field contains the name of the file from which SSG was executed.

SOURCE$ is the sixth SGS label defined ([$SGS,6,1,1]).

3.1.4.7. TIME$

The TIME$ SGS returns the current time. The presence of the U extra option (see 2.2 and Table 2-4) and the value of the UTC$ reserved global variable (see Table 3-9) determine if the time information is based on local time or UTC time.

Format

```
TIME$ hh:mm:ss,fffffffff,zzzz milliseconds-past-midnight hhmmss ;
      time-zone-offset,seasonal-offset,time-zone-mnemonic
```

where:

fffffffff

is the nine-digit fractional seconds; nanoseconds past *hh:mm:ss*.

zzzz

is the time zone mnemonic for the time, as selected by the U extra option and the UTC$ variable.

milliseconds-past-midnight

is the 12-digit decimal milliseconds past midnight, with leading zeroes; the value can be from 000000000000 through 000086399999.

time-zone-offset

is the signed offset from UTC time in seconds for the system. A minus sign precedes a negative value. No sign character is used for a positive value.

seasonal-offset

is the seasonal time offset in seconds for the system.

time-zone-mnemonic

is the system time zone mnemonic for the system.

The *time-zone-offset*, *seasonal-offset*, and *time-zone-mnemonic* subfields are based on the current values for the system, and are not affected by the presence of the U extra option or the value of the UTC$ reserved global variable.

A new date/time is retrieved each time the TIME$ SGS is referenced in the skeleton.

TIME$ is the seventh SGS label defined ([$SGS,7,1,1]).

3.1.4.8. SUPS$

The SUPS$ SGS returns the current values for the standard units of processing accumulated for the RUN.

Format

```
SUPS$  total-time    CPU-time    CC/ER-time    I/O-time ;
       Wait-time     CBSUP-time  ER-I/O        ER-IP ;
       CS-I/O        CS-IP
```

All fields in this SGS are listed in milliseconds.

SUPS$ is the eighth SGS label defined ([$SGS,8,1,1]).

3.1.4.9. DATE$

The DATE$ SGS returns the current date. The presence of the U extra option (see 2.2 and Table 2-4) and the value of the UTC$ reserved global variable (see Table 3-9) determine if the date information is based on local time or UTC time.

Format

```
DATE$ mm/dd/yy yymmdd yyyy mm/dd/yyyy yyyymmdd dow,ndow yyyy-mm-dd
```

where:

dow

is the three-character abbreviation for the day of the week (for example, Sun, Mon, or Tue). The *dow* subfield is derived from the SYSLIB SFDTBL$ table. This allows for localization, since the SFDTBL$ table can be modified at a site. For more information, refer to the *SYSLIB Programming Reference Manual* (7833 1733).

ndow

is the numeric day of the week (for example, 1 for Sunday, 2 for Monday, and 3 for Tuesday).

A new date/time is retrieved each time the DATE$ SGS is referenced in the skeleton.

DATE$ is the ninth SGS label defined ([$SGS,9,1,1])

3.1.5. SGS Examples

Example 1

This example uses mixed uppercase and lowercase SGS labels for easier reading. No distinction is made between them. The following unsorted SGSs are used:

```
Where     183,Adomain                Mpls          MN        56634
When      21,October,2005            +12465
Who       Cable,Able                 Where,1       When,2
Where     154,Robert                 St Paul       MN        55127
When      12,April,2005              564Z,,LAST
Who       Brian,Juliet               Where,2       When,1
```

This example also uses the variable Z (see 3.2.1), which has the value 1, and the process parameter references [#1] and [#2] (see 3.3), which have the values Who and 2.

The references shown in Table 3–8 would return the values indicated.

Table 3–8. References to Mixed Uppercase and Lowercase SGS Labels

Reference	Value	Comment
[Where]	2	Number of SGSs with the label Where. (NUMERIC)
[[#1],1]	3	Number of fields on the first SGS with the label Who. (NUMERIC)
[When,Z,2]	1	Number of subfields in the second field on the first SGS with the label When. (NUMERIC)
[Who,2,1,[#2]]	Juliet	Contents of the second subfield in the first field on the second SGS with the label Who. (SYMBOLIC)
[When,1,2,Z,4]	0	Returns the value 0 because not all characters in the first subfield in the second field on the first SGS with the label When are numeric. (NUMERIC)
[When,[#2],2,1,5]	4	Number of characters in the first subfield in the second field on second SGS with the label When. (NUMERIC)
[[Who,Z,[#2],1],; [[#1],Z,[#2],2],Z,; [#2]]	Adomain	Example of how SGSs can be nested to one level. The label is taken from [Who,Z,[#2],1] and is Where. The statement number is taken from [[#1],Z,[#2],2] and is 1. The field number is taken from the value of variable Z and is 1. The subfield number is taken from [#2] and is 2. Taken together, this is equivalent to [Where,1,1,2]. (SYMBOLIC)
[When,2,2,2,4]	0	A null subfield consists of 0 numeric characters. (NUMERIC)

Example 2

Given the following runstream:

```
@RUN ,,SSG
@ASG,T  PF.
@ASG,C  BETA/RD/WRIT.
@ASG,C  ELL*DELTA(2).
@USE ED, ELL*DELTA(2).
@SSG,BK  ,PF./NEWL,,,,,SGS,ED./OLDL
SGS  BETA/RD/WRIT./NEWL
SKEL
Qual for NEWL is [NEWL,0,Q].
Filename for NEWL is [NEWL,0,F].
Qual for OLDL is [OLDL,0,Q].
F-cycle for OLDL is [OLDL,0,C].
@EOF
@EOF
```

The following output is produced:

```
@ASG,T  PF.
@ASG,C  BETA/RD/WRIT.
@ASG,C  ELL*DELTA(2).
@USE ED,ELL*DELTA(2).
@SSG,BK  ,PF./NEWL,,,,,SGS,ED./OLDL

SSG   GENERATED OUTPUT STREAM PART 1
              1.       1         Qual for NEWL is SSG.
              2.       2         Filename for NEWL is BETA.
              3.       3         Qual for OLDL is ELL.
              4.       4         F-cycle for OLDL is 2.
```

You can reference only the last program file associated with the label NEWL by the external file name reference. The reference goes from left to right on the @SSG call and down through file-id statements.

3.2. SSG Variables

This subsection discusses creating local and global variables, referencing the value of variables, and using reserved global variables.

3.2.1. Creating Local and Global Variables

You can create a local variable using an *INCREMENT or *DO directive. You can create a global variable by using a *SET, *CLEAR, *MULTIPLY, or *DIVIDE directive. If the variable already exists, its value changes. Variable names can be uppercase or lowercase.

- A global variable name is a string beginning with an alphabetic character. You can use only the first 32 characters.
- A local variable name (*INCREMENT or *DO) is similar to a global variable name except that it is limited to a maximum of 8 characters. You will see a warning message if a local variable name exceeds 8 characters.

Note: *Some variable names (described in 3.2.3) are reserved by SSG. You should not use these names for any global or local variables that you define. Also, to avoid possible incompatibilities with future SSG levels, do not select global and local variable names ending with $.*

Local and global variables can contain 36-bit signed integer values. When used this way, they are referred to as numeric variables. Global variables can also contain up to 128 characters of data. When used this way, they are referred to as string variables.

When you make a reference to a variable, the variable names are searched as follows:

- The local variables are searched first.
- If there is a local variable with that name, it is used for the operation.
- If no local variable exists, or the name is too long for a local variable, the global variables are searched.
- If a global variable is found, it is used to satisfy the reference.
- If no global variable is found, a NO FIND error is generated or the variable is created.

3.2.2. Referencing the Value of a Variable

There are two methods of referencing the value of a variable. Method 1 uses bracketed references and can be used for both numeric and string variables. Method 2 uses nonbracketed references and can be used for numeric variables only.

3.2.2.1. Referencing Variables Using Bracketed References (Method 1)

```
[*varname]
```

This referencing method is the most general and the most powerful. It can be used in almost all cases.

Formats

The following bracketed references can be used for both numeric and string variables:

```
[*varname]
```

For numeric variables, this reference returns a decimal numeric string. For string variables, it returns the string value of the variable.

```
[*varname,H]
```

For numeric variables, this reference returns the value of the variable as a hexadecimal numeric string. For string variables, the reference is illegal.

```
[*varname,O]
```

For numeric variables, this reference returns the value of the variable as an octal numeric string. The number returned is always preceded by a zero. For string variables, the reference is illegal.

```
[*varname,O12]
```

For numeric variables, this reference returns the value of the variable as an octal numeric string. Te returned string is always 12 digits long, zero-filled. For string variables, the reference is illegal.

```
[*varname,B]
```

For numeric variables, this reference returns the value of the variable as a binary numeric string. For string variables, the reference is illegal.

```
[*varname,B36]
```

For numeric variables, this reference returns the value of the variable as a binary numeric string. The returned string is always 36 digits long, zero-filled. For string variables, the reference is illegal.

```
[*varname,sd]
[*varname,sd,nc]
[*varname,sd,sc,nc]
```

For string variables, these references allow use of all SGS string descriptors (see 3.1.2.2) except SGSSTR$ (6) and PSGSTR$ (18). Continuation using semicolon is not allowed for such references. Remember that a space is considered to be neither numeric nor alphabetic. In these references, *sd* can be provided by a string descriptor variable (see 3.2.3.1). For numeric variables, these references are illegal.

The following rules apply when composing method 1 variable references:

- The **varname* and format (H, O, O12, B and B36) fields must be strings. Bracketed references cannot be used to supply these fields.
- The *sd*, *sc* and *nc* fields can be integer expressions or numeric expressions.
- Any comma in the reference may be followed by one or more spaces.
- Any comma or space in the reference may be followed by a semicolon to indicate that the reference is continued on the next sequential image. Any characters after the semicolon are ignored. Leading spaces on the next image are ignored.

Examples

```
*SET  varlength = [*stringvar,LEN$]
```

After execution of this directive line, *varlength* will be a numeric variable whose value is the number of characters in the string variable *stringvar*.

```
#MASM,L  [*sourcefile].elt,[*relfile].elt
```

This bracketed referencing method is used to reference variables in a nondirective line. The only time that method 2 can be used in a nondirective line is when the variable is part of an SGS reference, specifying a statement number, field, subfield or string descriptor field value.

```
*IF  [*stringvar] = YES
```

This bracketed referencing method is used to ensure that the *IF directive will treat *stringvar* as a variable and not as an ASCII string. The directive line, *IF *stringvar* = YES will always evaluate to FALSE because SSG interprets *stringvar* as an ASCII string to be compared with the ASCII string YES.

```
*DISPLAY  'Decimal Value: [*numvar] Octal Value: [*numvar,O]'
```

Variable references in quoted strings, as in the preceding *DISPLAY directive, must use this bracketed variable referencing method. The only time that method 2 (see 3.2.2.2) can be used in a quoted string is when the variable is part of an integer expression or a numeric expression in a bracketed reference. The value of numeric variable "N" uses method 2 in the following example:

```
*SET N = [LABEL]
*DISPLAY 'There are [LABEL,N] fields on the last LABEL SGS'
```

3.2.2.2. Referencing Variables Using Nonbracketed References (Method 2)

```
varname
```

This referencing method can only be used for numeric variables. The variable name is specified without being enclosed in brackets. Several formats can be used.

Formats

```
varname
```

When specified in this format, this is an integer expression, as described in Table 4–1.

```
+varname

-varname
```

When specified with a leading unary + or -, this is a numeric expression, as described in Table 4–1. There are some contexts, such as the *IF directive, where *varname* must be specified in this format to be recognized as a variable name.

Examples

```
*SET  limit  TO  [label]
*INCREMENT  loop_cnt  TO  limit
#MASM,SVW  [label,loop_cnt,1,1]
*LOOP
```

In line 1 of this example, *label* is an SGS label, not a variable. [*label*] is not a bracketed variable reference because there is no * in front of *label*, as in variable referencing method 1.

In line 2, the *INCREMENT directive is expecting an integer or numeric expression for the *to-value*. An integer expression, as stated in Table 4–1, can be a variable name; hence, *limit* is resolved to its current value and used as the *to-value* for the loop.

Line 3 illustrates when a method 2 variable reference will be resolved in a nondirective line. The variable *loop_cnt* is used within the SGS reference as the statement number specification.

```
*SET  stringvar  TO  'a string'
*DISPLAY  'This is ' stringvar
```

In the preceding *DISPLAY directive line, *stringvar* is used correctly as a valid specification, which can be a numeric or string variable as well as a quoted string.

```
*IF  +number > 0
```

For the preceding *IF (Test for Relation Conditional) directive, the variable *number* is in the operand-1 field, which can be a numeric expression. According to Table 4–1, *+number* is a numeric expression, and therefore, a valid way to check if the value of the variable *number* is greater than zero.

```
*IF  VALUE  OF  number > 0
```

The VALUE OF integer expression in this directive is semantically equivalent to *IF +*number* > 0. The VALUE OF keywords ensure that *number* is treated as a numeric variable, not as an ASCII value to be compared to ASCII 0.

3.2.3. Using Reserved Global Variables

SSG automatically defines reserved global variables for special purposes. These variables should be used only in the correct context as described in Table 3–9 and Table 3–10.

You cannot modify the values of reserved global variables described as read-only in Table 3–9 and all of the reserved global variables in Table 3–10. If you attempt to do so, the following warning message appears:

```
*WARNING* - ATTEMPT AT LINE n TO MODIFY READONLY VARIABLE: name
```

All reserved global variables end with a single dollar sign. Since more reserved global variables might be added in future levels of SSG, you should avoid creating any global and local variable names that end in $.

Table 3–9. SSG Reserved Global Variables

Name	Description
ACCSGS$	Set to the label name of an SGS input with *ACCEPT,S (see 5.2.2).
ADD_GOS$	Initially set to 1. If the value of ADD_GOS$ is 0, the generated output stream will not be added (@ADD) regardless of the presence or absence of the B option on the SSG call line. If ADD_GOS$ is set to 1, the B option will control whether the generated output stream is added (@ADD) or not.
CASE$	Initially set to 0. When CASE$ is set to 0, the case of strings is left unchanged. When CASE$ is set to 1, all strings returned are converted to uppercase. This does not affect the internal representation of the string. When CASE$ is set to 2, all strings returned are converted to lowercase. This does not affect the internal representation of the string. If an LCSTR$ or UCSTR$ reference conflicts with the setting of the CASE$ variable, the setting of the CASE$ variable is used.
CASECHAR$	Initially set to 0. Determines the type of comparison performed by the ASTR$, BSTR$, and MSTR$ references when STRINGCHAR$ (ASTR$ and BSTR$ references) or STARTCHAR$ and STOPCHAR$ (MSTR$ reference) are alphabetic characters. When CASECHAR$ is set to 0, the case of the variable character must match the case of the string character (for example, 'A' does not match 'a'). When CASECHAR$ is nonzero, the cases of the variable character and the string character are ignored (for example, 'A' matches 'a').
COLED$	Column position of the next character to be output when edit mode is on. Ignored if not in edit mode. When edit mode is initiated, the value of COLED$ is set to 1 by SSG. Columns are numbered from left to right, starting at 1 (see 5.2.16).
CONFLC$	Initially set to 0. Counts the number of conflicts encountered while processing *CORRECT and *MERGE directives. The *BRKPT directive does not affect CONFLC$ as it does ERRCNT$.
CSFSTA$	Initially set to 0. Corresponds to the status returned by the Exec of the most recently fully interpreted *CSF directive (see 5.2.9).
EDITAB$	Initially set to 0. A nonzero value turns on recognition of tabs. Ignored if not in edit mode (see 5.2.16).

Table 3–9. SSG Reserved Global Variables

Name	Description
EOF$	Initially set to 0. If the E option is present on the *ACCEPT directive and a control statement is input, EOF$ is set to nonzero. If EOF$ is set to 1, an @EOF image was input (soft EOF). If EOF$ is set to 2, then either a non @EOF control image (hard EOF) was input between the SSG call and actual skeleton processing, or a hard EOF was input as the *ACCEPT response. If a noncontrol image was input, SSG will reset EOF$ to 0.
ERRCNT$	Initially set to 0. Counts the number of errors detected. SSG sets the value of ERRCNT$ to 0 each time a *BRKPT directive is encountered. If ERRCNT$ has a nonzero value and the B option is not on the @SSG call, a diagnostic is printed and SSG will not add (@ADD) the last part of the generated output stream.
FEATURE_LEVEL$	Set to the feature level for the currently executing SSG. See 3.2.3.5 and Table 3-11. Read only.
FLD$	When an *IF ROW SEARCH or *IF KEYWORD SEARCH is true, FLD$ equals the field number where the match was made (see 5.2.25.6).
GOSCNT$	Initially set to 0. Counts the number of images sent to the current part of the generated output stream. SSG sets the value of GOSCNT$ to 0 each time a *BRKPT directive is encountered (see 4.3). Read only.
IMGLEN$	Initially set to 80. Controls the width of the generated output stream. IMGLEN$ can be set from the skeleton to any value between 2 and 132 inclusive. IMGLEN$ can be manipulated any number of times during execution of the skeleton. If IMGLEN$ is set to a value less than 2 or greater than 132, SSG will adjust the value to 2 or 132 (see 4.3).
IOPT$	Initially set to 1 if the I option is present on the @SSG call. Otherwise, it is set to 0. Controls printing of the text of messages (*MESSAGE) and created SGSs (*ACCEPT, *ADD and *CREATE) in the SSG run log. For SSGs, the line number where the SGS is created is part of the displayed information.
LBRK$	Initially set to 0. When LBRK$ has a value of 0, a left bracket in a nondirective skeleton image is treated as the start of a reference. When LBRK$ has a nonzero value, a left bracket is treated as a normal character (see 4.3).
MOPT$	Initially 0 or 1, depending on the absence or presence of the M option on the OPT file specification. By setting MOPT$ to a 0 or nonzero value, you can turn off or on from the skeleton the effect of the M option (see Table 2–4).

Table 3–9. SSG Reserved Global Variables

Name	Description
NEW_BOOLEAN$	Initially set to 0. This variable controls the evaluation of Boolean expressions on *IF directives. Values can be • 0 = specifies traditional evaluation, where AND, OR, and XOR have the same priority. • Nonzero = specifies new evaluation, where AND has a higher priority than OR and XOR and where parentheses are allowed to change the order in which the Boolean operators are evaluated. Once a Boolean operator or a parenthesis is encountered on an *IF directive, this variable is changed to read-only. ***Note:*** *One type of Boolean evaluation must be selected and used throughout skeleton execution.* For more information, see 5.2.25.1.
NEW_PROCESS$	Initially 0, this variable controls whether *PROCESS parameters are handled under the old format or the new format. Once a *PROCESS directive is encountered, this variable is changed to "read only" to prevent mixed modes in the skeleton.
NOFND$	Initially set to 0. Counts the number of NO FIND diagnostics encountered. NOFND$ can be used to suppress NO FIND diagnostics by setting its value to NOFND$+100. (It is recommended that the value be decremented by the same amount prior to termination of skeleton processing.) The *BRKPT directive does not affect the value of NOFND$ as it does the value of ERRCNT$. The value of NOFND$ does not include NO FIND on *IF and *ELSEIF directives.
NOPT$	Initially 0 or 1 depending on the absence or presence of the N option on the OPT file specification. By setting NOPT$ to a 0 or nonzero value, you can turn off or on from the skeleton the effect of the N option (see Table 2–4). When NEW_BOOLEAN$ is nonzero, detailed *IF directive debug messages are output.
NSIGN$	Initially set to 0. Controls conversion of the NSIGNCHAR$ value to @ (see 4.3).
NSIGNCHAR$	Initially set to #. This string variable specifies the character that will be converted to an @ if NSIGN$ is clear.
PCFNAME$	Initially set to 0. A nonzero value turns on PCF set references (see 6.7).
PILO$	Initially set to 0. Controls conversion of the PILOCHAR$ value to * (see 4.3).
PILOCHAR$	Initially set to ". This string variable specifies the character that will be converted to an * if PILO$ is clear.
PROCESS_LEVEL$	Initially 0, this is set to the nesting level of process directives for new process handling ([#]).

Table 3–9. SSG Reserved Global Variables

Name	Description
PRTOFF$	Initially set to 0. Controls printing of the generated output stream. See 3.2.3.2 for a detailed description.
RTC$	Initially set to 1 if either the H option or *param-4* are specified on the @SSG call; otherwise, 0. By setting RTC$ to a 0 or nonzero value, you can turn off or on from the skeleton RTC images (see 5.2.7 and 6.10).
SFLD$	When an *IF ROW SEARCH or *IF KEYWORD SEARCH is true, SFLD$ equals the subfield number where the match was made (see 5.2.25.6).
SGSADDCNT$	Initially set to 0. Contains the number of SGSs created by the last *ADD statement executed.
SGSCNT$	Initially set to 0. Contains the current number of user-defined SGSs minus the number of automatically defined SGSs that have been removed with the *REMOVE directive.
SKELINE$	Reflects the revised skeleton line number at which the reference is made. A diagnostic aid. Read only.
SKELINECNT$	Cumulative count of the number of images in the revised skeleton stream that are interpreted. Comment images (*., see 4.5) are not interpreted. SKELINECNT$ can be used as a rough indicator of "work performed" by checkpointing its value across images in the skeleton. Read only.
SOPT$	Initially set to 1 if the S option is present on the @SSG call. Otherwise, set to 0. Controls the ordering of SGSs created with the *ACCEPT, *ADD, and *CREATE directives. See 3.2.3.3 for a detailed description.
SSGSGS$	Initially set to 9. Corresponds to the number of SGS labels defined by SSG (see 3.1.4).
STARTCHAR$	Initially set to a period (.). Used by the MIDDLE_STRING$ (MSTR$) string descriptor reference (see Table 3-2). If it is alphabetic, CASECHAR$ determines if the comparison is case sensitive or not.
STMT$	When an *IF COLUMN SEARCH is true, STMT$ equals the statement number where the match was made (see 5.2.25.6).
STOPCHAR$	Initially set to a period (.). Used by the MIDDLE_STRING$ (MSTR$) string descriptor reference (see Table 3-2). If it is alphabetic, CASECHAR$ determines if the comparison is case sensitive or not.
STRINGCHAR$	Initially set to a period (.). Used by the BEFORE_STRING$ (BSTR$) and AFTER_STRING$ (ASTR$) string descriptor references (see Table 3-2). If it is alphabetic, CASECHAR$ determines if the comparison is case sensitive or not.
TCFNAME$	Initially set to 1. A nonzero value turns on TCF set references (see 6.7).

Table 3–9. SSG Reserved Global Variables

Name	Description
UNS_CONTROL$	Initially set to -1 to disable unsolicited input. Set to 0 by the user to enable unsolicited input and to acknowledge an unsolicited input message. Set to 1 by SSG to notify the user that an unsolicited input message has been received and stored in UNS_INPUT$. See 3.2.3.4 for a detailed description.
UNS_INPUT$	Initially set to a null string. When an unsolicited input message is received, if unsolicited input is enabled (UNS_CONTROL$ set to 0), UNS_INPUT$ is set to the unsolicited input message text string. Read only. See 3.2.3.4 for a detailed description
UTC$	Initially set to 1 if the U extra option is specified. Otherwise, set to 0. Controls the time base used for the date/time values in the DATE$ and TIME$ SGSs. When set to 0 uses local time as the base; when set to a nonzero value uses UTC time as the base.
WARNINGCNT$	Initially 0, this counts the number of warning messages printed by SSG. Read only.
WARNINGFLG$	Initially 0, if set, turns off warning diagnostics for second pass type warnings.
ZOPT$	Initially 0 or 1 depending on the absence or presence of the Z option on the @SSG call. Reserved for site or user use (see Table 2–1).

3.2.3.1. String Descriptor Variables

To aid in reading the SGS string descriptors, you can use read-only variables in place of the fifth subfield number. These variables and their decimal values are shown in Table 3–10.

Table 3–10. Read-Only Variables

Full Name	Short Name	Value
SUB_STRING$	SUBSTR$	0
ALPHA_LENGTH$	ALEN$	1
QUOTE_LENGTH$	QLEN$	2
DQUOTE_LENGTH$	DQLEN$	3
NUMERIC_LENGTH$	NLEN$	4
LENGTH$	LEN$	5
SGS_STRING$	SGSSTR$	6
LOWER_CASE_LENGTH$	LCLEN$	7
UPPER_CASE_LENGTH$	UCLEN$	8
ALPHA_NUMERIC_LENGTH$	ANLEN$	9
SPECIAL_CHAR_LENGTH$	SCLEN$	10
LOWER_CASE_STRING$	LCSTR$	11
UPPER_CASE_STRING$	UCSTR$	12
BEFORE_STRING$	BSTR$	13
AFTER_STRING$	ASTR$	14
MIDDLE_STRING$	MSTR$	15
LEFT_STRING$	LSTR$	16
RIGHT_STRING$	RSTR$	17
PSGS_STRING$	PSGSTR$	18

3.2.3.2. PRTOFF$

Images of the generated output stream are listed. The reserved global variable PRTOFF$ is used to dynamically control these images. Initially, PRTOFF$ has a value of 0.

When PRTOFF$ has a nonzero value, images sent to the generated output stream are flagged not to be listed by SSG. The value of PRTOFF$ can be set to 0 or nonzero any number of times during skeleton execution. Even though the listing of images is suppressed, the images are still in the generated output stream.

If you set PRTOFF$ during skeleton execution and generated images are being listed, SSG prints the following message in the generated output stream listing:

```
**GENERATED OUTPUT STREAM LISTING SUSPENDED**
```

3.2.3.3. SOPT$

SOPT$ controls the ordering of SGSs created with the *ACCEPT, *ADD, and *CREATE directives.

Using the S and X Options to Initially Control SGS Ordering

The @SSG call line S and X options (see Table 2–1) control the ordering of SGSs created from the *param-2* and *param-7* parameters (see 2.1.2) and SGS file-id statements (see 2.2).

- If the S option is not present, all SGSs with the same label are added to the end of the chain of SGSs.
- If the S option is present, all SGSs with the same label are added to the chain of SGSs in ascending order, based on the content of the first field, first subfield.

When skeleton execution begins, the @SSG call line S option is transferred to the reserved global variable SOPT$.

- If the S option is not present, SOPT$ has an initial value of 0.
- If the S option is present, SOPT$ has an initial value of 1.

Modifying the Value of SOPT$ to Dynamically Control SGS Ordering

The ordering of SGSs created by the *ACCEPT, *ADD, and *CREATE directives is controlled by the value of SOPT$. You can control the ordering by modifying this variable during skeleton execution.

- If SOPT$ is zero, all SGSs with the same label are added to the end of the chain of SGSs.
- If SOPT$ is nonzero, SSG attempts to add SGSs with the same label to the chain of SGSs in ascending order based on the contents of the first field, first subfield.

Adding New SGSs in Ascending Order

The following are considerations when SOPT$ has a nonzero value and SGSs are added:

- The SGSs for the label being added must be in ascending order so the new SGSs can be added in the properly sorted order. The *SORT directive (see 5.2.40) may have to be used first to order the current SGSs based on the first field, first subfield and then again to reorder all the SGSs as you wish after the new ones have been added.
- If the SGSs for the label being added are not in ascending order, the order in which the new SGSs are added is unpredictable. The following conditions could cause the SSGs to be out of order:
  - The S option was not specified when the SGSs were created from @SSG call line parameters and SGS file-id statements.
  - A *SORT directive has reordered the SGSs in descending order or based on a subfield other than the first field, first subfield.
  - An *ACCEPT, *ADD, or *CREATE directive added at least one SGS to a sorted chain of SGSs when SOPT$ had a value of 0.
  - A *RESTORE directive (see 5.2.36) has restored SGSs with the same label.

  Note: *SGSs added with the *RESTORE directive are always added to the end of the chain of SGSs with the same label. The SGSs are restored in the same order in which they were saved by the *SAVE directive.*

3.2.3.4. UNS_CONTROL$ and UNS_INPUT$

Skeletons can be designed to accept and process unsolicited input. Up to six characters of unsolicited console operator input can be passed to the skeleton with the 'll *run-id data*' keyin. Up to four characters of unsolicited demand user input can be passed to the skeleton with the '@@X C,*data*' keyin. To assure compatibility with SSG 24R1 and earlier levels, this capability is disabled until your skeleton explicitly enables it.

This capability may be particularly useful if you have SSG skeletons that execute for an extended time period. Examples include unsolicited requests to display the current skeleton processing status and unsolicited requests for an early termination of skeleton processing. Appendix C.6.4 contains a skeleton example that illustrates both of these usages.

This capability is controlled by the reserved global variables UNS_CONTROL$ and UNS_INPUT$. UNS_CONTROL$ has an initial value of -1, which disables unsolicited input. If unsolicited input messages are received, they are ignored, as was the case with SSG 24R1 and earlier levels. If you want a skeleton to accept unsolicited input, it should perform the actions described below.

During skeleton initialization unsolicited input is enabled by setting UNS_CONTROL$ to 0, as follows:

```
*CLEAR UNS_CONTROL$
```

Periodically during execution, the skeleton should check if UNS_CONTROL$ is set to 1, indicating that unsolicited input data has been received and stored in the variable UNS_INPUT$. This is the optional data that is entered on 'll *run-id data*' or on '@@X C,*data*'. If the console operator or demand user does not include *data* on their keyin, then UNS_INPUT$ is set to a null string. The following illustrates a possible UNS_CONTROL$ check:

```
*IF UNS_CONTROL$ IS SET
*. ... Process unsolicited input in UNS_INPUT$
*ENDIF
```

Once UNS_INPUT$ is processed, the skeleton should set UNS_CONTROL$ to 0 to acknowledge that processing of UNS_INPUT$ is complete and to enable the next unsolicited input. Since unsolicited input is asynchronous to skeleton execution, when UNS_CONTROL$ is set to 0, UNS_INPUT$ may have new unsolicited input text as soon as the next skeleton directive is processed. To avoid the possibility of UNS_INPUT$ being overlaid with new unsolicited input text, one of the following two methods is recommended:

```
*IF UNS_CONTROL$ IS SET
*. ... Process unsolicited input in UNS_INPUT$
*. ... New unsolicited input in UNS_INPUT$ IS NOT allowed
*CLEAR UNS_CONTROL$
*ENDIF
```

or

```
*INCREMENT i TO 9999999 WHILE UNS_CONTROL$ IS SET
*SET save_input = UNS_INPUT$ . Save the unsolicited input
*CLEAR UNS_CONTROL$
*. ... Process unsolicited input in save_input
*. ... New unsolicited input in UNS_INPUT$ IS allowed
*LOOP
```

In the second method, the user variable name 'save_input' can be any name desired.

If unsolicited input is received while UNS_CONTROL$ is set to 1, the following message is sent to the console operator or demand user:

```
PREVIOUS UNSOLICITED INPUT NOT PROCESSED - THIS UNSOLICITED INPUT IGNORED
```

If the unsolicited input was entered by a console operator, the above message is sent to the default system console, which may not be the console from which the entry was made. If the unsolicited input was entered by a demand user and if PRINT$ is breakpointed to a file, the above message is written to the print file and the demand user will not see it.

If the skeleton has a main *INCREMENT or *DO loop, the top of the loop would be an ideal place to either check for unsolicited input or to call a define subroutine to perform this check.

3.2.3.5. FEATURE_LEVEL$

FEATURE_LEVEL$ is a read only variable that can be used to identify the SSG features supported for the currently executing SSG. You can test this variable with the *IF directive when you need to determine if a particular feature listed in Table 3-11 is supported by the currently executing SSG. SSG features implemented prior to level 24R3 do not appear in the table.

Table 3–11. FEATURE_LEVEL$ Features

Value	SSG Level	Feature Description
1	24R3	• FEATURE_LEVEL$ reserved global variable is implemented to identify supported features. See Table 3-9 and 3.2.3.5. • System technical architecture is available in automatically defined SGS subfield [SYSTEM$,1,3,3]. See 3.1.4.5. • Additional SSG options can be specified on the SSG call line *param-7* as OPT/*extra-options*. See 2.1.2. • Automatically defined DATE$, DATIME$, and TIME$ SGSs can be based on local time or UTC time as specified by the U extra option (see Table 2-4) and by the UTC$ reserved global variable (see Table 3-9). • New date/time formats are available in automatically defined SGS subfields, as follows: – DATIME$ SGS subfields [DATIME$,1,2,2], [DATIME$,1,2,3], [DATIME$,1,9,1], [DATIME$,1,10,1], [DATIME$,1,10,2], and [DATIME$,1,10,3]. See 3.1.4.1. – TIME$ SGS subfields [TIME$,1,1,2], [TIME$,1,1,3], [TIME$,1,3,1], [TIME$,1,4,1], [TIME$,1,4,2], and [TIME$,1,4,3]. See 3.1.4.7. – DATE$ SGS subfield [DATE$,1,7,1]. See 3.1.4.9.
2	24R4	• Long format TOC SGS date and time information can be based on the Reversed-TDATE$ or Modified-SWTIME format timestamp stored in program file elements. See 3.1.1. • New date and time formats are available in long format TOC SGS field 5. The new TOC SGS subfields are [*label,n,*5,4], [*label,n,*5,5], and [*label,n,*5,6]. See 3.1.1. • Long format TOC SGSs date and time information can be based on local time or UTC time as specified by the SGS U option (see Table 2-3 and 3.1.1).

The following example illustrates the use of FEATURE_LEVEL$:

```
*IF +FEATURE_LEVEL$ >= 1
*DISPLAY 'Date/time will be based on UTC time'
*SET UTC$
*ELSE
*DISPLAY 'UTC time NOT available'
*RETURN STOP . Exit the skeleton
*ENDIF
```

Note: *"+" or "VALUE OF" must precede "FEATURE_LEVEL$" to cause a numeric comparison.*

For SSG levels prior to 24R3, the variable FEATURE_LEVEL$ is undefined. When it is used in an *IF directive simple condition, a result of false is always returned (see 5.2.25.1). For example, "*IF +FEATURE_LEVEL$ = 0" and "*IF +FEATURE_LEVEL$ <> 0" will both evaluate as false for SSG levels prior to 24R3.

3.3. Process Parameter References

You can specify parameters to a define packet on the *PROCESS directive line. You can reference these parameters from within a define packet by using two different methods. These two methods are distinguished by the value of the global variable NEW_PROCESS$.

Method 1 is the default method used to reference the process parameter values. Method 2 allows process parameters to be referenced by using the much more powerful and flexible SGS mechanism. Use of all the SGS string descriptor references and access to the process parameter values from all nested *PROCESS calls is also allowed.

Method 1

The value of NEW_PROCESS$ is 0.

When the *PROCESS directive is interpreted, parameter expressions and references are evaluated and converted to their symbolic representations. If an SGS reference or set reference evaluates to a NO FIND, and the A option is specified on the @SSG call line, then a NO FIND process parameter is created. A NO FIND process parameter reference results in a NO FIND condition.

Note: *Make process parameter references from within a define packet only. You can reference parameters from nested calls to the same or other define packets. After an exit from a define packet, you can no longer reference process parameters for that specific call.*

Process parameter references have one of the formats shown in Table 3–12.

Table 3–12. Process Parameter References

Process Parameter Reference	Value Returned
[#]	Number of process parameters specified on the most recently fully interpreted *PROCESS directive. This count includes NO FIND process parameters. (NUMERIC value)
[#*n*]	The *n th* process parameter specified on the most recently interpreted *PROCESS directive. (A NO FIND parameter returns a NO FIND condition; otherwise, a SYMBOLIC value is returned.)
[#,*define-name*]	The number of process parameters specified on the most recently interpreted *PROCESS directive with the *define-name*. Applies only before exiting the define packet with the specified *define-name*. (NUMERIC value)
[#*n*,*define-name*]	The *n th* process parameter specified on the most recently interpreted *PROCESS directive with the specified *define-name*. Applies only before an exit from the define packet with the specified *define-name*. (A NO FIND parameter returns a NO FIND condition. Otherwise, a SYMBOLIC value is returned.)

The following rules apply when composing method 1 process parameter references:

- The *n* field can be a string of numeric characters or a variable name. Bracketed references cannot be used to supply *n*.
- The *define-name* field can be a string or a bracketed SGS, variable or set reference that returns a string. See 3.4 for a restriction on using a process parameter reference to supply *define-name*.
- A comma in the reference may be followed by one or more spaces.
- A comma or space in the reference may be followed by a semicolon to indicate that the reference is continued on the next sequential image. Any characters after the semicolon are ignored. Leading spaces on the next image are ignored.

Method 2

The value of NEW_PROCESS$ is nonzero. To use this method, set NEW_PROCESS$ to 1. This variable cannot be changed after the first *PROCESS directive is encountered.

When the *PROCESS directive is interpreted, bracketed references within the process parameters are resolved (if LBRK$ is clear) and converted to their symbolic representations. Numeric expressions are not evaluated, but are read as symbolic characters.

References to process parameters are treated like SGSs with a label of #. If you nest *PROCESS directives within define packets, then additional pseudo-SGSs are created and include the label #. References to parameters from these nested calls are referenced like multiple SGSs with the same label. Each pseudo-SGS statement contains the parameters for one call to a define packet, created in the order that the *PROCESS directives were issued. After an exit from a define packet, the pseudo-SGS statement containing the parameters for that call is removed, as with the *REMOVE SGS directive.

Note: *Make process parameter references from within a define packet only. You can reference parameters from nested calls to the same or other define packets. After an exit has been made from a define packet, you can no longer reference the process parameters for that specific call.*

To reference the parameters for the current define packet, use the construct [#,[#],*f*,*s*]. To aid in reading the skeleton, a global variable has been defined that has the same value as [#]: PROCESS_LEVEL$. Thus the reference [#,PROCESS_LEVEL$,2,1] is equivalent to [#,[#],2,1]. For a description of parameter referencing, see 3.1.2 on SGS referencing. All SGS string descriptor references are also valid process parameter references.

The rules for composing method 2 process parameter references are identical to the rules for composing SGS references described in 3.1.2.4.

Note: *When it is stated in this document that a parameter can be supplied by a process parameter reference, this applies to the references in Table 3–12 that are valid when NEW_PROCESS$ is zero. When NEW_PROCESS$ is nonzero, process parameter references are valid only in contexts where SGS references are valid.*

Example

The following skeleton

```
*DEFINE Print
#SYM,U [#,PROCESS_LEVEL$,1,1].,[#,PROCESS_LEVEL$,2,2],PR
*ENDDEFINE
*SET NEW_PROCESS$
*PROCESS Print qual*file copies,5
```

results in

```
@SYM,U qual*file.,5,PR
```

3.4. Bracketed Reference Restrictions

SSG recognizes four types of bracketed references:

- SGS reference: [*lab,ne,ne,ne,ne,ne,ne*]—see 3.1
- PCF or TCF set reference: [*set,ne,ne,ne*]—see 6.7
- Process parameter reference method 1: [#*n,str*]—see 3.3
- Process parameter reference method 2: [#,*ne,ne,ne,ne,ne,ne*]—see 3.3
- Variable reference method 1: [**variable-name*]—see 3.2

where:

lab

is an SGS label.

ne

is an integer expression or a numeric expression.

set

is a set name.

#n

is an integer denoting the desired process parameter.

str

is a string representing a subroutine name.

variable-name

is the name of a variable.

A nested bracketed reference occurs whenever the *lab*, *ne*, *set*, or *str* are themselves supplied by a bracketed reference.

Note: *A bracketed reference cannot be supplied for* #n *or* variable-name.

The following example illustrates nested bracketed references:

```
*CREATE SGS:  LAB LAB14
*SET F = 3
*SET S = 1
*DISPLAY '[[LAB,1,1,1],1,[*F],[*S]] is the same as [LAB14,1,3,1]'
```

Different SSG reference types have different rules as to what type of nested references they can contain and when they can be used as nested references. These reference types are as follows:

- Variable references
  - *variable-name* may not be supplied by a bracketed reference.
  - Bracketed variable references (method 1) may be used to supply *lab*, *ne*, *set*, or *str* and are acted on immediately when *variable-name* is defined.
  - Unbracketed variable references (method 2) may be used to supply *ne*.
- Process parameter references (method 1)

  Generally, a simple process parameter reference such as [#*n*] with no *str* can be placed anywhere and correctly handled. When the process parameter reference has a *str* specification, nesting breaks down. The following references cause a format error:

```
*DEFINE X
PARAM IS [#1,[#1,X]]
*END
*PROCESS X X
```

  If the reference [#1,[#1]] is used, it works.

- Process parameter (method 2), SGS, and set references

  SGS and set references cause different problems. The *lab* and *set* fields can be supplied by a bracketed reference that returns a symbolic value.

  For components past the first, if *ne* contains an SGS reference or set reference, that SGS reference or set reference must be the first supplied addend. In addition, no more than one nested SGS reference or set reference is permitted per component.

  The nested SGS reference or set reference can never contain another SGS reference or set reference.

Table 3–13 illustrates the possible problems.

Table 3–13. Nested Bracketed Reference Restrictions

Reference Example	Analysis of This Reference
[X,A+B]	No problems—There are no nested, bracketed references.
[X,6-[Y]]	Problems—The nested SGS reference is not the first addend in its component. Switch the order of evaluation so the nested SGS reference is the first addend in its component, as in the following example: [X,-[Y]+6]
[X,-[Y]+[Z]]	Problems—There are nested SGS references, one of which is not the first addend in its component. Save the value of [Z] in a variable, for example V, and use the following reference: [X,-[Y]+V]
[X,[Y,[TCF,3,2,1,5]]]	Problems—A set reference supplies a component of an already nested SGS reference. The nest is too deep. Save the value [TCF,3,2,1,5] in a variable, for example V, and use the following reference: [X,[Y,V]]

- String concatenation using bracketed references

  It is often desirable to concatenate bracketed references and strings to form longer strings. The object of the following example is to create the string 'LAB3B':

```
*SET NUM = 3
*SET ALPHA = 'B'
... LAB[*NUM][*ALPHA] ...
```

  Whether or not the third line will produce the desired string 'LAB3B' depends on the context of the references.

  Contexts that will produce 'LAB3B':

  – A nondirective image—see 4.3
  – A directive image that is scanned twice (these directives are marked with ←)—see 4.4
  – As part of the *CREATE directive *sgs-image*—see 5.2.2.8
  – As process parameters on the *PROCESS directive when NEW_PROCESS$ is nonzero—see 5.2.34

Contexts that might produce a format error or unexpected results:

– A directive image that is not scanned twice

– A nested reference (this construct cannot be used to supply *lab*, *ne*, *set*, and *str*)

Examples

The following examples illustrate various string concatenation contexts.

– Examples of contexts where the concatenated string 'LAB3B' will be processed as desired:

```
#ASG,T TAPE,T,LAB[*NUM][*ALPHA]
*ACCEPT LAB[*NUM][*ALPHA] 'Enter value'
*SET LAB[*NUM][*ALPHA] TO 361
*CREATE SGS: LAB[*NUM][*ALPHA] Ver,[*NUM] Rev,[*ALPHA]
*SET NEW_PROCESS$
*PROCESS SUB7 LAB[*NUM][*ALPHA]
```

– Examples of contexts where the concatenated string 'LAB3B' will not be produced:

```
*CLEAR NEW_PROCESS$
*PROCESS SUB7 LAB[*NUM][*ALPHA]
*REMOVE SGS LAB[*NUM][*ALPHA]
#ASG,T TAPE,T,[LAB[*NUM][*ALPHA],1,1,1]
```

Section 4
Skeleton and Symstream

This section describes Symstream, the language used to write programs to be interpreted by the SSG processor.

4.1. Introduction

A call to the SSG processor causes the interpretive execution of a program, commonly called a skeleton, which is written in Symstream. Table 4–1 contains the Symstream definitions.

Table 4–1. Symstream Definitions

Term	Definition
character	Any ASCII or Fieldata character.
alphabetic characters	A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z (as well as lowercase).
numeric characters	0,1,2,3,4,5,6,7,8,9 .
special characters	Space ( ), period (.), comma (,), plus sign (+), minus sign (-), right bracket ( ]), left bracket ([ ), slash (/), equal sign (=), greater than sign (>), less than sign (<), semicolon (;), and asterisk (*). The first six special characters are break characters that should be used only in the proper context, because they might trigger a numeric evaluation. The following are additional break characters on an *IF directive: • The slash (/), greater than (>), less than (<), and equals sign (=) • The left and right parentheses ( ) only if the reserved global variable NEW_BOOLEAN$ has a nonzero value. Otherwise, they are not special characters. A leading left bracket ([ ) is always a break character (unless LBRK$ is set). A semicolon (;) is not a break character. Depending on the usage context it may signify continuation on the next image.

Table 4–1. Symstream Definitions

Term	Definition
string	Any collection of non-special characters; any collection of characters enclosed in single quotation marks ('...'); or any collection of characters enclosed in pairs of single quotation marks (''...''). A string is limited in size to 128 characters.
number	-34359738367,...,-1,0,1,...,34359738367.
H'hexadecimal number'	Same as number but base 16. Each character belongs to the set 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F.
O'octal number'	Same as number but base 8 and digits from the set 0,1,2,3,4,5,6,7.
B'binary number'	Same as number but base 2. Each character belongs to the set 0,1
integer expression	String of numeric characters Variable name Bracketed reference that returns a string of numeric characters or a variable name An integer expression is symbolic—it does not have a numeric value unless more information is supplied (for example, the context in which it is used).
numeric expression	Bracketed reference that returns a NUMERIC value ± integer expression ± numeric expression Integer expression ± integer expression ± ... Integer expression ± numeric expression ± ... Numeric expression ± integer expression ± ... Numeric expression ± numeric expression ± ... ± integer expression ± ... ± numeric expression ± ...
arithmetic expression	Integer expression Numeric expression Arithmetic expression * arithmetic expression Arithmetic expression / arithmetic expression where the result is a truncated integer value

Table 4–2 contains examples of Symstream definitions.

Table 4–2. Symstream Definition Examples

Expression	Type of Expression
12	Integer (string of numeric characters)
VAR	Integer (variable name)
[LAB,1,2,6]	Integer (bracketed reference that returns a variable name or number)
[#7]	Integer (bracketed reference that returns a variable name or number)
-146	Numeric (± integer expression)
[LAB]	Numeric (bracketed reference that returns a NUMERIC value)
[*VAR]	Numeric or string (bracketed reference that returns a variable name or number)
[TCF]	Numeric (bracketed reference that returns a NUMERIC value)
A+B-[#C]	Numeric (integer expression ± integer expression ± integer expression)

4.2. Element and Version Name Restrictions

The Exec allows the minus character (-) in element names and version names, but this is not always allowed by SSG. Minus characters are allowed in these situations:

- When an input correction stream is read and an **element-name/version-name* image is found, a minus sign is treated correctly as a legitimate character.
- When a *CREATE PERM (or TEMP) directive is encountered, a minus sign is also handled correctly.
- When a *MERGE, *CORRECT, *REMOVE, or *IF element-name/version-name HAS COR directive is analyzed, the minus sign acts as a break character, and is treated as a field separator, not as a valid character.

  For example, the following line causes a format error:

```
*IF a/b-c HAS CORRECTIONS
```

  SSG recognizes the *IF and reads the next field. The slash (/) acts as a break character, so the field is "a." Since the field terminator was a slash, SSG knows this is a HAS COR type *IF clause and reads the next field, the version, which returns "b." Note that the minus sign acts as a break character. The terminator is not checked. The next field is read and returns "c."

4.3. Nondirective Images

Any image in the revised skeleton stream that does not have an asterisk in the first character position, column 1, is a nondirective image. All nondirective images encountered in a skeleton are sent to the generated output stream, unless encountered between *CORRECT and its matching *ENDCORRECT.

The generated output stream may be listed (K option), inhibited from being dynamically added by SSG (B option), or output to a file or element (see Table 2–1). You can specify the file or element as *param-3* on the @SSG call, or as *name* on a *BRKPT directive.

You can create nondirective images by using any combination of characters, SGS references, process parameter references, variable references, and set references. These references are distinguished from simple character strings by the brackets around them. All references are satisfied before SSG prints and outputs the image, unless LBRK$ is set.

When LBRK$ is nonzero, SSG treats a left bracket ([ ) as a normal character and transfers it and any right brackets ( ]) to the generated output stream, rather than treating it as the beginning of a reference.

LBRK$ has a default value of 0, and can be modified any number of times from within the skeleton.

You can also use the *EDIT directive to create nondirective images (see 5.2.16).

SSG increments the value of the reserved global variable GOSCNT$ by 1 for each nondirective image sent to the generated output stream (see Table 3–9).

If a nondirective image exceeds the maximum number of characters defined by the value of IMGLEN$, SSG inserts a semicolon (continuation character) in the last column position (determined by IMGLEN$) and continues the line on the next image. Each continuation line increments the value of GOSCNT$ by 1.

A semicolon specified in any character position of a nondirective image appears as output in the generated output stream. SSG does not interpret a semicolon as a continuation character in a nondirective line.

If you specify a PILOCHAR$ character in the first character position of a nondirective image, and the reserved global variable PILO$ has a value of 0, the PILOCHAR$ character is converted to an *. If PILO$ has a nonzero value, there is no conversion.

If you specify an NSIGNCHAR$ character in the first character position of a nondirective image, and the reserved global variable NSIGN$ has a value of 0, the NSIGNCHAR$ character is converted to an @. If NSIGN$ has a nonzero value, there is no conversion.

PILO$ and NSIGN$ have a default value of 0. You can modify them any number of times from within the skeleton. The initial value of NSIGNCHAR$ is #; for PILOCHAR$ it is double quotes (").

Before resolving any references, SSG checks column 1 of nondirective images for possible conversion of PILOCHAR$ and NSIGNCHAR$ characters.

Example

This example uses the following skeleton lines:

```
1.  *SET pilovar = '"'
2.  [*pilovar] nondirective line
```

These skeleton lines result in the following output:

```
" nondirective line
```

The character in column 1 of line 2 in the skeleton is ([ ), which is not the PILOCHAR$ character (") or the NSIGNCHAR$ character (#), resulting in no conversion. The [*pilovar] reference is then resolved, and the nondirective line is added to the generated output stream.

4.4. Directive Images

An asterisk (*) in the first character position of a skeleton image defines that image as a directive. Directives are used to control the flow of processing and manipulate images in the skeleton. You can change the directive signifying character by using the construct

```
OLD/NEW
```

The new character can be any valid ASCII character, including a space.

Example

```
*/-
-SET A = 12
-/?
?DISPLAY A
```

The Symstream directives are shown below and are also described in detail in Section 5.

*ABORT	*DEFINE	*ENDCORRECT	*LIST	*REMOVE
*ACCEPT←	*DISPLAY←	*ENDDEFINE	*LOOP	*RESTORE←
*ADD←	*DIVIDE	*ENDDO		*RETURN←
	*DLOC←	*ENDIF	*MERGE	
*BRKPT←	*DO	*EXIT←	*MESSAGE←	*SAVE←
	*DUMP		*MULTIPLY	*SET←
*CLEAR←		*IF		*SORT←
*COPY	*EDIT	*INCREMENT	*PCONTROL←	
*CORRECT	*EJECT	*ITERATE←	*PROCESS	*UNLIST
*CREATE	*ELSE			
*CSF←	*ELSEIF			*WAIT←

Directive images marked with ← are scanned twice. The first scan satisfies all bracketed references, resulting in a new image that cannot be greater than 132 characters in length. The new image is then treated as if it were the original image, except that bracketed references are ignored. Such directive images require text composition (as described in the following text).

A directive that requires text composition can be thought of as a two-phase process:

1. If LBRK$ is clear, bracketed references are satisfied. Any number of bracketed references may be included. Any bracketed reference that allows continuation may be continued on the next image. Semicolons outside of bracketed references are ignored in this phase. Once all bracketed references are satisfied, the result is a single image that cannot exceed 132 characters. If LBRK$ is set, all bracketed references are ignored and all semicolons (including any in bracketed references) are ignored and the original image is processed in the next phase.
2. The specific directive syntax of the phase 1 image is analyzed, as described in the subsections of 5.2. Any left brackets in the image are treated as normal characters and bracketed references are not satisfied. If the image contains a semicolon character, the rules for the specific directive determine if it is treated as a continuation character. If the directive allows continuation, the next sequential image is read and phases 1 and 2 are repeated for the continued image.

For easier reading, you can place spaces in a directive image between the asterisk and directive name. Every image in the skeleton that has an asterisk in the first character position must be one of the preceding directives, or have a period following the asterisk. A period anywhere on a directive image causes SSG to terminate the scan of that image unless the period is in a quoted string. (For other exceptions, see *BRKPT, *CSF, *ADD, *CREATE, *COPY, *EDIT, and *MESSAGE.)

The following rules apply when composing directives:

- The directive itself must be a string.
- Most of the subsections of 5.2 contain 'Parameters' topics that describe the valid types of references allowed for each field of the directive.
- Wherever a space is shown in the directive format, any number of spaces may be specified.
- Any comma in the directive may be followed by one or more spaces.
- In addition to space and comma, some directives allow +, -, /, *, <, =, >, (and) to act as field-ending break characters. Any allowed break character may be followed by one or more spaces.
- Continuation of a directive is not allowed until after the directive name, directive options and a space character.
- If the directive allows continuation, any comma, space or allowed break character may be followed by a semicolon to indicate that the directive is continued on the next sequential image. Any characters after the semicolon are ignored. Leading spaces on the next image are ignored.

- Specific directives may document exceptions to the above general continuation rule.
- If a format error occurs before the semicolon is encountered, the semicolon will be ignored. The next sequential image will be treated as a nondirective image.
- If SSG stops scanning a directive because all required and optional fields have been processed or because a period has terminated the scan, a following semicolon will be ignored. The next sequential image will be treated as a nondirective image.
- Bracketed references on directive images have their own composition rules for fields, spaces and continuation. See 3.1.2.4 (SGSs), 3.2.2.1 (variables), 3.3 (process parameters) and 6.7 (PCF and TCF sets).

4.5. Defining Skeleton Image Sequences (Closed Subroutines)

The skeleton is partitioned into two sections: the define section and the control section.

Location and Contents of the Define Section

The define section must be first in all skeletons. It contains any number of sequences of skeleton images (directive, nondirective, or both) that are placed in blocks delimited by a *DEFINE directive and a matching *ENDDEFINE directive. Each block of skeleton images, referred to as a define packet, is treated by SSG as a closed subroutine and stored for later use.

Beginning of the Control Section

Any skeleton image encountered signals the beginning of a control section if

- It is not in a define packet.
- It is not a comment (*.).
- It is not an *EJECT directive without the K option specified.
- It is not a *COPY directive.

That skeleton image is the first image in the control section.

Note: *A *DEFINE directive encountered in the control section or a define packet is in error.*

Retrieving and Processing Define Packets

You can retrieve and process define packets with the *PROCESS directive. A *PROCESS directive can appear in the control section or within a define packet.

Size Limitations

- There is no limit on the number of define packets allowed.
- Even though the *number* of define packets is not limited, the *size* of each define packet is limited to approximately octal 0600000 or decimal 197,000 words. Assuming an average skeleton image length of 20 characters, this limit is approximately 25,000 images. If a define packet exceeds this limit, the following error messages are output:

```
*ERROR* - DEFINE PACKET IS TOO LARGE TO PROCESS
*ERROR* - SIZE CAN BE REDUCED BY MOVING PARTS OF DEFINE
*ERROR* - PACKET INTO SMALLER DEFINE PACKETS - SEE SSG PRM (7831 1784)
*ERROR* - ERROR DETECTED ON LINE m , ON OR BEFORE COLUMN n
```

- The size of the control section has the same size limitations as a define packet. If it exceeds the limit, the following error messages are output:

```
*ERROR* - MAIN SKELETON IS TOO LARGE TO PROCESS
*ERROR* - SIZE CAN BE REDUCED BY MOVING PARTS OF MAIN
*ERROR* - SKELETON INTO DEFINE PACKETS - SEE SSG PRM (7831 1784)
*ERROR* - ERROR DETECTED ON LINE m , ON OR BEFORE COLUMN n
```

In both cases, the skeleton image on which the limit was exceeded is output after these messages.

Section 5
SSG Directives

This section describes the SSG directives and explains their use.

5.1. Logical Directive Groups

There are nine groups of Symstream directives. The following subsections describe these groups and list the directives in each.

5.1.1. Skeleton Creation

These directives define the way you create a skeleton. They are evaluated on the first pass of the skeleton and cannot be controlled by any of the logic control directives.

*DEFINE and *ENDDEFINE	See 5.2.10 and 5.2.21
*COPY	See 5.2.6

5.1.2. Skeleton Listing Control

These directives affect the revised skeleton listing only. They are also evaluated on the first pass and cannot be controlled by logic control directives.

*EJECT	See 5.2.17
*LIST	See 5.2.28
*UNLIST	See 5.2.41

5.1.3. Skeleton Logic Control

These directives control the processing flow within the skeleton. Using these directives, you can make decisions about directives to interpret, images to output, and the order in which images are processed.

*IF, *ELSE, *ELSEIF, and *ENDIF	See 5.2.25, 5.2.18, 5.2.19, and 5.2.23
*INCREMENT, *LOOP	See 5.2.26
*DO, *ENDDO	See 5.2.14
*PROCESS	See 5.2.34
*ITERATE	See 5.2.27
*EXIT	See 5.2.24
*RETURN	See 5.2.37
*ABORT	See 5.2.1

5.1.4. Variable Modification

These directives allow you to create, modify, and remove both numeric and string variables. For a description of variable referencing, see 3.2.

*SET	See 5.2.39
*CLEAR	See 5.2.5
*INCREMENT, *LOOP	See 5.2.26
*DO, *ENDDO	See 5.2.14
*MULTIPLY	See 5.2.32
*DIVIDE	See 5.2.12
*REMOVE	See 5.2.35
*DUMP	See 5.2.15

5.1.5. Skeleton Communication

These directives allow the skeleton to communicate with the run during skeleton processing. You can use them to print messages, obtain input, and perform print control.

*ACCEPT	See 5.2.2
*DISPLAY	See 5.2.11
*PCONTROL	See 5.2.33
*MESSAGE	See 5.2.31

5.1.6. Skeleton Output Control

These directives control the generated output stream.

*BRKPT	See 5.2.4
*EDIT	See 5.2.16

5.1.7. SGS Modification

These directives create, remove, and sort SGSs.

*CREATE	See 5.2.8
*ADD	See 5.2.3
*REMOVE	See 5.2.35
*SORT	See 5.2.40
*SAVE	See 5.2.38
*RESTORE	See 5.2.36

5.1.8. PCF/TCF Modification

These directives manage permanent correction file/temporary correction file (PCF/TCF) sets and their associated entries.

*CORRECT, *ENDCORRECT	See 5.2.7 and 5.2.20
*CREATE	See 5.2.8
*MERGE	See 5.2.30
*REMOVE	See 5.2.35
*SORT	See 5.2.40

5.1.9. Miscellaneous

These directives are used to submit control statement images, allow use of secure privileges, and allow delays in skeleton processing.

*CSF	See 5.2.9
*DLOC	See 5.2.13
*WAIT	See 5.2.42

5.2. Directive Descriptions

The following alphabetically ordered subsections describe the directives in detail.

5.2.1. *ABORT

The *ABORT directive causes SSG to terminate skeleton processing.

Format

```
*ABORT
```

Description

The *ABORT directive causes abnormal termination of the skeleton in execution. The line number and the source of the skeleton image (SKEL$ or define packet name) where the *ABORT was encountered are displayed. For example, if the *ABORT is encountered at line 25 while processing the main skeleton, the following message is displayed:

```
**** ABORT ENCOUNTERED AT LINE 25 OF SKEL$
```

At program termination, bit 27 of the condition word is set.

5.2.2. *ACCEPT

The *ACCEPT directive allows SSG to function interactively.

Format 1

*ACCEPT(,*options*) *variable-name val* (*val val*...)

Format 2

*ACCEPT,S(*options*) *val* (*val val*...)

Format 3

*ACCEPT,L(*options*) *label val* (*val val*...)

Parameters

options

B Performs a PRINT followed by a READ rather than a TREAD.

E Allows control images (images with @ as the first character) to be input. If a control image is input, EOF$ will be set to 1 or 2.

L Creates a stream generation statement (SGS) with the specified *label*. The input becomes the fields and subfields.

N Specifies that bracketed references on the input image are not to be satisfied.

O Solicits input from the operator console

S Creates an SGS from the input

variable-name

The name of the string variable generated from the *ACCEPT input.

label

The label of the SGS generated from the *ACCEPT input.

val

Message printed before the read. The parameter *val* can be a string variable name, a numeric variable name, or a quoted string.

Description

The *ACCEPT directive allows the SSG skeleton to obtain input from you and from the operator.

- With no options, SSG creates a global string variable from the input.
- The O option sends the message text to the operator and allows the operator to provide the input.
- By using either the S or L options, you can create an SGS from the input.
  - If you specify the S option, your input is used to create the SGS label, fields, and subfields. The label of the SGS is stored in the SSG global variable ACCSGS$.
  - If you specify the L option, an SGS is created using the *label* specified on the *ACCEPT directive, and your input becomes the fields and subfields.
  - The order in which the SGS is added to the chain of SGSs with the same label is determined by the value of the reserved global variable SOPT$ (see 3.2.3.3).
  - If the value of the reserved global variable lOPT$ is nonzero, the SGS is displayed in the SSG run log, preceded by the skeleton line number and a colon (:).

- If you do not use the E option, any Exec control image (@) input causes an error exit. This is also true if you use a "hard EOF" (non @EOF image) between the SSG call and actual skeleton execution.
  - If you use the E option and the control image is an @EOF (soft EOF), the global variable EOF$ is set to 1. SSG can accept more input if the control image is a "soft EOF."
  - If you use the E option and the control image is a non @EOF (hard EOF), then EOF$ is set to 2.
  - If there is no control image, then EOF$ is 0.

Any bracketed references in the *ACCEPT input are satisfied if LBRK$ is clear and the N option is not specified. This means you can enter bracketed SGS and variable references on input.

The maximum number of input characters for *ACCEPT is 121. If you use the L option, the label you specify is included as part of the 121-character limit.

Example 1

```
*ACCEPT,B name 'What is your name?'
```

Example 2

```
*ACCEPT,OS 'Enter boot tape SGS'
```

5.2.3. *ADD

The *ADD directive performs dynamic expansion of the SGS chain.

Format

```
*ADD(,opts)  SGS(,options)  specification
```

Parameters

opts

A Do not terminate in error if the source is not found or is not obtainable.

options

The same SGS options are available as on the SGS file identification statement. See Table 2–3 for a description of the options.

specification

The same four *specification* formats are available as on the SGS file identification statement. See 2.2 for a description of the four formats. For Formats 1 and 4 the *filename* can be an SDF file, a standard program file (PF), or a large program file (LPF). For Format 2 the *filename* can be a PF, an LPF, or a large element program file (LEPF). For Format 3 the *filename* can be a PF or an LPF.

Description

This directive allows you to dynamically add SGSs to the internal chain. All bracketed references are satisfied first. The order in which the SGSs are added to the chain of SGSs with the same label is determined by the value of reserved global variable SOPT$ (see 3.2.3.3). If the value of reserved global variable IOPT$ is nonzero, the SGSs are displayed in the SSG run log, preceded by the skeleton line number and a colon (:).

An SGS *label* must be specified for *specification* formats 2, 3, or 4. In all cases, the *label* is from 1 to 12 characters from the set of alphabetics (A through Z and a through z), numerics (0 through 9), dollar sign ($), and hyphen (-).

Example 1

```
*ADD SGS              TEMP.SGSLIST
```

Example 2

```
*ADD SGS              WORKFILE./ELEMENTS
```

Example 3

```
*ADD SGS              DUMMY.
```

Example 4

```
*ADD SGS,ES           SOMEFILE.ANELT   SI
```

5.2.4. *BRKPT

The *BRKPT directive positions the generated output stream.

Format

```
*BRKPT(,options) (name)
```

Parameters

options

Use any combination of characters. If LBRK$ is clear, bracketed references resolve to a string.

- K List the generated output images following this breakpoint until the next *BRKPT directive, or *EXIT, is encountered (see PRTOFF$, 3.2.3.2.
- P Create the output file/element in output symbiont PRINT$ format. If the P option is not present, output is generated in general symbolic format. This option must be present if a page eject (*EJECT) is required in the output file/element.
- R Set the *BRKPT K option value to match the value on the SSG call line. If the K option was present on the @SSG, the R option has the same effect as the K option specified on *BRKPT.

name

Use any combination of characters. If LBRK$ is clear, use bracketed references. Can be a file or an element and must be in standard filename.eltname notation. For this SSG directive, only a space-period-space (Δ.Δ) terminates the scan of the image. The name specifies the destination of all generated output images after the *BRKPT directive until the next *BRKPT directive is encountered or you exit. If you omit name and the generated output stream needs maintenance, SSG assumes that there is an internal file.

Description

All generated output images encountered before the first *BRKPT directive in a skeleton are controlled by the presence or absence of the K option on the @SSG call. The images are sent to the file or element specified in *param-3* of the @SSG call.

The B option on the @SSG call and the variable ADD_GOS$ control the last part of the generated output stream. This last part is the collection of output images generated between the last-interpreted *BRKPT directive in the skeleton (or start of skeleton execution if no *BRKPT directive is encountered), and the exit. If the *BRKPT directive is not used, the B and K options control the entire generated output stream.

Upon encountering a *BRKPT directive, SSG sets the values of ERRCNT$ and GOSCNT$ to 0; WARNINGCNT$ is not reset.

5.2.5. *CLEAR

The *CLEAR directive creates and changes variable values.

Format

```
*CLEAR  variable-name
```

Parameter

variable-name

Can be supplied by a string satisfying the description in 3.2.1 or a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

Description

SSG determines the variable to be cleared as follows:

- If a local variable of that name exists, SSG sets the value to zero.
- If no local variable of that name exists, SSG searches the global variable chain for that name, and if that name exists, sets it to 0.
- If SSG finds no global variable with that name, it creates one and gives it the value of 0. Once created, this global variable exists for the remainder of skeleton processing, unless it is removed (*REMOVE).

Example 1

```
*CLEAR  VAR
```

Example 2

```
*CLEAR  [VAR,1,1,1]
```

Example 3

```
*CLEAR  [#P]
```

5.2.6. *COPY

The *COPY directive adds skeleton images from other files or elements into the SSG skeleton.

Format

*COPY(,*options*) (*file.*)(*element*)

Parameters

options

A Do not terminate in error if the *file* or *file.element* is not found.

N Do not print the copied images.

(file.)(element)

Copy skeleton images from the file with this name.

Description

The *COPY directive allows you to copy in common skeleton images to help compose a main skeleton. This takes place on the first pass of the skeleton (skeleton verification, and so forth), so you cannot conditionally copy or put global variable references in *COPY statements.

- If the *file* or *file.element* does not exist, no copy takes place.
- If the A option is present and the source of the images is not found, you see a warning message but the skeleton is processed normally.
- If *file* is not specified, a search of SSG$PF, source-of-skel file, and TPF$ is made, in that order, until the reference is satisfied. The SSG$PF file must be assigned to the run in order for the search of that file to take place. If the *element* is not found, you see a warning message.

Example 1

```
*COPY,N TEMP.DEFINES
```

Example 2

```
*COPY,A TPF$.USER
```

Example 3

```
*COPY COPY-ELT
```

5.2.7. *CORRECT

The *CORRECT directive merges temporary and permanent correction streams.

Format

```
*CORRECT(,options) element-name(/version-name) ;

({PERM} | {TEMP}) (PCFxxxx) (WITH TCFyyyy) (GIVING PCFzzzz)

(skeleton generated corrections)

*ENDCORRECT
```

Parameters

options

Can be supplied by a string, variable reference, process parameter reference, SGS reference, or set reference that returns a string. Valid values are

none	In the absence of a (possibly assumed) GIVING phrase, the result of the correct operation is directed to the generated output stream.
K	Used in conjunction with the P option or GIVING phrase to additionally direct the images resulting from the correct operation to the generated output stream.
P	Operation of this option depends on PCF*xxxx* and PCF*zzzz*. The P option is ignored if PCF is not a P-option PCF set. If PCF*zzzz* is not specified, the P option forces SSG to assume that a GIVING PCF*xxxx* phrase was specified. If PCF*zzzz* is not specified and if PCF*xxxx* is not a P-option PCF set, the assumption above is not made and the P option is ignored. If PCF*zzzz* is a P-option PCF set, the images resulting from the merge are sent to the program file attached to PCF*zzzz*. A new symbolic element is created (the old one, if it existed, is replaced) with the same name as the element name (see 6.4).

element-name/version-name

Specifies the name of the element from which the correction streams are taken. Can be supplied by a string satisfying the description in 6.3 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

PCF*xxxx*

Specifies the input permanent stream. Can be supplied by a string satisfying the description in 6.4 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

- If this phrase is omitted, PCF is assumed.
- If TEMP is supplied, PCFxxxx is not allowed (and PCF is not assumed).
- If PCFxxxx is specified, it must be a defined PCF set.

TCF*yyyy*

Specifies the input temporary stream. Can be supplied by a string satisfying the description in 6.5 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

- If this phrase is omitted, TCF*xxxx* is assumed and the TCF set corresponding to the selected PCF set is used.
- If PERM is supplied, TCF*yyyy* is not allowed (and no assumption is made).
- If TCF*yyyy* is specified, it must be a defined TCF set. Otherwise, a NO FIND condition results.

PCF*zzzz*

Specifies the output permanent stream. Can be supplied by a string satisfying the description in 6.4 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

If this phrase is omitted, and if the P option is specified on the *CORRECT directive and PCFxxxx is a P option PCF set, PCFxxxx is assumed. Otherwise, the results of the merge are always sent to the generated output stream.

skeleton generated corrections

Specifies the symbolic images generated as a result of interpreting the skeleton images delimited by the *CORRECT directive and matching *ENDCORRECT directive.

Description

All phrases are optional except *element-name/version-name*. SSG merges the specified correction streams with any skeleton-generated corrections.

- If you specify PERM, the input permanent corrections and skeleton-generated corrections are merged.
- If you specify TEMP, the input temporary corrections and skeleton-generated corrections are merged.
- If you specify neither PERM nor TEMP, input permanent corrections, input temporary corrections, and skeleton-generated corrections are merged.

Skeleton-generated corrections are always used in the merge. If an element (correction stream) that is specified (or assumed) does not exist, the merge consists of the remaining specified (or assumed) entries.

Skeleton-generated correction images never correct a PCF set or TCF set element entry, only a base symbolic element.

Conflicts in line numbers on correction images are resolved as follows:

- TCF set corrections override PCF set corrections and skeleton corrections.
- PCF set corrections override skeleton corrections.

Example

```
@SSG,AE
SKEL
#ELT,UL      F.[P,1,1,1]
*CORRECT       [P,1,1,1]
-13,19
  A
  B
  C
*ENDCORRECT
@EOF
PCF
*DATA
-10,11
  A12
  A13
  A14
@EOF
  .
  .
  .
```

When you specify PERM on a *CORRECT directive, a revised temporary correction (RTC) entry is not created. No temporary stream is used in the merge. In addition, a created (*CREATE) element entry (primary TCF set only), has no correction stream attached to it when you create it; in this case, therefore, an RTC entry is not created. See 6.10 for more information about the RTC stream.

The P option is ignored for an element entry that does not already exist in PCF*zzzz* (PCF*xxxx* if assumed). To create a symbolic element in an attached program file for an element that does not already exist as an element entry, specify a GIVING phrase on the *CORRECT,P directive. In the absence of the GIVING phrase, the P option is ignored for an element entry that does not already exist in PCF*xxxx*. When an element in an attached program file is revised and reinserted into the program file, an update process is simulated. In either case, placing a GIVING phrase on the *CORRECT,P directive ensures modification of the attached program file.

Conflicts in line numbers are flagged. (See Appendix A for diagnostic messages.) Temporary corrections override permanent corrections. Permanent corrections override skeleton-generated corrections, if a conflict occurs. Since conflicts that cause overrides are not fatal errors, such a feature can be used to change the resulting output stream. When a conflict does occur, the highest priority corrections are saved as part of the output. Lowest priority corrections are displayed in the SSG run log. They follow the override message and are ignored. The reserved, global variable CONFLC$ can be used to monitor this (see Table 3–9).

-N- conflicts with -N when merging corrections. If the input temporary stream looks like this:

```
*element-name/version name
-557-
-557
```

then the following diagnostic is produced:

```
ERROR IN PROCESSING ELEMENT element-name/version-name
SEQUENCE OR FORMAT ERROR ON FOLLOWING IMAGE
-577
```

However, if the -577- or -577 image is in the input permanent stream and the other image, -577 or -577-, is in the input temporary stream, a conflict results and the following diagnostic is issued:

```
TEMP CORRECTIONS OVERRIDE PERM CORRECTIONS IN ELEMENT ;
element-name/version-name
-577(-)
```

Where the image is listed, -577 or -577- is derived from the input permanent stream and does not appear in the output permanent stream.

If -577- and -577 both appear in the input permanent stream and not in the input temporary stream, no error or conflict is noted and the output permanent stream is correct (see 5.2.30).

Some anomalies occur when using skeleton-generated corrections. These include use of edit mode, continuing a correction image, and skipping images due to overrides. If there are no errors or overrides, skeleton-generated corrections can be assumed to be correct. However, you should not use edit mode to create an image as the results may not be correct.

Note: *When resolving line number conflicts, the WITH phrase of *MERGE specifies the input temporary stream that has the same priority level as if it were a permanent stream. In addition, where *CORRECT assumes PCF and a matching TCF set, *MERGE makes no such assumption. The GIVING phrase for both directives is a similar assumption. Use of *CORRECT,P and *MERGE directive is analogous. Compatibility with previous levels of SSG is retained when logic dictates.*

5.2.8. *CREATE

The *CREATE directive performs dynamic expansion of internal chains.

Formats

```
*CREATE SGS(:)  sgs-image
```

$$\text{*CREATE} \left\{ \begin{matrix} \text{PERM:} \\ \\ \text{TEMP:} \end{matrix} \right\} \ element-name\ (/version-name)$$

```
*CREATE SGS filename./label
```

Parameters

sgs-image

See 3.1.1 for construction of an SGS image. Can be supplied by any combination of characters and, if LBRK$ is clear, bracketed references.

element-name/version-name

See 6.3 for construction of an element entry name. Can be supplied by any combination of characters and, if LBRK$ is clear, bracketed references.

filename./label

See 2.1.2, *param-2*, Format 2 for construction of this entry. Can be supplied by any combination of characters and, if LBRK$ is clear, bracketed references.

Description

After all bracketed references have been satisfied, an SGS based on *sgs-image* is created. The order in which the SGS is added to the chain of SGSs with the same label is determined by the value of reserved global variable SOPT$ (see 3.2.3.3). If the value of reserved global variable lOPT$ is nonzero, the SGS is displayed in the SSG run log, preceded by the skeleton line number and a colon (:).

After all bracketed references are satisfied, SSG creates an element based on *element- name/version-name*. In the absence of the O or Q options (used to alter the point of insertion into PCF and TCF sets, respectively) on the @SSG call, the element entry is normally inserted at the end of the PERM (primary PCF set) or TEMP (primary TCF set) chain. If /*version-name* is omitted, a blank version name is assumed. Once the new element is created, you can reference it like other elements. Created elements are treated as empty by SSG. The actual files used for input are not changed.

The following format is used to associate the named file with the SGS *label*:

```
*CREATE SGS filename./label
```

The SGS file name references can then be used to reference information about the file. The named file does not need to be a program file. Even if *filename* is a standard program file (PF), a large program file (LPF), or a large element program file (LEPF), no table of contents (TOC) SGSs are created.

If SSG is unable to assign the named file, the following message is displayed and the *CREATE directive is ignored:

```
*ERROR* – INVALID FILE SPECIFICATION – IGNORED
```

Example 1

```
*CREATE SGS:  LAB [lab1,1,1,1] [label2,1,2,3]
```

Example 2

```
*CREATE SGS:  DEFAULT IS ...
```

Example 3

```
*SET IOPT TO IOPT$
*CLEAR IOPT$
*CREATE SGS:  INTERNAL SGS IS ...
*SET IOPT$ TO IOPT
```

Example 4

```
*CREATE PERM: ELT
```

Example 5

```
*CREATE TEMP: ELT/VER
```

Example 6

```
*CREATE SGS FILE./TEST
```

Example 7

```
*EDIT ON
*CREATE SGS: LAB THIS,ONE,&
SGS CREATED,USING THE &
EDIT DIRECTIVE
```

This example would create the following SGS:

```
LAB THIS,ONE,SGS CREATED,USING THE EDIT DIRECTIVE
```

It illustrates the use of the *EDIT directive (see 5.2.16) to create SGSs.The created image must not exceed the maximum number of characters defined by IMGLEN$ (see Table 3–9).

Example 8

```
*CSF @CAT,P MYFILE(+1).,F
*CREATE SGS MY*FILE./INFO
*DISPLAY 'File cycle is [INFO,0,C]'
```

This example would display the absolute file cycle of the empty file created with the *CSF directive. It illustrates that the *CREATE directive can be used to create file information for files other than program files.

5.2.9. *CSF

The *CSF directive submits Exec control statement images.

Format

*CSF(,*options*) *image*

Parameters

options

Can be supplied by any combination of characters. If LBRK$ is clear, bracketed references resolve to a string. The L option prints the CSF image in the SSG run log. All other specified options are ignored.

image

Can be supplied by any combination of characters and, if LBRK$ is clear, bracketed references. Leading spaces are ignored. However, the first nonspace character must be @.

Description

The *CSF directive directs SSG to perform an ER ACSF$ with the supplied image. Leading spaces are ignored, but the first nonspace character must be an @. (SSG shifts the image left so that the @ is in column 1). SSG processes *image* as if it is a quoted string, even though quotation marks are not required and should not be specified. SSG verifies only that the first character is @, not that the entire image is a valid ER ACSF$ request. Continuation is not allowed anywhere in the *CSF directive, except within bracketed references, if LBRK$ is clear. If a semicolon is included in the image, an ER (error type 04) error code 040 contingency occurs.

Completion of an ER ACSF$ request occurs when the Exec returns control to SSG. Upon completion, the status is returned in register A0 by the Exec, and is provided in two ways. The value is available in the reserved, global variable CSFSTA$, and an edited value is available through the SGS reference [CSFSTA$,1,1,1] (see 3.1.4). You can determine the status and use this information to affect ERRCNT$, or verify destinations for parts of the generated output stream.

Note: *If you use *CSF to direct the SSG run log (and possibly other listings generated by SSG) to an alternate print file (@BRKPT), you can not assume that margins will be consistent. This is because SSG makes no assumptions about image contents.*

Example 1

```
*CSF  @ASG,T FILE[*VAR].,F/1
```

Example 2

```
*CSF  @ADD selected-stream
```

Example 3

```
*CSF  @BRKPT,V  RUNSTREAM.
```

Example 4

```
*CSF  @BRKPT,W  DOCUMENT.UPDATE[*I]
```

5.2.10. *DEFINE and *ENDDEFINE

The *DEFINE directive and *ENDDEFINE directive define skeleton subroutines.

Format

```
*DEFINE  define-name
(skeleton images)
*ENDDEFINE
```

Parameter

define-name

Can be a string, SGS reference, or a set reference. SSG uses only the first 20 characters. The *define-name* is case independent. That is, NAME is the same as name. Only alphanumeric, $, and _ characters are allowed. The complete *define-name* must be on the same image as the *DEFINE directive. Continuation is not allowed anywhere in the *DEFINE directive, not even within an SGS reference or set reference.

Description

All images between the matching *DEFINE and *ENDDEFINE directives constitute the define packet. None of the images are interpreted until a call for that define packet is made by a *PROCESS directive, or the define name is preceded by an * in column 1. From a define packet, nested calls for other define packets and calls for the same define packet are allowed to any depth. When a *PROCESS directive causes define packet images to be interpreted, parameters can be passed to the define packet. For an explanation of how to reference these parameters, see 3.3.

Example 1

```
*DEFINE  NAME
```

Example 2

```
*DEFINE  [NAME,1,1,1]
```

Example 3

```
*DEFINE  LOWER_to_upper
```

Example 4

```
*Define  lower_to_UPPER
```

Examples 3 and 4 are equivalent.

5.2.11. *DISPLAY

The *DISPLAY directive prints text in the SSG run log.

Format

```
*DISPLAY(,options) val (val...)
```

Parameters

options

E Increment the value of ERRCNT$.

O Display the text on the operator console.

val

Text displayed in the SSG run log. The parameter can be a string variable name, a numeric variable name, or a quoted string.

Description

The *DISPLAY directive prints text in the SSG run log. A space must separate each *val*. On output the strings will be concatenated, ignoring spaces unless the strings are in quotes or are part of a string variable value. To enclose a quoted string, use single quotation marks.

Example

```
*SET A = 132
*SET MSG = 'There are '
*DISPLAY MSG A ' cars in the parking lot, '
*DISPLAY 'but there isn''t a single yellow one. '
```

This example prints the following string:

```
There are 132 cars in the parking lot,
but there isn't a single yellow one.
```

5.2.12. *DIVIDE

The *DIVIDE directive performs variable division.

Format

```
*DIVIDE number BY number GIVING variable-name-1(,variable-name-2)
```

Parameters

number

Integer expression or numeric expression.

variable-name

Can be supplied by a string satisfying the description in 3.2.1 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

Description

The *DIVIDE directive directs the division of the first number by the second number. It sets the first *variable-name* equal to the quotient, and the second *variable-name*, if present, equal to the remainder. The second *variable-name* can duplicate the first *variable-name* saving only the remainder. For each *variable-name*, if no local variable name exists, SSG searches for a global variable of that name. If no global variable exists, SSG creates one. Once created, this global variable exists until it is removed (*REMOVE).

Note: *If you attempt to divide by 0, an error diagnostic appears in the SSG run log and the quotient and remainder are set to 0.*

Example 1

```
*DIVIDE A+B BY C-D GIVING QVO,REM
```

Example 2

```
*DIVIDE 1+[#P] BY 7 GIVING REM,REM
```

Example 3

```
*DIVIDE A BY B  GIVING  C
```

5.2.13. *DLOC

The *DLOC directive obtains or releases secure privileges.

Format

$$\text{*DLOC} \quad \left\{\begin{matrix}\text{ON} \\ \text{OFF}\end{matrix}\right\}$$

Description

The *DLOC directive provides the capability to obtain or release secure privileges while executing in SSG. These privileges can be obtained only if the user-id for the run in execution is authorized to obtain such privileges.

The initial privileged state of the run is contained in [INFO$,1,4,2] and has a symbolic value of ON or OFF. This can be use to determine if a *DLOC directive is necessary (see example 1) and can be used to reset the privileged state to its initial value (see example 2).

The *DLOC directive uses ER MODPS$ to modify the privileged state of the run. If this ER is not authorized for the run, a security error will occur. In this case, secure privileges may still be obtained by using the directive

```
*CSF @ASG,A SYS$*DLOC$/read-key/write-key
```

if the correct read key and write key for the file SYS$*DLOC$ are specified.

Secure privileges may subsequently be released using the directive

```
*CSF @FREE SYS$*DLOC$.
```

Refer to 5.2.9 for additional information about the *CSF directive.

Example 1

```
*IF [INFO$,1,4,2]=OFF
*DLOC ON
*ENDIF
```

Example 2

```
*DLOC [INFO$,1,4,2]
```

5.2.14. *DO and *ENDDO

The *DO directive allows repetitive interpretation of a block of statements.

Format

```
*DO (VARNAME=)variable-name (FROM(=)from-value) ;
```

$$(\text{TO}(=)to-value\ (\text{BY}(=)by-value\ (\text{WHILE}(=)variable-name\ \text{IS}\ \left\{\begin{matrix}\text{SET}\\ \\ \text{CLEAR}\end{matrix}\right\})$$

$$(skeleton\ images)$$

```
*ENDDO
```

Description

All parameters function the same as the *INCREMENT directive (see 5.2.26). The equal sign can be replaced by a space, but the equal sign is recommended.

5.2.15. *DUMP

The *DUMP directive (described in Table 5–1) is a debug aid, and displays values of variables in the SSG run log.

Format 1

$$\text{*DUMP}\ \left\{\begin{matrix}\text{LOCAL}\\ \\ \text{GLOBAL}\end{matrix}\right\}$$

Format 2

$$\text{*DUMP}\ (\left\{\begin{matrix}\text{LOCAL}\\ \\ \text{GLOBAL}\end{matrix}\right\})\ \ variable-name-1\ (,\ldots.variable-name-n)$$

Parameter

variable-name

Can be supplied by a string satisfying the description in 3.2.1 or a variable reference or process parameter reference that returns such a string.

Table 5–1. DUMP Directive

Directive	Description
*DUMP GLOBAL	Displays the values of all global variables in the SSG run log. (The global variable chain includes the SSG reserved variables.)
*DUMP LOCAL	Displays the values of all local variables in the SSG run log.
*DUMP LOCAL *variable-name,...* ; *variable-name-n*	Displays the values of the specified local variables in the SSG run log. If a specified variable is not found, the words NO FIND are substituted for the value of the variable.
*DUMP *variable-name* ; *,...,variable-name-n*	Displays the values of the specified local and/or global variables, searching first the local variable chain and then the global variable chain, in the SSG run log. If a specified variable is not found, the words NO FIND are substituted for the value of the variable.

Note: *SGS references and set references cannot be used to supply a variable name.*

5.2.16. *EDIT

The *EDIT directive concatenates nondirective images.

Format 1

This format is used if EDITAB$ has (or will have using the C option) a value of 0.

$$\text{*EDIT}(,options)\ \text{ON}\ \begin{bmatrix} edtsym \\ edtsym\ .\ comment \\ .\ comment \end{bmatrix}$$

Format 2

This format is used if EDITAB$ has (or will have, using the S option) a nonzero value.

```
*EDIT(,options) ON (edtsym(edtimg(edtsym(coled ... ))))
```

Format 3

This format is used to unconditionally terminate edit mode.

```
*EDIT(,options) OFF
```

Parameters

options

Can be supplied by a string or a variable reference, process parameter reference, SGS reference, or set reference that returns a string.

C Clear the value of EDITAB$ (value of 0)

S Set the value of EDITAB$ (value of 1).

edtsym

A one-character string or a variable reference, process parameter reference, SGS reference, or set reference that returns a string. The first character returned is used.

The edit symbol (*edtsym*) is optional. When it is omitted, an ampersand (&) is assumed.

The period (.) is a valid edit symbol so *comment* can be supplied only when *edtsym* is specified and EDITAB$ has (or will have) a value of 0 (format 1).

Note: *It is recommended that if EDITAB$ has (or will have) a nonzero value;* edtsym *cannot be a special character.*

edtimg

Any combination of characters and, if LBRK$ is clear, bracketed references. The image cannot contain the edit symbol (*edtsym*). Characters are edited until an occurrence of *edtsym* or end of image is encountered. End of image terminates edit mode and the edited image is output. An occurrence of *edtsym* initiates a search for *coled*. If *coled* is omitted, editing continues with the next skeleton image.

coled

Can be supplied by an integer or numeric expression. The value of COLED$ (see Table 3–9) is replaced with the value of the expression. Since COLED$ is also maintained by SSG, a "skip count" can be specified by using COLED$+ some value.

If *edtsym* terminates the scan of *coled*, SSG assumes another *edtimg*. Editing then continues with the current image. This new *edtimg* can be followed by *edtsym* and, if desired, another *coled*, and so on.

If any character other than *edtsym* (period or space) terminates the scan of *coled*, editing continues with the next skeleton image.

Note: *The value of LBRK$ (see Table 3–9) does not affect the evaluation of* coled.

Description

Following are tasks associated with the *EDIT directive:

- Building a nondirective or *CREATE SGS image

  Use edit mode to build a nondirective or a *CREATE SGS image from one or a group of skeleton images. Depending on the value of EDITAB$, edit mode operates in two ways.

  – If EDITAB$ has a value of 0, the occurrence of *edtsym* on a nondirective image terminates the scan of that image. Edit mode continues with the next skeleton image.

  – If EDITAB$ has a nonzero value, multiple *edtsym* characters are allowed on the image now being edited. This allows you to specify with *coled* on which columns to start the next *edtimg*. You can specify columns by placing *edtsym* on the *EDIT directive or on a nondirective image.

    The edited image starts with the first *edtimg*. The first *coled* is evaluated and used to set the value of COLED$ (specify a column position). The next *edtimg* is edited, the next *coled* is evaluated, and so on.

- Turning on edit mode

  Turn on edit mode with the *EDIT ON directive. The initial image is space-filled and the value of COLED$ is set to 1.

  – When edit mode is on, all nondirective or *CREATE images terminated by *edtsym* are concatenated. The value of COLED$ is increased as the image is built.

  – If the image being edited becomes larger than the number of characters defined by the value of IMGLEN$ (see 4.3), a semicolon is automatically inserted and the image is output. Editing then begins with a new space-filled image and the value of COLED$ is set to 1.

    Note: *An edit-mode *CREATE image cannot exceed the maximum number of characters defined by the value of IMGLEN$.*

- Execution when the edit mode is on

  You can execute directive images (other than *BRKPT, *CORRECT, and *MERGE) while you edit.

  – Any nondirective image without *edtsym* terminates edit mode.

  – The *EDIT OFF directive terminates edit mode.

  – The final created image is output to the generated output stream or the PCF*zzzz* specified on a *CORRECT directive, or both.

  - SSG ignores any *EDIT OFF directive outside of edit mode. The value of COLED$ is set to 0.

  - An *EDIT ON directive encountered in edit mode destroys partially edited images and repositions the column pointer, COLED$, to column 1.

- Editing a *CREATE image

  When you edit a *CREATE image, it must follow the *EDIT ON directive.

  - You can execute any other directive image (other than *BRKPT, *CORRECT, and *MERGE) while edit mode is on, but the occurrence of any portion of the *CREATE image without *edtsym* terminates edit mode.

  - You can terminate edit mode with the *EDIT OFF directive.

  - The final image is added to the chain of *CREATE images.

- Modifying COLED$ and EDITAB$

  You can use a global variable modifying directive to modify the values of COLED$ and EDITAB$.

  - You can control image construction and type using an edit image. Changes to COLED$ are not recognized until the next nondirective image or *edtimg* is processed, and EDITAB$ has a nonzero value (see 4.3).

  - If EDITAB$ has a nonzero value and the first character in the first *edtimg* specified on the *EDIT directive is " or #, the conversion based on the value of PILO$ or NSIGN$ is performed.

  - For other (nondirective) images, the character in the first column position is checked.

Example 1

```
*EDIT ON825
```

Example 2

```
*EDIT OFF
```

Example 3

```
*EDIT ON &A&11&b&21&C
```

Example 4

```
*EDIT ON &LAB&11&L,U&21&A0,[#1]&39&.
```

Example 5

```
*EDIT,S ON &
LAB&11
L,U&21
A0,X1&39
```

Example 6

```
*EDIT,S ON &
LAB&11&L,U&21
A0,X1&39&. This is a &
COMMENT
```

5.2.17. *EJECT

The *EJECT directive controls page ejects.

Format

*EJECT(,*options*)

Parameter

options

Can be supplied by a string or an SGS or set reference that returns a string. Valid values are

none	Assumes the E option.
D	When the revised skeleton stream is listed by using the E option in combination with the D option on the @SSG call (see Table 2–1), a page eject occurs. That is, the next image of the revised skeleton stream, if it exists, appears on the next page.
E	When the revised skeleton stream is listed by using the E option without the D option on the @SSG call (see Table 2–1), a page eject occurs. That is, the next image of the revised skeleton stream, if it exists, appears on the next page.
K	When the generated output stream is listed by using the K option on the @SSG call (see Table 2–1) or an *BRKPT directive (see 5.2.4), a page eject occurs. That is, the next image of the generated output stream, if it exists, appears on the next page.

Description

The *EJECT directive makes the revised skeleton stream and generated output streams easier to read. It separates logically independent sections of output. The E, D, and K options can appear on the same *EJECT directive, in any order.

Note: *The options specification on an *EJECT directive is determined at the time the revised skeleton stream is built and before it is processed. You cannot use a process parameter reference to supply options, and cannot modify the options during processing.*

5.2.18. *ELSE

The *ELSE directive specifies the start of the block of code to be executed if the *IF condition is not true.

5.2.19. *ELSEIF

The *ELSEIF directive has the same affect as an *ELSE followed immediately by an *IF.

5.2.20. *ENDCORRECT

The *ENDCORRECT signifies the end of a *CORRECT block.

5.2.21. *ENDDEFINE (,E)

The *ENDDEFINE directive signifies the end of a *DEFINE block. The E option causes a page eject following the *ENDDEFINE if you did not use the D option on the SSG processor call.

5.2.22. *ENDDO (,E)

The *ENDDO directive signifies the end of a *DO block. The E option causes a page eject following the *ENDDO if you did not use the D option on the SSG processor call.

5.2.23. *ENDIF (,E)

The *ENDIF signifies the end of an *IF, *ELSE, or *ELSEIF block. The E option causes a page eject following the *ENDIF if you did not use the D option on the SSG processor call.

5.2.24. *EXIT

The *EXIT directive allows early termination of a loop.

Format

```
*EXIT  (varname)
```

Description

The exit directive terminates an *INCREMENT or *DO loop.

- If *varname* is not specified, the innermost loop is terminated.
- If *varname* is specified, the loop using that loop control variable is terminated.
- If *varname* does not exist, a NO FIND occurs and the directive is ignored.

Example

```
*INCREMENT COUNT TO [PARK]
  *IF [PARK,COUNT,1,1]=STOP
    *EXIT
  *ENDIF
  *CSF @ASG,T [PARK,COUNT,1,1]
*LOOP
```

5.2.25. *IF, *ELSEIF, *ELSE, and *ENDIF

*IF and *ELSEIF are decision-making directives designed to test (evaluate) *conditions* and choose logical paths in the skeleton. This is accomplished by either interpreting or skipping a block of skeleton images. There are four general formats for *IF in a skeleton.

Format 1

```
*IF condition
(skeleton images)
*ENDIF
```

You can put any directive or nondirective skeleton image except *DEFINE between the matching *IF and *ENDIF directives.

- If *condition* is true, images between the *IF and *ENDIF directives are interpreted.
- If *condition* is false, they are skipped.

Format 2

```
*IF condition
(skeleton images)
*ELSE
(skeleton images)
*ENDIF
```

You can put any directive or nondirective skeleton image except *DEFINE between the matching *IF and *ELSE and the matching *ELSE and *ENDIF directives.

- If *condition* is true, the skeleton images between the *IF and *ELSE directives are interpreted, and the images between the *ELSE and *ENDIF directives are skipped.
- If *condition* is false, the skeleton images between the *IF and *ELSE directives are skipped and the images between the *ELSE and *ENDIF directives are interpreted.

Format 3

```
*IF condition-1
(skeleton images)
*ELSEIF condition-2
(skeleton images)
      .
      .
      .

*ELSEIF condition-n
(skeleton images)
*ENDIF
```

You can put any directive or nondirective skeleton image except *DEFINE between the matching *IF and *ELSEIF, *ELSEIF and *ELSEIF, ..., and *ELSEIF and *ENDIF directives.

- If *condition-1* is true, the skeleton images between the *IF directive and first *ELSEIF directive are interpreted. All other images are skipped. If *condition-1* is false, *condition-2* is evaluated.
- If *condition-2* is true, images between the first and second *ELSEIF directives are interpreted. All other images are skipped. If *condition-2* is false, the next *condition-i* is evaluated until a *condition-i* is true or there is no further *condition-i* to evaluate.
- If no *condition-i* is true, no images are interpreted.

Note: *The *ELSF and *END directives are still valid, but their use is discouraged in new skeletons. The *ELSEIF and *ENDIF directives should be used instead.*

Format 4

```
*IF condition-1
(skeleton images)
*ELSEIF condition-2
(skeleton images)
      .
      .
      .

*ELSEIF condition-n
(skeleton images)
*ELSE
(skeleton images)
*ENDIF
```

You can put any directive or nondirective skeleton image except *DEFINE between the matching *IF and *ELSEIF, *ELSEIF and *ELSEIF, ..., *ELSEIF and *ELSE, and *ELSE and *ENDIF directives.

- If *condition-1* is true, the skeleton images between the *IF directive and first *ELSEIF directive are interpreted. All other images are skipped. If *condition-1* is false, *condition-2* is evaluated.
- If *condition-2* is true, images between the first and second *ELSEIF directives are interpreted. All other images are skipped. If *condition-2* is false, the next *condition-i* is evaluated until a *condition-i i*s true or there is no further *condition-i* to evaluate.
- If no *condition-i* is true, images between the *ELSE and *ENDIF directives are interpreted.

An *ELSE or *ELSEIF directive is assumed to match the last *IF directive that has not been matched with an *ENDIF or an *ELSE and *ENDIF directive. A matched set of *IF, *ELSEIF, *ELSE, and *ENDIF directives is an IF packet. The IF packet shown in Format 4 can be nested statically or dynamically to any level. No overlap of IF packets with other IF packets, or with increment loops, is allowed. Each increment loop or IF packet must be nested entirely within another packet or loop (unless it starts the nest). Verification that *IF directives and increment loops are properly nested and that *IF, *ELSEIF, *ELSE, and *ENDIF directives are correctly matched is performed when you build the revised skeleton.

Example allowed

```
*IF condition-1

------
------
------

*ELSE
   *INCREMENT A TO 3
      ------
      ------
      ------
   *LOOP

   *IF condition-2
     ------
     ------
     ------
   *ENDIF

*ENDIF
```

Example not allowed

```
  *IF condition-1

  ------
  ------
  ------

       *IF condition-2
           ------
           ------
           ------

       *ELSE
        ------
          ------
        ------

  *ELSE
       ------
       ------
       ------
  *ENDIF
       ------
       ------
       ------
  *ENDIF
```

5.2.25.1. Description of Conditions

This subsection discusses simple and compound conditions.

Simple Conditions

There are six types of conditions that can be evaluated as true or false. For complete syntax and detailed explanations, refer to 5.2.25.2 through 5.2.25.7.

Condition 1

Test if a numeric variable is nonzero or zero (see 5.2.25.2).

$$variable-name \text{ IS } \left\{\begin{matrix}\text{SET}\\ \\ \text{CLEAR}\end{matrix}\right\}$$

Condition 2

Test if satisfying a bracketed reference results in a NO FIND (see 5.2.25.3).

expression

Condition 3

Test if a numeric expression is nonzero or zero (see 5.2.25.4).

expression

Condition 4

Compare two operands symbolically or numerically (see 5.2.25.5).

$$operand-1 \left\{\begin{matrix}<\\ <=\\ =\\ \\ <>\\ >=\\ >\end{matrix}\right\} operand-2$$

Condition 5

Search selected SGSs, PCF sets, or TCF sets for a symbolic value (see 5.2.25.6).

$$\left\{\begin{matrix}\text{ROW}\\ \text{COLUMN}\\ \text{KEYWORD}\end{matrix}\right\} \text{ SEARCH } (\left\{\begin{matrix}\text{ASC}\\ \text{ASCENDING}\\ \text{DESC}\\ \text{DESCENDING}\end{matrix}\right\}) \text{ FROM } label\left(,n\left(,f\left(,s\right)\right)\right) \text{ FOR } expression$$

Condition 6

Test if an element has permanent or temporary corrections (see 5.2.25.7).

$$element-name(/version-name)\ \text{HAS (ANY)}\quad (\{^{\text{PERM}}_{\text{TEMP}}\})\quad \text{COR(RECTIONS)}$$

The following applies to evaluation of all six conditions:

- Failure to satisfy references

  A failed attempt to satisfy any reference immediately results in the entire condition being set to false. No further examination of the condition is performed. The variable NOFND$ is not incremented. The normal NO FIND error messages are not output during condition evaluation. None of the following error messages is output:

  ```
  *ERROR* - NO FIND BEFORE COLUMN c ON FOLLOWING IMAGE
  Variable 'variable' is undefined.
  SGS reference '[reference]' is undefined.
  ```

- Format errors

  Format errors result in the normal error messages, in ERRCNT$ being incremented and in the entire condition being set to false. No further examination of the condition is performed.

- Supplying condition keywords

  The condition syntax includes the keywords IS, SET, CLEAR, VALUE, OF, <, <=, =, <>, >=, >, ROW, COLUMN, KEYWORD, SEARCH, ASC, ASCENDING, DESC, DESCENDING, FROM, FOR, HAS, ANY, PERM, TEMP, COR, and CORRECTIONS. Strings, variable references, or process parameter references that return the symbolic keyword may supply all of these. If a NO FIND results when you attempt to satisfy the references, the condition is set to false.

- Boolean keywords within conditions

  The keywords AND, OR, and XOR are Boolean operators described below for compound conditions. Incorrect evaluation of the condition or a format error may result if a string, variable reference, or process parameter reference returns a Boolean operator for any part of the condition. The Boolean operators may appear within a condition if they are supplied by an SGS reference or a set reference.

Compound Conditions

The *IF and *ELSEIF directives can test more than one condition by use of compound conditions, also called logical or Boolean expressions. The compound conditions evaluate as true or false using one or more conditions and the logical or Boolean operators NOT, AND, OR, and XOR.

Format

$$(\text{NOT})\ condition-1 \quad (\{\text{AND}\ \text{OR}\ \text{XOR}\}\ (\text{NOT})\ condition-2 \quad (\{\text{AND}\ \text{OR}\ \text{XOR}\}\ \ldots))$$

where:

condition-n	Is any one of the six types of simple conditions.
NOT, AND, OR, XOR	Are the Boolean operators. See "Results of Boolean Operations."
parentheses ( )	Used to indicate an optional entry in the syntax of a compound condition, as in (NOT). Can also be used to change the evaluation order of Boolean expressions. See "Using Parenthetical Expressions to Change Evaluation Order" and also "Constraints Against Using Parenthetical Expressions."

Description

This subsection describes in more detail the syntactical components for compound conditions.

Results of Boolean Operations

Table 5–2 contains the possible combinations of conditions and the true/false results produced by the Boolean operators.

Table 5–2. Results of Boolean Operations Using AND, OR, XOR, and NOT

Condition-1	Condition-2	Result of AND	Result of OR	Result of XOR	NOT Condition-1
false	false	false	false	false	true
false	true	false	true	true	true
true	false	false	true	true	false
true	true	true	true	false	false

Priorities of Boolean Operators

Table 5–3 shows the priorities of the Boolean operators, where 1 is the highest priority and 3 is the lowest. High priority operators are evaluated before low priority operators and operators with the same priority are evaluated from left to right.

Traditionally SSG has assigned the same priority to AND, OR, and XOR (see column 2 of this table). Beginning with SSG 24R1, it is possible to have SSG assign a higher priority to AND than to OR and XOR, as is customary for most languages that support Boolean operations (see column 3 of Table 5–3).

Table 5–3. Boolean Operator Priorities

Operator	Traditional Evaluation Priority	New Evaluation Priority
NOT	1	1
AND	2	2
OR	2	3
XOR	2	3

Using NEW_BOOLEAN$ to Control Evaluation Priorities

Skeletons must be designed to operate with either the traditional or the new evaluation priorities, but not with both. The reserved global variable NEW_BOOLEAN$ controls which evaluation priorities are to be used during skeleton execution.

- When NEW_BOOLEAN$ has the default value of zero, the traditional evaluation priorities are used, and skeletons will continue to operate as they always have.
- When NEW_BOOLEAN$ is explicitly set to a nonzero value, the new evaluation priorities are used.

Note: *You must set NEW_BOOLEAN$ at the beginning of skeleton execution, because once a NOT, AND, OR, or XOR operator is encountered, NEW_BOOLEAN$ is set to read-only and cannot be changed for the remainder of the skeleton execution.*

Using Parenthetical Expressions to Change Evaluation Order

When NEW_BOOLEAN$ is nonzero, left and right parentheses can be used to bracket any simple or compound condition, thereby creating a parenthetical expression. A parenthetical expression can be used to change the order in which Boolean expressions are evaluated. The evaluation of a Boolean expression may be interrupted to evaluate a parenthetical expression contained within it. This might produce an evaluation order that differs from the one shown in Table 5–3.

- Format of a parenthetical expression
  - Left parentheses are only allowed to the left of a NOT Boolean operator and to the left of a simple condition.
  - Right parentheses are only allowed to the right of a simple condition.
  - Each left parenthesis must have a matching right parenthesis.
  - To improve readability, any number of space characters may be included before and after parentheses.
- Attributes of a parenthetical expression
  - Evaluates as true or false.
  - Evaluates as a separate unit.
  - Allowed anywhere a simple condition is allowed.
  - May be nested up to 25 levels.

When a parenthesis is encountered, the reserved global variable NEW_BOOLEAN$ is set to read-only and cannot be changed for the remainder of the skeleton execution.

Constraints Against Using Parentheses

- Parentheses are not allowed within simple conditions.

  Simple condition formats 2, 3, 4, and 5, described in 5.2.25.3, 5.2.25.4, 5.2.25.5, and 5.2.25.6, respectively, all allow numeric expressions such as VALUE1-VALUE2-VALUE3. The use of parentheses in a numeric expression such as VALUE1-(VALUE2+VALUE3) is not allowed and results in a format error.

- Parentheses are not allowed within any bracketed reference.

  Bracketed references may contain numeric expressions, but parentheses within a numeric expression are not allowed and will result in a format error. For example, The -A-B-C in [LABEL,-A-B-C,1,1] is valid, but [LABEL,-(A+B+C),1,1] is not.

Compatibility Issues When NEW_BOOLEAN$ is Nonzero

- Parentheses are special characters for the new evaluation. *IF directives that contain parentheses may operate when NEW_BOOLEAN$ is zero, but not when it is nonzero.
  - Strings that contain parentheses might cause format errors, as in this example:

    ```
    *IF COLUMN SEARCH FROM TEXT,1,1,1 FOR Copyright(c)
    ```

  - If variable names, SGS labels, PCF set names, or TCF set names contain left or right parentheses, they may cause format errors, as in this example:

    ```
    *SET SUB(1)TOT = A+B+C+D+E
    *SET SUB(2)TOT = I+J+K+L+M
    *SET SUB(3)TOT = V+W+X+Y+Z
    *IF SUB(1)TOT+SUB(2)TOT < +SUB(3)TOT
    ```

Evaluating Boolean Expressions Without Parentheses

The following steps describe the procedure for both the traditional evaluation of a Boolean expression (NEW_BOOLEAN$ = 0) and the new evaluation of a Boolean expression that does not include parentheses (NEW_BOOLEAN$ = nonzero).

1. Evaluate *condition-1* as true or false and save it as *result-1*. If the optional NOT operator precedes *condition-1*, negate *result-1* (as shown in Table 5–2). If *result-1* is true, change it to false. If *result-1* is false, change it to true.
2. If this is the end of the expression, proceed to step 6. Otherwise, this should be an AND, OR, or XOR operator, which is saved as *operator-1*.
   a. If *result-1* is false and this is the AND operator, store false in *result-2*. Ignore the optional NOT operator and *condition-2* that follow.
   b. If *result-1* is true and this is the OR operator, store true in *result-2*. Ignore the optional NOT operator and *condition-2* that follow.
   c. Otherwise, evaluate *condition-2* as true or false and save it as *result-2*. If the optional NOT operator precedes *condition-2*, negate *result-2*.
3. If this is the end of the expression, proceed to step 5. Otherwise, this should be an AND, OR, or XOR operator.
   a. If traditional evaluation is in use, evaluate the expression "*result-1 operator-1 result-2"* (as shown in Table 5–2). Store the evaluation result in *result-1* and return to step 2 to process this operator as the new *operator-1*.
   b. If new evaluation is in use and the priority of *operator-1* is the same as or higher than the priority of this operator, then evaluate the expression "*result-1 operator-1 result-2*" and store the evaluation result in *result-1*. Return to step 2 to process this operator as the new *operator-1*.
   c. If new evaluation is in use and the priority of *operator-1* is lower than the priority of this operator, then store this operator as *operator-2*. This can only occur if *operator-1* is a low priority OR or XOR operator and *operator-2* is a high priority AND operator. Proceed to step 4.
4. If new evaluation is in use, process the condition following the AND operator as follows:
   a. If *result-2* is false, ignore the optional NOT operator and *condition-3* that follow. Return to step 3.
   b. If *result-2* is true, evaluate *condition-3* as true or false and save it as *result-3*. If the optional NOT operator precedes *condition-3*, negate *result-3*. Evaluate the expression "*result-2* AND *result-3"* and then store the evaluation result in *result-2*. Return to step 3.
5. Evaluate the expression "*result-1 operator-1 result-2"*. Store the evaluation result in *result-1*.
6. The final result of the Boolean expression is in *result-1*.

Evaluating Boolean Expressions With Parentheses

When NEW_BOOLEAN$ is nonzero, parenthetical expressions are allowed. This makes the evaluation of expressions much more complex.

- In steps 1, 2c, and 4b of the previous procedure, any of the conditions to be evaluated (*condition-i*) may each be a parenthetical expression. In this case, the evaluation of the current Boolean expression is interrupted, and steps 1 through 6 are performed to evaluate the parenthetical expression as true or false. The NOT operator may be used to negate an entire parenthetical expression. Once the final value of the parenthetical expression has been determined, the interrupted expression evaluation resumes, using the final value as *condition-i*.
- In steps 2a, 2b, and 4a, the *condition-i* to be ignored may each be a parenthetical expression. In this case, everything after the left parenthesis—including all nested parenthetical expressions—is ignored until the matching right parenthesis is found.

Components of Boolean Expressions

The following is a description of the syntactical components of a Boolean expression:

condition-i	Any combination of the six simple conditions can be used to provide the *condition-i* in any order.
Boolean operators	Any combination of the AND, OR, and XOR Boolean operators can be used in any order. These operators can be supplied by a string, variable reference, or process parameter reference. ***Note:*** *For clarity, readability, and maintainability, it is recommended that you do not use variable references or process parameter references to supply Boolean operators.*
Left and right parentheses	When NEW_BOOLEAN$ is nonzero, the left and right parentheses can only be supplied by strings.

Boolean Operations

- Skipping unnecessary evaluations

  To improve efficiency, SSG does not evaluate the condition to the right of a Boolean operator unless it is necessary to determining the result. Skipped evaluations are shown in the previous procedure.

  – In steps 2a and 4a, the evaluation of the condition after "false AND" is skipped. Whether the skipped condition is true or false, the result of the evaluation will be false.

  – In step 2b, the evaluation of the condition after "true OR" is skipped. Whether the skipped condition is true or false, the result of the evaluation will be true.

  - In steps 2a, 2b, and 4a, if the condition or parenthetical expression being skipped has format errors, they will not be detected because SSG is searching only for the next AND, OR, or XOR operator.

- Handling NO FIND condition and format errors
  - In steps 1, 2c, and 4b, if a *condition-i* evaluation results in a NO FIND, the corresponding *result-i* is set to false. SSG skips the remainder of the condition and searches for the next AND, OR, or XOR operator.
  - If a format error is encountered anytime during the evaluation of the Boolean expression, the final result of the evaluation is false and the remainder of the expression is ignored.
- Tips for debugging Boolean expressions

  When NEW_BOOLEAN$ is nonzero, it is possible to have complex expressions with many levels of nested parenthetical expressions.

  - When debugging these *IF directives, it may be helpful to set the reserved global variable NOPT$ (see Table 3–9) to a nonzero value immediately before the directive and clear it immediately after the directive. This produces a detailed trace of the Boolean logic as the expression is being evaluated.
  - The OPT file identification N option (see Table 2–4) can also be used to set NOPT$ to a nonzero value at the beginning of skeleton execution. This is likely to produce many more lines of output than setting and clearing NOPT$.

Searching for a Point to Continue Evaluation

There are three instances where SSG searches for an AND, OR, or XOR Boolean operator as the point to continue evaluation:

- When attempting to satisfy any reference in any simple or compound condition results in a NO FIND.
- When a result is false and the next operator is AND (steps 2a and 4a).
- When the result is true and the next operator is OR (step 2b).

Note: *This search will not operate correctly if any part of the condition being ignored contains a string, variable reference, or process parameter reference that returns a Boolean operator. The search will operate correctly if the Boolean operator is supplied by an SGS reference or by a set reference.*

Example 1

This example uses the following *IF directive and evaluates identically for traditional evaluation (NEW_BOOLEAN$ = zero) and new evaluation (NEW_BOOLEAN$ = nonzero).

```
*IF B IS SET AND VALUE OF A = -1 OR NOT [X,1,1,1] = STOP
```

Six operations are performed in the following order:

1. Test if variable B is not equal to 0
2. Test if variable A = -1 (not evaluated if B = 0)
3. Result of (1) AND result of (2)
4. Test if [X,1,1,1] = STOP (not evaluated if result of (3) is true)
5. NOT the result of (4)
6. Result of (3) OR result of (5) (steps (4) and (5) are ignored if B = 0 and A = -1)

The result of (6) is the value of the *IF test.

Example 2

This example uses the following *IF directive and evaluates identically for traditional and new evaluation.

```
*IF NOT ELTA HAS COR AND COLUMN SEARCH FROM LAB,6,3,4 FOR ELTA
```

Four operations are performed in the following order:

1. Test if PCF or TCF contains an element entry with name ELTA
2. NOT the result of (1)
3. Perform a column search beginning with the sixth SGS with the label LAB, field 3, subfield 4 for the image ELTA (not evaluated if ELTA has corrections)
4. Result of (2) AND result of (3)

The result of (4) is the value of the *IF test.

Example 3

This example illustrates the difference between the traditional and the new evaluation priorities.

```
*SET TRUE
*CLEAR FALSE
*IF +TRUE OR +TRUE AND +FALSE
*DISPLAY 'TRUE WHEN NEW_BOOLEAN$ <> 0'
*ELSE
*DISPLAY 'FALSE WHEN NEW_BOOLEAN$ = 0'
*ENDIF
```

With traditional evaluation, OR and AND have the same priority so the *IF directive is evaluated from left to right. First, '+TRUE OR +TRUE' is evaluated with a result of 'true'. Second, 'true AND +FALSE' is evaluated with a final result for the *IF directive of 'false'. The message 'FALSE WHEN NEW_BOOLEAN$ = 0' will be displayed in the SSG run log.

With new evaluation, AND has a higher priority and is evaluated before OR. First, '+TRUE AND +FALSE' is evaluated with a result of 'false'. Second, '+TRUE OR false' is evaluated with a final result for the *IF directive of 'true'. The message 'TRUE WHEN NEW_BOOLEAN$ <> 0' will be displayed in the SSG run log.

Example 4

This example illustrates how, when NEW_BOOLEAN$ is nonzero, parentheses can be used to change the default order of the evaluation shown in example 3.

```
*SET NEW_BOOLEAN$
*SET TRUE
*CLEAR FALSE
*IF (+TRUE OR +TRUE) AND +FALSE
*DISPLAY 'TRUE WITHOUT PARENTHESES'
*ELSE
*DISPLAY 'FALSE WITH PARENTHESES'
*ENDIF
```

Even though AND has a higher priority than OR, the parenthetical expression will be evaluated first. First '(+TRUE OR +TRUE)' is evaluated with a result of 'true'. Second, 'true AND +FALSE' is evaluated with a final result for the *IF directive of 'false'. The message 'FALSE WITH PARENTHESES' will be displayed in the SSG run log.

Example 5

This example illustrates how, when NEW_BOOLEAN$ is nonzero, nested parentheses can be used to create complex Boolean expressions.

```
*IF NOT((+PERCENT > 50 OR EXCEPTION IS SET) AND ([*DAY] = SAT OR [*DAY] = SUN))
```

Proper use of the Boolean operators reduces skeleton processing time since you can avoid redundant nesting of *IF directives and duplication of skeleton images. (Anticipating NO FIND conditions is another way to reduce processing time.) The following example illustrates two functionally equivalent *IF constructions:

Construction 1

```
*IF [L]
      *IF [L,1,1,1] = doit
            SKEL to doit
      *ELSE
            SKEL not to doit
      *ENDIF
*ELSE
      SKEL not to doit
*ENDIF
```

Construction 2

```
*IF [L,1,1,1] = doit
     SKEL to doit
*ELSE
     SKEL not to doit
*ENDIF
```

The second construction assumes that if a NO FIND condition exists while evaluating the condition "[L,1,1,1] = doit", the condition is set to false. For a more complicated example, suppose an input SGS with label TAPE directs a skeleton to assign (@ASG) and write a tape. The file name is in field 1, subfield 1. If field 1, subfield 2 is coded as "ASG", the tape should not be released (@FREE).

The following constructions are each an attempt to solve this problem:

Construction 1
(correct, inefficient)

```
*IF [TAPE]
ASG
   *IF [TAPE,1,1,2]
          *IF [TAPE,1,1,2] = ASG
          *ELSE
              #FREE [TAPE,1,1,1].
          *ENDIF
   *ELSE
          #FREE [TAPE,1,1,1].
   *ENDIF
*ENDIF
```

Construction 2
(correct, efficient)

```
*IF [TAPE] AND NOT [TAPE,1,1,2] =
    #FREE [TAPE,1,1,1].
*ENDIF
```

5.2.25.2. Test for Variable Value Conditional

Format

$$^{*}\text{IF } variable-name \text{ IS } \begin{Bmatrix} \text{SET} \\ \\ \text{CLEAR} \end{Bmatrix}$$

Parameter

variable-name

A variable name satisfying the description in 3.2.1. Can be supplied by a string or by a variable reference, process parameter reference, SGS reference, or set reference that returns a string.

Description

You can test either a local or global variable for a nonzero (SET) or a 0 value (CLEAR). When you search for *variable-name* the local variable chain is searched first, from the most recently created to the last, and the global variable chain is searched. The first variable found with the given name is tested. If the variable does not exist, this is a test for existence conditional (see 5.2.25.3) and is false.

Example 1

```
*IF VAR IS SET
```

Example 2

```
*IF [lab,1,1,1] IS CLEAR
```

5.2.25.3. Test for Existence Conditional

Format

```
*IF expression
```

Parameter

expression

Can be a string, variable reference, process parameter reference, SGS reference, set reference, or any other defined conditional whose attempted evaluation could produce a NO FIND condition.

Description

This test checks to see if the expression exists.

- If a string is supplied, the conditional is always true.
- If a reference is supplied, SSG attempts to satisfy that reference.
  - If evaluation results in a "NO FIND" condition, the conditional is false.
  - If the reference can be satisfied and returns a SYMBOLIC value, the test is true.
  - If the reference returns a NUMERIC value, this is a test for 0 conditional (see 5.2.25.4).
- If during evaluation of any conditional a NO FIND condition exists, the conditional immediately becomes a test for existence that is always false.

Note: *A test for existence preceded by a NOT negates the test (*IF NOT expression).*

Example

```
*IF [LAB,1,2,1]
```

5.2.25.4. Test for Zero Conditional

Format

```
*IF expression
```

Parameter

expression

Can be a numeric expression, VALUE OF integer expression, or VALUE OF numeric expression.

Description

- If *expression* has a value of 0, the test is false.
- If *expression* has a nonzero value, the test is true.

Example

```
*IF +VALID
```

5.2.25.5. Test for Relation Conditional

Format

```
*IF operand-1 relation operand-2
```

Parameters

operand-n

Can be supplied by a string, variable reference, process parameter reference, SGS reference, set reference, numeric expression, VALUE OF integer expression, or VALUE OF numeric expression.

relation

Can be <, <=, =, <>, >=, or >.

Description

operand-1 is compared to *operand-2* according to the *relations* shown in Table 5–4.

Table 5–4. *IF Relations

Relation	Description
<	Tests if *operand-1* is less than *operand-2*
<=	Tests if *operand-1* is less than or equal to *operand-2*
=	Tests if *operand-1* equals *operand-2*
<>	Tests if *operand-1* is not equal to *operand-2*
>=	Tests if *operand-1* is greater than or equal to *operand-2*
>	Tests if *operand-1* is greater than *operand-2*

Types of Relational Tests

There are two types of relational tests: NUMERIC and SYMBOLIC. Both operands are evaluated and all numeric expressions and bracketed references are satisfied. Then, the type of *operand-1* determines the kind of test as follows:

- If *operand-1* is NUMERIC, *operand-2*, if it is not already NUMERIC, is converted to a NUMERIC value (an integer expression, variable name, or number) and a NUMERIC comparison is performed. To ensure a NUMERIC comparison, insert a '+' before *operand-1*.
- If *operand-1* is SYMBOLIC, *operand-2* (if NUMERIC), is converted to a SYMBOLIC value (edited) and a SYMBOLIC comparison is performed.

SYMBOLIC Comparisons Based on ASCII

Since SSG operates internally in ASCII, all SYMBOLIC comparisons are based on the ASCII collating sequence.

- If special characters (see Table 4–1) are embedded in a symbolic literal operand, enclose the operand in pairs of single quotation marks, not double quotation marks.
- Bracketed references that resolve to strings containing special characters do not need to be surrounded by pairs of single quotation marks.
- A bracketed reference that is surrounded by single quotation marks is not resolved and is treated as a symbolic literal operand.

  Note: *For SYMBOLIC comparisons, the X option on the @SSG call (see Table 2–1) does not play a role.*

Example 1

```
*IF  A+B  >  C+D
```

Example 2

```
*IF  [#P]-1 <= [L,A,B,1]
```

Example 3

```
*IF  [LAB,A-B,C,C+E] = STING
```

Example 4

```
*IF  [*A_STRING] = ''THIS VALUE''
```

Example 5

```
*IF +10 < [*NUMBER]
```

Example 6

```
*CREATE SGS: FILE ''FILE.ELT''
*IF [FILE,1,1,1] = ''FILE.ELT''
```

Example 7

```
*SET FILEVAR = 'FILE.ELT'
*IF [*FILEVAR] = ''FILE.ELT''
```

5.2.25.6. Row, Column, or Keyword Search Conditional

You can view SGSs of a given label as two-dimensional tables. A row in the table corresponds to one statement. A column corresponds to field and subfield in all statements.

The following SGSs

```
LABL  A,Q    Z,L,P    R6
LABL  B      S,T      Y
LABL  D,C
```

produce a two-dimensional table like the one shown in Table 5–5.

Table 5–5. Row/Columns Search Table

	Column					
Row	**Field 1, Subfield 1**	**Field 1, Subfield 2**	**Field 2, Subfield 1**	**Field 2, Subfield 2**	**Field 2, Subfield 3**	**Field 3, Subfield 1**
Statement 1	A	Q	Z	L	P	R6
Statement 2	B		S	T		Y
Statement 3	D	C				

Format

*IF { ROW / COLUMN / KEYWORD } SEARCH ({ ASC / ASCENDING / DESC / DESCENDING }) FROM ;

label(,*start-stmt*)(,*start-field*)(,*start-subfield*) FOR *expression*

Parameters

ROW, COLUMN, or KEYWORD

Specifies the type of search to be performed. One of the three types must be specified. Can be supplied by a string, process parameter reference, or variable reference that returns a symbolic value.

ASC, ASCENDING, DESC, or DESCENDING

Optional keywords that specify the direction of the search. ASC and ASCENDING specify an ascending search. DESC and DESCENDING specify a descending search. Can be supplied by a string, process parameter reference, or variable reference that returns a symbolic value.

label

Can be one of the following:

- An SGS label (see 3.1.1)
- PCF*xxxx* (affected by the value of PCFNAME$, see 6.4 and 6.7)
- TCF*xxxx* (affected by the value of TCFNAME$,)
- P (always primary PCF set)
- T (always primary TCF set)
- $SGS (see 3.1.3)
- $PCF (see 6.6)
- $TCF

Can be supplied by a string or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

start-stmt

The statement number (see 3.1.2) with the specified *label* where the search is to begin. Can be supplied by an integer expression or numeric expression.

start-field

The field number (see 3.1.2) on the specified *start-stmt* with the specified *label* where the search is to begin. Can be supplied by an integer expression or numeric expression.

start-subfield

The subfield number (see 3.1.2) in the specified *start-field* on the specified *start-stmt* with the specified *label* where the search is to begin. Can be supplied by an integer expression or numeric expression.

expression

The image to be searched for. Can be supplied by a string, variable reference, process parameter reference, SGS reference, set reference, numeric expression, VALUE OF integer expression, or VALUE OF numeric expression. All NUMERIC expressions are evaluated. The result is converted to a SYMBOLIC string (edited) for the search.

Description

If the ASC, ASCENDING, DESC, or DESCENDING keywords are omitted, an ascending search is assumed. The *start-stmt*, *start-field*, and *start-subfield* can be omitted and, if omitted, are assumed to have the value 1.

- Row search

  A row search starts from a particular statement, field, and subfield for a given *label*.

  - If an ascending search is specified or assumed, the search proceeds to the next subfield in the current field. If the current subfield is the last in the current field, the search proceeds to the first subfield in the next field.
  - If a descending search is specified, the search proceeds to the previous subfield in the current field. If the current subfield is the first in the current field, the search proceeds to the last subfield in the previous field.
  - If a descending search is specified, *start-field* and *start-subfield* can be arbitrarily large values. In this case, SSG begins the search from the specified *start-stmt*, the highest field, and the highest subfield.
  - The search compares each subfield to the specified *expression*.
    - If a match is found, the conditional is true and SSG sets the values of the reserved global variables FLD$ and SFLD$ to the field and subfield numbers (see 3.1.2), respectively, where the match is made.
    - If no match is found, the conditional is false and the values of FLD$ and SFLD$ remain unchanged.

- Keyword search

  A keyword search functions the same as a row search, except only the specified subfield of each field is compared to the specified *expression*. For example, if subfield 1 is specified on the keyword search, the value of subfield 1 of each field on the specified SGS statement is compared to the specified expression.

  - If an ascending search is specified or assumed, the search proceeds to the next field.
  - If a descending search is specified, the search proceeds to the previous field.
  - If a descending search is specified, *start-field* can be an arbitrarily large value. In this case, SSG begins the search from the specified *start-stmt*, the highest field, and the specified *start-subfield*.
  - FLD$ and SFLD$ are set the same as in a row search.

- Column search

  A column search starts from a particular statement, field, and subfield for a given *label*.

  – If an ascending search is specified or assumed, the search proceeds to the next statement.

  – If a descending search is specified, the search proceeds to the previous statement.

  – If a descending search is specified, *start-stmt* can be an arbitrarily large value. In this case, SSG begins the search from the highest statement, the specified *start-field*, and the specified *start-subfield*.

  – The search compares each subfield, if it exists, to the specified *expression*. If the field and subfield do not exist, an ascending search proceeds to the next statement, and a descending search proceeds to the previous statement.

    o As soon as a match is found, the conditional is true and SSG sets the value of the reserved global variable STMT$ to the statement number (see 3.1.2) where the match is made.

    o If no match is found, the conditional is false and the value of STMT$ remains unchanged.

Note: **IF COLUMN (ROW,KEYWORD) SEARCH is more efficient than incrementing (*INCREMENT; see 5.2.26) through columns (rows,keywords) and performing a test for equality (see 5.2.25.5). *IF [SGS label] is more efficient than *IF COLUMN SEARCH FROM $SGS FOR SGS label.*

Example 1

```
*IF COLUMN SEARCH FROM [#1],[#2],[#3],[#4] FOR [#6]
```

Example 2

```
*IF ROW SEARCH FROM LAB,I FOR STRING
```

Example 3

```
*IF KEYWORD SEARCH FROM LABEL,1,2,I FOR STRING
```

Example 4

```
*IF COLUMN SEARCH DESCENDING FROM ENTRY,9999999,4,3 FOR [DATE$,1,5,1]
```

5.2.25.7. Test for Element Entry Conditional

Format

$$^{*}\text{IF} \quad element-name\ (/version-name)\ \text{HAS (ANY)} \quad \left(\left\{\begin{matrix}\text{PERM}\\ \\ \text{TEMP}\end{matrix}\right\}\right)\ \text{COR(RECTIONS)}$$

Parameters

element-name/version-name

The name of an element entry satisfying the description in 6.3. Can be supplied by a string or by a variable reference, process parameter reference, SGS reference, or set reference that returns a string.

Description

You can direct SSG to search for the existence of an element entry. The PCF and TCF sets searched depend on how you specify ANY, PERM, and TEMP (see Table 5–6).

Table 5–6. IF PCF TCF Search

Specification			Search Results
		COR	The primary PCF and TCF sets are searched.
	PERM	COR	The primary PCF set is searched.
	TEMP	COR	The primary TCF set is searched.
ANY		COR	All PCF and TCF sets are searched.
ANY	PERM	COR	All PCF sets are searched.
ANY	TEMP	COR	All TCF sets are searched.

If the element entry is found, the conditional is true. Otherwise, it is false.

An **element-name/version-name* image in an input stream (or the existence of a program file element when using the P or T options) causes an element entry to be created.

If an element entry is created with the *CREATE, *CORRECT, or *MERGE directives, the *IF test for that entry's existence is true.

Example 1

```
*IF  [NAME,1,1,1] HAS ANY COR
```

Example 2

```
*IF  ELT/VER HAS PERM COR
```

Example 3

```
*IF  NOT ELT/VER HAS ANY TEMP COR
```

5.2.26. *INCREMENT and *LOOP

Increment loops can repetitively interpret a block of skeleton images. A local variable (see the *variable-name* parameter) is created and exists for the life of the increment loop. The variable name can then be referenced freely in the loop. A local variable name can duplicate the name of a global variable. In such an instance, references to *variable-name* result in the local variable value.

Format

*INCREMENT *variable-name* (FROM *from-value*) (TO *to-value*) ;

(BY $by-value$) (WHILE $variable-name$ IS $\left\{\begin{matrix}\text{SET}\\ \\ \text{CLEAR}\end{matrix}\right\}$

($skeleton\ images$)
*LOOP

Parameters

variable-name

Can be supplied by a string satisfying the description in 3.2.1 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

from-value
to-value
by-value

Is an integer expression or numeric expression.

Description

An *INCREMENT directive requires a matching *LOOP directive. The images between an *INCREMENT directive and its matching *LOOP directive constitute the loop. Each execution of an increment loop directs execution (images are interpreted from top to bottom). You can nest loops statically, by placing one loop in another, and dynamically (*PROCESS) to any level.

The FROM, TO, BY, or WHILE phrases can be in any order or omitted. When omitted, *from-value*, *to-value*, and *by-value* are assumed to be 1, and no WHILE-phrase checks are performed.

The *INCREMENT directive creates a local variable of the name specified and initially sets its value equal to *from-value* (or 1). The first iteration of the increment loop is performed if *from-value* has not exceeded *to-value* and the WHILE phrase is true (or absent). Note that *from-value*, *to-value*, and *by-value* are evaluated only once, on the first iteration of the loop. If variables or bracketed references are contained in the *from-value*, *to-value*, or *by-value* expressions, changes to the variables or bracketed references during loop processing do not affect the tests that are used at the end of each iteration to determine if the loop should be terminated.

After the first execution of the increment loop, the value of *variable-name* is incremented by *by-value* (or 1). Then, if the new value of *variable-name* has not exceeded *to-value* and the WHILE phrase is true (or absent), the next iteration takes place. This increment test process is repeated after every iteration. When a test fails, the increment loop is terminated, image interpretation ends, and the local variable is removed. The next image is then the first noncomment skeleton image following the *LOOP directive.

The value of *variable-name* exceeds *to-value* if

```
( (present value of variable) - (to-value) ) x (by-value) > 0
```

With the above test, you can set the *by-value* negative. SSG allows for a decrementation process. The WHILE phrase tests the specified variable (local or global) in that phrase to see if its value is 0 (CLEAR) or nonzero (SET).

Notes:

- *If the name of a global variable is used to supply the* variable-name *in the WHILE phrase, the global variable should not be removed (*REMOVE; see 5.2.35) nor should it be changed to a string type variable (*SET, Format 2; see 5.2.39) until the loop terminates.*
- *The local variable should not be changed to a string variable (*SET, Format 2; see 5.2.39).*

Example 1

To repetitively interpret a block of skeleton images until some condition is false:

```
*Increment LOOP to very big number
   *If condition
      DO something
   *ELSE
      *EXIT LOOP
   *ENDIF
*Loop
```

Evaluating *condition* may depend on the value of LOOP.

Example 2

To scan all user defined SGS labels:

```
*Increment S from SSGSGS$+1 to [$SGS]
   Handle [$SGS,S,1,1]
*Loop
```

You can scan all PCF and TCF set names by removing the FROM phrase and replacing $SGS with $PCF or $TCF, as desired.

Example 3

To scan all user defined SGS labels when you might remove (*REMOVE) all SGSs with a given label:

```
*Increment S from [$SGS] to SSGSGS$+1 by -1
   Handle or remove [$SGS,S,1,1]
*Loop
```

The scan is done backwards to alleviate conflicts with $SGS. See 6.2 for a discussion of the effect that removing SGSs has on the SGS label chain. Since the *REMOVE directive does not affect what PCF and TCF sets are defined, there is probably no need to scan $PCF and $TCF backwards.

Example 4

To create an index set:

```
*Increment L to [label]
   *If [label,L,...] should be handled
       *Create SGS:  INDEX some function of L and label
   *ENDIF
*Loop
```

Example 5

To access data through an index set:

```
*Increment I to [INDEX]
   Handle [label,[INDEX,I,1,1],...]
*Loop
```

For more information, see 5.2.14.

5.2.27. *ITERATE

The *ITERATE directive allows the next iteration of a loop to take place at any point in the loop.

Format

```
*ITERATE (varname)
```

Description

The *ITERATE directive increments the loop control variable. If the TO value has not been reached and the WHILE clause is still true, the next iteration of the loop takes place.

- If *varname* is not specified, the iteration takes place on the innermost loop.
- If *varname* is specified, the loop using the loop control variable specified is iterated.
- If *varname* does not exist, a NO FIND occurs and the directive is ignored.

Example

```
*INCREMENT A TO 50
   *INCREMENT B TO 10
      *IF +A>30 AND +A<40 AND +B=6
         *ITERATE A
      *ENDIF
      [*A],[*B]
   *LOOP
*LOOP
```

5.2.28. *LIST

The *LIST directive provides skeleton listing control.

Format

```
*LIST
```

Description

The *LIST directive gives you control over the listing of the skeleton. The *LIST directive turns listing of the skeleton on in the presence of the E or D options on the SSG call. An initial *LIST directive is assumed at the start of each skeleton. No directive is required to initially start skeleton listing.

5.2.29. *LOOP(,E)

The *LOOP directive signifies the end of an *INCREMENT block (see 5.2.26). The E option causes a page eject following the *LOOP.

5.2.30. *MERGE

The *MERGE directive merges temporary streams.

Format

```
*MERGE(,options) element-name(/version-name) (TCFxxxx) ;
(WITH TCFyyyy) (GIVING TCFzzzz)
```

Parameters

options

Can be a string or a variable reference, process parameter reference, SGS reference, or set reference that returns a string. Valid values are

none — In the absence of a (possibly assumed) GIVING phrase, the result of the correct operation is directed to the generated output stream.

K — Used in conjunction with the T option or GIVING phrase to additionally direct the images resulting from the merge operation to the generated output stream.

R — Causes the merge to be a relative merge.

T — Is ignored if TCF is not a T-option TCF set.

If TCF*zzzz* is not specified, the T option forces SSG to tentatively assume that a GIVING TCF*yyyy* phrase was specified. It is a tentative assumption because, if TCF*yyyy* is not a T-option TCF set, the assumption is not made. If the assumption is made, it is the same as if GIVING TCF*yyyy* was specified on the *MERGE directive.

If TCF*zzzz* (possibly assumed to be TCF*yyyy*) is a T-option TCF set, the images resulting from the merge are sent to the program file attached to TCF*zzzz*. A new symbolic element is created (the old one, if it existed, is replaced) with a name equal to the name of the specified element entry (see 6.5)

element-name/version-name

Specifies the name of the element entry from which the correction streams are to be taken. Can be supplied by a string satisfying the description in 6.3 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

TCFxxxx

Specifies the input temporary stream. If it is omitted, there is no input temporary stream. TCF*xxxx*, if specified, must be a defined TCF set. Can be supplied by a string satisfying the description in 6.5 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

TCFyyyy

Specifies the input "permanent" stream. (This stream is acted upon as if it were a permanent stream from a PCF.) If it is omitted, there is no input "permanent" stream. TCF*yyyy*, if specified, must be a defined TCF set. Can be supplied by a string satisfying the description in 6.5 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

TCFzzzz

Specifies the output temporary stream. If it is omitted and if the T option is specified on the *MERGE directive and TCF*yyyy* is a T option TCF set, TCF*yyyy* is assumed; otherwise, the results of the merge are always sent to the generated output stream. Can be supplied by a string satisfying the description in 6.5 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

Description

All phrases are optional except *element-name*/*version-name* and one TCF set.

*MERGE directs SSG to merge zero, one, or two TCF streams. In the normal case, when two input streams are specified, they are given the names input temporary (TCF*xxxx*) and input "permanent" (TCF*yyyy*) for two reasons. First, if a conflict occurs, corrections from the input "permanent" stream are discarded. In other words, temporary overrides "permanent". Second, when the R option is used, a relative merge is performed.

A relative merge allows the input temporary stream to modify the input "permanent" stream. This is similar to the way an input temporary stream modifies an input permanent stream when processed with the *CORRECT directive. (Note, however, that a *MERGE directive does not require a matching *ENDCORRECT directive.) If a specified element entry does not exist, the merge consists of the remaining specified entry. If both input streams are omitted, the output resulting from the merge is an empty stream. This creates an element in TCF*zzzz* and attaches an empty stream to it.

SSG ignores the T option for an element entry that does not exist in TCF*zzzz* (TCF*yyyy* if assumed). To create a symbolic element in an attached program file for an element that does not exist as an element entry, specify a GIVING phrase on the *MERGE,T directive. In the absence of the GIVING phrase, the T option is ignored. Normally, an element in an attached program file is revised and reinserted into the program file, simulating an update process. In either case, place a GIVING phrase on the *MERGE,T directive to ensure modification of the attached program file.

If you do not specify the R option, SSG combines the two input streams. Line change statements in both streams are based on a PCF stream and base level symbolic element. (The result of this merge will be used to revise that PCF stream.) The corrections are put in sequential order. (Each correction stream must be in sequential order before the merge.) In this type of merge, TCF*xxxx* does not make relative corrections to TCF*yyyy*. All relative corrections in both streams are based on a PCF stream. Sequence errors are flagged and the associated correction images are discarded. In the case of conflicts, the input temporary stream takes precedence over the input "permanent" stream. You can use the reserved, global variable CONFLC$ (see Table 3–9) to monitor this.

If you specify the R option on the *MERGE directive, SSG assumes that the input "permanent" stream has already been merged with a PCF stream, producing a revised PCF stream. SSG also assumes relative corrections in the input temporary stream are relative to the revised PCF stream. SSG performs a merge between the input temporary stream and the revised PCF stream, producing a new revised PCF stream. The output of the merge is the TCF stream needed to revise the original PCF stream. Sequence errors are flagged and the associated correction images are discarded. In the case of conflicts, the input temporary stream takes precedence over the input permanent stream. You can use the reserved, global variable CONFLC$ (see Table 3–9) to monitor this. This is a powerful merge with interesting applications. For an application of *MERGE,R, refer to Appendix B.

Note: *When you are resolving line number conflicts, the WITH phrase of *MERGE specifies the input temporary stream that has the same priority level as if it were a permanent stream.In addition, where *CORRECT assumes PCF and a matching TCF set, *MERGE makes no such assumption. Use of the *CORRECT,P and *MERGE directive is analogous. Compatibility with previous levels of SSG is retained when logic dictates.*

Restrictions

Several restrictions apply when using *MERGE because of conflicts that occur and erroneous merges that are accepted. See 5.2.7 for *CORRECT-related restrictions.

- -N-conflicts with -N when merging corrections. For example, an input temporary stream that looks like this

  ```
  *element-name/version-name
  -577-
  -577
  ```

  produces the following diagnostic messages:

  ```
  ERROR IN PROCESSING ELEMENT element-name/version-name
  SEQUENCE OR FORMAT ERROR ON FOLLOWING IMAGE -> ;
  POSSIBLE MERGE INTERNAL CONFLICT
  -577
  ```

  The preceding diagnostic indicates an error because the -577- and -577 images are in the same temporary stream. The last image, -577, is displayed and does not appear in the output temporary stream.

If the images are each in different temporary streams, a merge produces a conflict, not an error. A conflict produces the following diagnostic messages:

```
CONFLICT IN RELATIVE CORRECTIONS: *MERGE facsimile
-577(-)
```

The image listed in the message, either -577 or -577-, is derived from the input permanent stream and does not appear in the output temporary stream.

- Conflicts—The following correction lines result in a conflict:

```
-n,m/0,0/n-k,new-m
-greater-than-new-m-but-less-than-m,does-not-matter
```

For example, the following construction would not work:

```
-10,20/0,0/5,15
-17,18
```

In this example, -17,18 conflicts with -10,20 and, due to the nature of a merge, is treated as an error. However, *CORRECT can correctly handle this TCF stream. The following construction can circumvent the problem:

```
-10,20/0,0/5,15
-10,20/n/17,18
```

In this example, *n* is the number of data images following -10,20.

- An erroneous merge may be accepted given the following two TCF streams

```
A)   *ELT               B)  *ELT
     -2,2/0,0/3,3           -2,2/0,0/3,3
     -2/0,0/1,1             *ELT
                              -2/0,0/1,1
```

and given the two possibilities for *MERGE

```
1)  *MERGE              2)  *MERGE,R
```

The only combination that correctly issues a sequence error message is B2. The others produce this construction:

```
*ELT
-2,2/0,0/3,3
-2/0,0/1,1
```

This TCF stream is erroneous and always rejected by *CORRECT.

- *MERGE,R restrictions—The possible temporary streams shown in Table 5–7, input to MERGESKEL in SYSKEL utility file, all produce erroneous results. These constructs are not often used. If you encounter the incorrectly generated result, correct the runstream as indicated before continuing.

Table 5–7. Input to MERGESKEL

Input	Generated Result	Corrected Input
*ELT -1/4/5 TLine1 *ELT -5/1/6	*ELT -1/4/5 TLine1 -1/-1/6	*ELT -1/4/5 TLine1 -1/4/6
*ELT -1/4 -1/4/5 TLine1 *ELT -5/1/6	*ELT -1/4 -1/4/5 TLine1 -1/0/6	*ELT -1/4 -1/4/5 TLine1 -1/4/6
*ELT -1/4 -1/4/5 TLine1 *ELT -5/1,1/6	*ELT -1/4 -1/4/5 -1/0/6	*ELT -1/4 -1/4/5 -1/4/6
*ELT -1/4 -1/4/5 TLine1 *ELT -5/1,2/6	*ELT -1/4 -1/4/5 -1/9,5/6	*ELT -1/4 -1/4/5 -1/5,5/6

5.2.31. *MESSAGE

Format

```
*MESSAGE(,options) text
```

Parameters

options

Can be supplied by any combination of characters and, if LBRK$ is clear, bracketed references resolving to a string (see Table 5–8).

Table 5–8. MESSAGE Options

Option	Description
E	Increments the value of ERRCNT$ by 1.
I	Displays this message in the SSG run log. *****MESSAGE (,*options*) *text*
N	Prevents the display of the message in the SSG run log.
L	Generates an ER ACSF$ of the following image: @LOG SSG*MESSAGE (,*options*) *text*
O	Displays the message upon the operator console. The T option is assumed when the O option is specified.
T	Affects both contents of the message displayed in the SSG run log and the text of the @LOG image submitted to ACSF$. The image used consists solely of the text portion of the message, where text refers to those characters following the first space character after *options*. Provides for exclusively user-formatted messages.
X	Increments the value of ERRCNT$ by one. Aborts the run via ER ERR$. An ER ERR$ is not performed if the run is a demand run, the run is not in @BKRPT mode and the last image in the runstream read by SSG was not an @ADD image. In any case, SSG always unconditionally terminates processing.

text

Can be supplied by any combination of characters and if LBRK$ is clear, by bracketed references. The text of the message, including (,*options*), is truncated at 120 characters. SSG processes *text* as if it is a quoted string, even though quotation marks are not required and should not be specified. Continuation is not allowed anywhere in the *MESSAGE directive, except within bracketed references, if LBRK$ is clear. If a semicolon is included in the image it is processed as part of the *text*. If a semicolon is included and if the L option is specified, an ER (error type 04) error code 040 contingency occurs.

Description

Processing a *MESSAGE directive is transparent. You can place this directive anywhere in the skeleton except between an image and its continuation image.

In the absence of the N or I options, SSG uses the value of IOPT$ to control message display in the SSG run log. In other words, the message is displayed if IOPT$ has a nonzero value.

Note: *You can also use the I and N options to override (not change) the current value of IOPT$.*

After the message is or is not displayed, the L option is processed. Then the E and X options are processed. If the E and X options are specified together on the same *MESSAGE directive, the value of ERRCNT$ is incremented once by 1.

Note: *Using the *MESSAGE directive to display a message in the SSG run log is much more efficient than using *CREATE to create SGSs.*

Example 1

```
*MESSAGE, IT  *WARNING*  [#1] is ignored because ...
```

Example 2

```
*MESSAGE, IET  *ERROR* ...
```

Example 3

```
*MESSAGE, XT  *FATAL ERROR* ...
```

Example 4

```
*MESSAGE, NO  WRITE RING IN 532 PLEASE
```

5.2.32. *MULTIPLY

The *MULTIPLY directive performs variable multiplication.

Format

```
*MULTIPLY number BY number GIVING variable-name
```

Parameters

number

Is an integer expression or numeric expression.

variable-name

Can be supplied by a string satisfying the description in 3.2.1 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

Description

The product of the first *number* multiplied by the second *number* is the value of the given *variable-name*. If no local variable with that name exists, a global variable is searched for. If no global variable with that name exists, one is created. Once created, this global variable exists until it is removed (*REMOVE; see 5.2.35).

Note: *If evaluation of either number results in an arithmetic overflow or if the multiplication produces an arithmetic overflow, an error diagnostic is displayed in the SSG run log and the largest or smallest appropriate representable value is used as the result.*

Example 1

```
*MULTIPLY    A+B BY C-D GIVING E
```

Example 2

```
*MULTIPLY    [#A]+[B,1,C+D,2] BY 7 GIVING [#P]
```

5.2.33. *PCONTROL

The *PCONTROL directive allows the SSG user to perform an ER APRTCN$.

Format

```
*PCONTROL 'APRTCN$ image'
```

Description

After all bracketed references have been satisfied, use the image enclosed in quotation marks as the print control image for an ER APRTCN$. To obtain a single quotation mark in the print control image, use pairs of single quotation marks.

Example 1

```
*PCONTROL  'd,@@ESC O'
```

Example 2

```
*PCONTROL  'H,,,[Heading,1,1,1,SGS_STRING$,99,99] '
```

Example 3

```
*PCONTROL  'H,,,that''s the last one'
```

5.2.34. *PROCESS

Format 1

```
*PROCESS(,option) define-name (proc-para-1 proc-para-2 ...)
```

Format 2

```
*define-name (proc-para-1 proc-para-2 ...)
```

Parameters

option

A Ignores error if NO FIND occurs.

define-name

For format 1, this parameter can be supplied by a string satisfying the description in 5.2.10 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string. For format 2, no references are allowed. SKEL$ reinvokes the control section of the skeleton.

proc-para

Can be supplied by a string, variable reference, process parameter reference, SGS reference, set reference, or numeric expression. If NEW_PROCESS$ is set to 0, a single asterisk (*) followed by a bracketed reference directs the evaluation of the string that satisfies an indirect reference.

Description

The *PROCESS directive directs SSG to retrieve and process a define packet and passes process parameters to that packet.

If an asterisk (*) is found in column 1, and the directive following it is not an SSG directive, a *PROCESS directive is assumed. This allows for calls of define packets. Thus, *XYZ is the same as *PROCESS XYZ. (The *define-name* for the assumed *PROCESS directive call cannot contain process parameter, SGS, set, or variable references.)

If NEW_PROCESS$ is set to zero, SSG examines all the parameters specified on the *PROCESS directive, evaluating all numeric expressions and references and converting any resulting numeric values to their symbolic representations. Usually, only SYMBOLIC parameters are passed to a define packet. If an SGS reference or set reference supplying a process parameter results in a NO FIND condition, and the A option is specified on the @SSG call, a NO FIND process parameter is created. A NO FIND process parameter results in a NO FIND condition when referenced. However, a test for existence conditional can be used to test for such a process parameter.

If NEW_PROCESS$ is nonzero, SSG satisfies all bracketed references, and then creates an SGS with a label of # that may be referenced in the define packet like an SGS. However, this # SGS cannot be referenced by using the $SGS reserved label.For information on referencing *PROCESS parameters within a define packet, refer to 3.3.

Example 1

```
*PROCESS  NAME
```

Example 2

```
*DRAW_SCALE
```

Example 3

```
*PROCESS  [NAME,1,1,1]
```

Example 4

```
*PROCESS  MULTIPLY  A+B  C-D  E+F+G
```

Example 5

```
*PROCESS  VERY_LONG_DESCRIPTIVE_NAME
```

Example 6

```
*SET_MARGINS    88,8,4
```

5.2.35. *REMOVE

The *REMOVE directive performs dynamic contraction of internal chains.

Format 1

```
*REMOVE SGS sgs-label(,sgs-start-stmt(,number-to-remove))
```

Format 2

$$\text{*REMOVE} \quad \left\{\begin{matrix}\text{PERM} \\ \\ \text{TEMP}\end{matrix}\right\} \quad element-name(/version-name) \ (\text{FROM } set-name)$$

Format 3

```
*REMOVE VARIABLE variable-name
```

Parameters

sgs-label

Specifies the SGSs to be removed. Can be supplied by a string satisfying the description in 3.1.1 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

sgs-start-stmt

Specifies the statement number with the given *sgs-label* where the removal will start. Can be an integer expression or a numeric expression. This number is optional (1 is assumed if omitted).

number-to-remove

Specifies the number of SGSs to remove. Can be an integer expression or a numeric expression. This number is optional (1 is assumed if omitted); *number-to-remove* can evaluate to zero.

element-name/version-name

Specifies the element entry to be removed from *set-name*. Can be supplied by a string satisfying the description in 6.3 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

set-name

Specifies the PCF (PERM) or TCF (TEMP) set from which the specified element entry is to be removed. Can be supplied by a string satisfying the descriptions in 6.4 (PERM) or 6.5 (TEMP) or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

variable-name

Specifies the global variable to be removed. Can be supplied by a string satisfying the description in 3.2.1 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

Description

The *REMOVE directive performs three functions.

- Removes a specified number of SGSs with the specified SGS label starting at the first or specified SGS statement.

  If there are no SGSs within the range specified, no SGS are removed. If there are no SGSs with label *sgs-label* left before or after the attempted removal, *sgs-label* is removed from the SGS label chain, which can affect references using $SGS. This means if *sgs-label* occupied the *i* th slot in the SGS label chain ([$SGS,*i*,1,1]), it no longer does; and [$SGS,*i*,1,1] either results in a NO FIND condition or returns the SGS label that occupied slot *i*+1; [$SGS,*i*,1,1] becomes what was [$SGS,*i+1*,1,1].

- Removes the element entry with the specified name from the PERM chain (the primary PCF set is assumed in the absence of *set-name*) or TEMP chain (the primary TCF set is assumed in the absence of *set-name*).

  Once removed, the element entry and any attached correction stream are destroyed during the SSG execution. This change is internal to SSG. The files used for input are not changed. If *set-name* is supplied, it must match the PERM or TEMP keyword supplied. If the element entry does not exist, the *REMOVE directive is ignored. The PCF or TCF *set-name* is not removed from *set-specified*, the PCF or TCF set chain, even if there are zero element entries left in *set-name* before or after removal.

- Removes *variable-name* from the global variable chain.

  If *variable-name* does not exist in the global variable chain, the *REMOVE directive is ignored. If a reserved global variable is removed, SSG retains the last value associated with it, even if it is redefined (see use of the $ character in 3.2.3). A global variable specified on the WHILE phrase of an *INCREMENT directive should not be removed while processing that increment loop.

The *REMOVE directive makes available for reuse parts of the D-bank of the SSG processor that were used to store data no longer needed.

Example 1

To remove all SGSs with a given *label*:

```
*REMOVE SGS label,1,[label]
```

This also removes label from the SGS label chain.

Example 2

```
*CREATE SGS:  NAME  ELT/VER
*REMOVE [NAME,1,1,1]
```

Example 3

```
*REMOVE  ELT/VER FROM TCFXYZ
```

5.2.36. *RESTORE

The *RESTORE directive provides a method of restoring an SGS environment that was previously saved via the *SAVE directive.

Format

*RESTORE(,*option*) SGS FROM *file.*(*element*)

Parameters

option

A Do not terminate in error if the source does not exist or if SSG did not create the source.

file.(element)

Restore the SGS environment from the file with this name.

Description

The *RESTORE directive provides a method of establishing a previously saved environment in SSG. The keywords SGS and FROM are required before the file specification. The file specification must be in system standard file name notation. The information in the defined source must have been created by SSG. If the A option is not specified, SSG terminates if the specified file or element does not exist or was not created by SSG. Any number of SGS sets can be restored in a given skeleton.

As the SGSs are restored, they are always added to the end of the chain of SGSs with the same label in the same order in which they were saved. The SGSs are saved and restored in SSG internal format; they are never printed in the SSG run log.

Note: *With the *RESTORE directive, the values of the reserved global variables SOPT$ and IOPT$ are ignored.*

5.2.37. *RETURN

The *RETURN directive allows early termination of a define packet or the main skeleton.

Format

```
*RETURN (STOP)
```

Description

If *RETURN is encountered in a define packet, control is returned to the statement following the define packet call (*PROCESS). If *RETURN or *RETURN STOP is encountered in the main skeleton or if *RETURN STOP is encountered in a define packet, normal skeleton termination takes place. All remaining directives are skipped and the generated output stream is handled as described in 2.4.

Example

```
*IF error IS SET
   *RETURN
*ENDIF
```

If the variable ERROR is nonzero, the skeleton execution is stopped and normal exit is taken.

5.2.38. *SAVE

The *SAVE directive provides a method of saving the SGS environment.

Format

*SAVE SGS (LABEL,*label*) IN *name*

Parameters

label

Is an SGS label; only the SGSs with this label will be saved. If no SGSs for the specified label exist, processing will continue with the next directive line.

name

Can be a file or an element, and must be in standard *filename.eltname* notation. Use any combination of characters. If LBRK$ is clear, use bracketed references. If the file does not exist, SSG creates a temporary file. If a program-file element already exists, then the existing element will be marked for deletion.

Description

The *SAVE directive saves the user-defined SGS environment or the SGSs with the indicated label in the location specified on the directive line. The information is saved in omnibus format. None of the SGSs defined by SSG are saved by the *SAVE directive. The LABEL keyword must be specified if a one-label save is desired; otherwise, the entire user-defined SGS environment will be saved.

5.2.39. *SET

The *SET directive creates and changes variable values.

Format 1

```
*SET variable-name (TO number)
```

Format 2

```
*SET variable-name (TO 'string')
```

Parameters

variable-name

Can be supplied by a string satisfying the description in 3.2.1 or by a variable reference, process parameter reference, SGS reference, or set reference that returns such a string.

number

Is an arithmetic expression (see Table 4–1). An undefined variable within an expression has a value of 0. If the expression contains a bracketed reference to an undefined variable, then a NO FIND error is generated.

'string'

Can be any combination of characters. If LBRK$ is clear, all bracketed references will be satisfied. To obtain a single quotation mark in the string, two single quotation marks should be used.

Description

An equal sign (=) can be used instead of the keyword TO. If the TO or = phrase is omitted, *number* is assumed to be 1.

The value of the last local variable with the given name is set to *number*. If no local variable with the given name exists, a global variable with that name is searched for and set to number. If no global variable with that name exists, one is created and given the value number. Once created, this global variable exists until it is removed (*REMOVE).

Use the *SET directive to create or change the value of a numeric or string variable. To allow the use of nonprintable ASCII characters in string variables, the construct C '*octal-number*' is also allowed. The number is left justified and zero-filled to the nearest quarter word (9 bits).

Example

```
*SET ESC = C'033'
*SET CLEAR_SCREEN = '[*ESC]e[*ESC]M'
```

This example results in the variable CLEAR_SCREEN having the following internal ASCII value:

```
033145033115
```

That value can then be printed by the following entry:

```
*DISPLAY clear_screen
```

You can also use the *SET directive to make numeric computations. The arithmetic operators +, -, /, and * are recognized in expressions; spaces between operands and operators are ignored. The expression is evaluated from left to right, with multiplication and division performed before addition and subtraction. In order to allow for hexadecimal, octal, and binary arithmetic, the constructs H'*hex-number*', O'*octal-number* ' and B'*binary-number* ' are allowed.

Note: *If evaluation of number results in an arithmetic overflow, an error diagnostic is displayed in the SSG run log and the largest or smallest representable value, as appropriate, is used.*

The *SET directive is scanned twice with all bracketed references being satisfied on the first scan. You can use it to substitute an equation, used several times, for a *SET directive.

Example 1

```
*SET EQUATION = '3*Radius*Radius'
*SET AREA1 = [*EQUATION] + 10
      .
      .
      .
*SET AREA6 = [*EQUATION] + 13
```

Example 2

```
*SET  VAR
```

Example 3

```
*SET  VAR TO A+B-C *3 +2
```

Example 4

```
*SET  [VAR,I,1,1]
```

Example 5

```
*SET  [#P] to [VAR,I,1,1]+VAR
```

Example 6

```
SET VALUE = 23 * O'17' + H'F' / B'101'
```

Example 7

```
*SET STRING = 'This is a string variable'
```

Example 8

```
*SET STRING_TWO TO ' [*STRING], it is very nice'
```

Example 9

```
*SET NUMBER = H'E325' * H'1A' - O'52'
```

Note: *Be careful when putting a string variable reference within a string variable set since a single quotation mark anywhere in the variable reference terminates scanning.*

Example 10

```
*SET ST1 = 'Where''s the fire'
*SET ST2 = '[*ST1] truck? '
```

This example sets the value of ST1 to

```
Where's the fire
```

and the value of ST2 to

```
Where
```

Example 11

```
*SET ST1 = 'Where''''s the fire'
*SET ST2 = '[*ST1] truck? '
```

This example sets the value of ST1 to

```
Where''s the fire
```

and the value of ST2 to

```
Where's the fire truck?
```

5.2.40. *SORT

The *SORT directive allows dynamic sorting of SGSs, PCFs, and TCFs.

Format

$$^{*}\text{SORT}(,options)\ \begin{Bmatrix}\text{SGS}\\ \text{TCF}\\ \text{PCF}\end{Bmatrix}\ (name)$$

Parameters

options

Can be supplied by any combination of characters and, if LBRK$ is clear, bracketed references are resolved.

A Do not terminate in error if the item to sort does not exist.

D Sort the values in descending order.

L Sort across all fields.

X Sort numerically. The subfield values must be numeric characters.

Note: *This numeric sort is different from the alphanumeric sort performed with the @SSG call line X option shown in Table 2–1.*

name

For sorting TCF or PCF sets, the set name is used. For sorting SGSs, the following format of the NAME specification is used:

```
(label(,start-sgs(,start-field(,start-subfield))))
```

If *start-sgs*, *start-field*, or *start-subfield* are omitted, the default value of 1 is used. If the *name* parameter is omitted, then all labels or sets are sorted.

Description

The *SORT directive allows dynamic sorting of SGSs and PCF/TCF set entries. Sorting is done in ascending order, unless otherwise requested by the D (descending) option.

- For PCF/TCF sets, sorting is always done on field 3 (the concatenated element/version name). Duplicates are not removed as they are with the Q or O options.
- For SGSs, *start-sgs* is the first statement number to sort. All statements following *start-sgs* will be sorted; *start-field* and *start-subfield* indicate the subfield position across which the sorting will take place.

- If the L option is specified and a tie occurs while sorting, the following fields are used to determine the proper order. If a field or subfield does not exist, it is taken to be a low value.

  Note: *The L option does not sort PCF/TCF sets. This option has no meaning if the X option is specified.*

Following a *SORT directive, the *ACCEPT, *ADD, *CREATE, and *RESTORE directives can be used to add new SGSs to the sorted chain of SGSs with the same label, with these constraints:

- Once you add new SGSs, the chain may not be in the same order as before. If possible, wait until all the new SGSs have been added before using the *SORT directive.
- When the global variable SOPT$ is nonzero, the order in which SGSs are added to the chain of SGSs with the same label is unpredictable under these conditions:
  - The SGS label has been sorted with the D option specified.
  - The SGS label has been sorted with a *start-field* and/or *start-subfield* value other than 1.

For more details, refer to 3.2.3.3.

Example 1

```
*SORT SGS TEST
```

Example 2

```
*SORT,L SGS Elements
```

Example 3

```
*SORT,D TCF
```

Example 4

```
*SORT,DL SGS
```

Example 5

```
*SORT PCF SET1
```

Example 6

```
*SORT,X SGS VALUES,1,3,2
```

5.2.41. *UNLIST

The *UNLIST directive provides skeleton listing control.

Format

```
*UNLIST
```

Description

The *UNLIST directive gives the SSG programmer control over the listing of the skeleton. The *UNLIST directive turns off the listing of the skeleton following the *UNLIST directive. (Turn the listing on again with the *LIST directive.)

Note: *This directive works only with the E or D options specified on the SSG call line.*

5.2.42. *WAIT

The *WAIT directive allows delays to occur in skeleton processing.

Format

```
*WAIT arithmetic-expression
```

Description

Use the *WAIT directive to delay skeleton execution. You may want to do this when printing messages intended to be read on a demand terminal or when skeleton interpretation needs to be slowed down.

The arithmetic expression can include the * and / operators. (For acceptable arithmetic expressions see Table 4–1.) The value obtained by evaluating the arithmetic expression is the number of seconds to wait.

An undefined variable within an expression has a value of 0. If the expression contains a bracketed reference to an undefined variable, then a NO FIND error is generated. A maximum of 86,400 seconds (24 hours) is allowed.

Example 1

```
*WAIT 30
```

Example 2

```
*WAIT TIME_DELAY
```

Example 3

```
*WAIT 30 * WAIT_VALUE - 12
```

Section 6
Permanent and Temporary Streams

This section describes the permanent correction file (PCF) and the temporary correction file (TCF). It describes access and references to the files, line changes, and the revised temporary correction (RTC) stream.

6.1. Introduction

The Executive Control Language (ECL) applies a group of correction images to a symbolic element, producing a new, possibly virtual symbolic element. To see the correction image formats, refer to the *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949).

Example

```
@MASM,U filename.element-name...

(correction images)

@EOF
```

6.2. PCF and TCF Sets

You often need a skeleton to generate a symbolic stream (generated output stream) that contains correction images, as shown in the example in 6.1. To make program corrections separately from the skeleton, you can group correction streams into PCF and TCF sets (see Appendix B).

A PCF set maintains groups of changes (element entries) to some base level symbolic elements.

A TCF set modifies or adds changes to an existing PCF entry. A TCF entry can contain two types of line-change statements:

- Insertion—those relative to the base level symbolic elements being modified.
- Modification—those relative to an already existing line-change statement in the PCF. Both statements can be mixed in a TCF stream.

Each PCF and TCF set can contain unlimited element entries. An element entry can have a group of symbolic images attached. This is generally referred to as a correction stream. Specifically, correction streams (element entries) in PCF sets are referred to as permanent correction streams (PCF streams). Those in TCF sets are referred to as temporary correction streams (TCF streams).

You can use skeleton directives to merge a permanent stream and a temporary stream (*CORRECT) or two temporary streams (*MERGE). You can direct the result of a merge to the generated output stream or a program file, or both.

Note: *Sequence errors in the input permanent stream are not flagged. Always using SSG to maintain the PCF does not cause problems because the input permanent stream is derived from old input temporary streams that are checked for sequence errors.*

6.3. Element Entries

An image containing an asterisk (*) in the first character position, column 1, immediately followed by an *element-name/version-name* defines an element entry. All images that follow such an **element-name/version-name* image are attached to that element entry until the next entry is defined. The *element-name* and *version-name* can each contain up to 12 characters; alphabetic, numeric, -, or $. The */version-name* is optional.

The format of the entry is as follows:

```
*element-name/version-name [subfield 1-1,subfield 1-n...subfield 2-1, ;

subfield 2-n...subfield n-n...]

(corrections associated with element-name/version-name)
```

The *element-name* and *version-name* are not case sensitive. That is, ABC/DEF is the same as aBc/DeF.

Enter element entries after you define the PCF or TCF set to which they will belong. Attach the name of the element entered (*element-name/version-name*) to the selected PCF or TCF set, and the correction stream to the element entry. An SGS is created, with the set name as the label, element/version as field 1, and any fields specified on the element entry as fields 2 through n. An attached element entry is said to be "in" a PCF or TCF set. A PCF or TCF set can contain any number of element entries. You can flag duplicate element entries by specifying the O and Q options on the @SSG call (see Table 2–1).

6.4. Permanent Streams (PCF)

You can define many PCF sets, each with a maximum seven-character, case-independent *set-name*, PCF*xxxx*.

Each PCF set is a collection of element entries. Images defining element entries can come from any combination of system data format (SDF) files or symbolic elements specified on the @SSG call on a PCF*xxxx* file-id statement, or as images in the runstream following a PCF*xxxx* file-id statement.

There are two other means of defining a PCF set. You can specify the P option (see Table 2–1) in conjunction with a program file (*param-7*, see 2.1.2) on the @SSG call or use the P option (see Table 2–2) in conjunction with a program file on a PCF*xxxx* file-id statement. With this type of input, a program file is attached to the selected PCF set, and the name and contents of each element entry are taken from each nondeleted symbolic element in the attached program file. You can define a PCF set using SDF input. Later you can make a P option PCF set by using a PCF*xxxx*,P file-id statement. Similarly, you can define a P option PCF set and later extend it using SDF input from PCF*xxxx* file-id statements. You can attach one program file to a given PCF set. Also, since each element entry is taking its name and content from the symbolic element in a program file, you should not use an **element-name/version-name* image at the start of the symbolic images.

Another way of defining a PCF set is through the GIVING phrase on the *CORRECT directive.

The primary PCF set, $PCF, is always defined, even when empty (see 6.6).

You can list the input or output PCF sets, or both, using the F and/or J/R options on the @SSG call. You will see the following message after the stream images in the PCF set are listed:

```
******** PCFxxxxSTREAM TOTAL IMAGES:n ********
```

where:

PCF*xxxx*

is the name of the PCF set listed.

n

is the total number of change images for the elements listed. The count does not reflect images for elements not listed. With the use of the J option, all elements cannot be listed.

6.5. Temporary Streams (TCF)

You define TCF sets with a maximum seven-character, not case sensitive, *set-name*, TCF*xxxx*.

Each TCF set is a collection of element entries. Any number of element entries can be in a TCF set. Images defining element entries can come from any combination of SDF files or symbolic elements specified on the @SSG call, on a TCF*xxxx* file-id statement, or as images in the runstream following a TCF*xxxx* file-id statement.

You can also define a TCF set by specifying the T option (see Table 2–1) in conjunction with a program file (*param-7*, see 2.1.2) on the @SSG call or by using the T option (see Table 2–3) in conjunction with a program file on a TCF*xxxx* file-id statement. With this type of input, a program file is attached to the selected TCF set and the name and contents of each element entry are taken from each nondeleted symbolic element in the attached program file.

You can define a TCF set that uses SDF input. Later you can make it into a T option TCF set by using a TCF*xxxx*,T file-id statement. Similarly, you can define a T option TCF set and later extend it using SDF input from TCF*xxxx* file-id statements. One program file can be attached to a given TCF set. Also since the element entries get their names and contents from the symbolic elements of a program file, you should not use an **element-name/version-name* image at the start of the symbolic corrections.

Another way of defining a TCF set is through the GIVING phrase on the *MERGE directive.

The primary TCF set, $TCF, is always defined, even when empty (see 6.6).

You can list the input TCF set, output TCF set, or both, by using the G or U option on the @SSG call. You will see the following message after the stream images in the TCF set are listed:

```
******** TCFxxxxSTREAM TOTAL IMAGES:n ********
```

where:

TCF*xxxx*

is the name of the TCF set listed.

n

is the total number of correction images for the elements listed.

6.6. PCF and TCF Set Access

Two reserved labels, $PCF and $TCF, are defined. They function in a manner similar to $SGS and SGS labels. [$PCF,*i*,1,1] returns the name of the *i* th-defined PCF set, and [$TCF,*i*,1,1] returns the name of the *i* th-defined TCF set. [$PCF] and [$TCF] return the number of PCF and TCF sets defined. [$PCF,1,1,1] and [$TCF,1,1,1] are always PCF and TCF. In other words, the primary PCF and TCF sets are always defined.

PCF and TCF sets are defined in the order input. SSG reads across the @SSG call and then down through file-id statements.

6.7. Set References

You can use a special mechanism, similar to the SGS referencing mechanism described in 3.1.2, to access element entries. For each PCF and TCF set you define, use *set-name*, PCF*xxxx* or TCF*xxxx*, to access the entries within the set.

Since a *set-name* is attached to the PCF or TCF entry and to an SGS, two reserved global variables, PCFNAME$ and TCFNAME$, are defined.Initially, PCFNAME$ has a value of 0, and TCFNAME$ has a value of 1.

- If PCFNAME$ (TCFNAME$) is set to a nonzero value, SSG attempts to satisfy a bracketed reference that looks like a PCF (TCF) set reference by searching the PCF set (TCF set) chain.
  - If the *set-name* is found, the reference is assumed to pertain to a valid PCF (TCF) set reference as described in Table 6–1.
  - If the *set-name* is not found, SSG assumes the reference is an SGS reference and the SGS label chain is searched.
- If PCFNAME$ (TCFNAME$) has a value of 0, the reference is assumed to be an SGS reference and the PCF set (TCF set) chain is not searched for a matching *set-name*.

The primary PCF set has a synonym, P, for referencing. A reference with label P refers to PCF, independent of the value of PCFNAME$. Similarly, the primary TCF set has a synonym, T, which refers to TCF, independent of the value of TCFNAME$.

Table 6–1 assumes PCFNAME$ and TCFNAME$ have nonzero values. If the *set-name* referenced does not exist, the following set references automatically become SGS references (see 3.1.2) with a label, *l*, of *set-name.*

Table 6–1. PCF and TCF Set References

Set Reference	*Value Returned*	*Type*
[*set-name*]	Number of element entries in *set-name*. If *set-name* is empty, the value returned is 0.	NUMERIC
[*set-name*,*n*, 1,1]	Element name for the *n* th element entry in *set-name*. If there is no *n* th entry in *set-name*, a NO FIND condition exists.	SYMBOLIC
[*set-name*,*n*, 2,1]	Version name for the *n* th element entry is *set-name*. If there is no *n* th entry in *set-name*, a NO FIND condition exists.	SYMBOLIC
[*set-name*,*n*, 3,1]	Concatenated element and version name. If there is no *n* th entry in *set-name*, a NO FIND condition exists.	SYMBOLIC

The order of the element entries in a PCF or TCF set, unless controlled by the O or Q options (see Table 2–1) is determined by the order in which they are encountered on input. SSG reads across the @SSG call and then down through file-id statements.

The extended SGS subfield references described in 3.1.2.2 can be used. The existence of version-name for a particular element can be tested using the extended reference [*set-name*,*n*,2,1,LEN$]. For example, [TCF0001,5,2,1,LEN$] would be non-zero if and only if the fifth element in TCF set TCF0001 has a non-space version name.

For those PCF and TCF sets that have an associated program file, the SGS file name references described in 3.1.2.3 can be used. To avoid ambiguity, the PCF and TCF set names used in the references must be different from SGS labels associated with file names.

The rules for composing set references are identical to the rules for composing SGS references, described in 3.1.2.4.

Set Reference Example

This example uses the following input:

```
@ssg,ik
TCF001
*ELTA
-2,2
*ELTB/VERSIONB
-3,3
@eof
TCF002
*ELTC
*ELTD
-22,33
*ELTE
-1,10
@EOF
```

SSG logically structures the set information as follows:

```
TCF0001 ELTA
TCF0001 ELTB/VERSIONB
TCF0002 ELTC
TCF0002 ELTD
TCF0002 ELTE
```

Set references return the following:

```
Reference              Value

[tcf0001]               2
[tcf0001,1,1,1]         ELTA
[tcf0001,1,2,1]                    There is no version name.
[tcf0001,1,3,1]         ELTA
[tcf0001,2,3,1]         ELTB/VERSIONB
[tcf0002,2,1,1]         ELTD
```

Example

The following example shows how you can use TCFNAME$ and PCFNAME$ to check TCF or PCF set entries for fields on the element cards.

```
*SET CASE$                          .strings case insensitive
*SET PCFNAME$                       .satisfy on PCF chain first
*INCREMENT A TO [PCF001]
*SET name-hold = '[PCF001,A,3,1]'   .save the element/version-name
*CLEAR PCFNAME$                     .search SGS chain only
*IF COLUMN SEARCH FROM PCF001,1,1,1 FOR [*name-hold]
*IF ROW SEARCH FROM PCF001,STMT$,1,1 FOR DELETE
#DELETE,CS PCF.[*name-hold]
#DELETE,CS BASE.[*name-hold]
*ENDIF
*ENDIF
*SET PCFNAME$                       .back to PCF chain
*LOOP
*CLEAR CASE$                        .restore normal case
```

This example increments through each entry in the PCF set 'PCF001'. If the keyword DELETE is present on the element entry, that element will be deleted from the BASE and PCF files.

6.8. Line–Change Format

TCF line change statements, relative to an already existing line change statement in the PCF, have the following formats:

Format 1

```
-perm-line-nos/relative-line-nos
```

Format 2

```
-perm-line-nos/relative-line-nos/new-line-nos
```

where:

perm-line-nos

are the same line numbers as in the line-change image in the PCF stream being revised.

relative-line-nos

are the relative line numbers where a change is to be made to the data images in the PCF stream.

For relative referencing purposes, the line-change statement itself in the PCF is numbered relative line 0. Data images following that line-change statement, until the next line-change statement in the same element entry, are numbered as relative lines 1,2, ..., and so forth.

relative-line-nos can be of two forms:

n

indicates that any data images in the TCF following this line change statement, which is relative to the PCF, are to be inserted into the PCF stream after relative line *n*. Relative line *n* must exist in the PCF stream.

n,m

indicate that relative lines *n* through *m* (*n* <= *m*) of the PCF stream are to be removed. Data images in the TCF following this line change statement are to be inserted into the PCF stream after relative line *m*. Relative line *m* and, therefore, relative line *n* must exist in the PCF stream.

new-line-nos can be of four forms:

```
p

p-

p,q

p,q-
```

where line change statements with *new-line-nos* specified direct SSG to insert *new-line-nos* from the TCF into the PCF stream. The *new-line-nos* are inserted before inserting any data images and are inserted into the PCF stream after relative line *n* or *m* (where *n* or *m* depends on the type of *relative-line-nos*).

Example 1

The following example shows how line change statements and data images in a PCF stream are numbered for relative referencing purposes:

Relative Line Numbers	PCF Stream
0	-2,3
1	A
2	B
3	C
0	-10
1	XYZ
0	-14,16

Therefore, any image in a PCF stream can be referenced, first according to the line-change statement associated with it, and then according to its position relative to that line-change statement.

Notes:

- *If the first image in a PCF or TCF stream is a data image, SSG assumes the existence of a -0 line-change statement (relative line 0). However, in this case, a -0/0 or -0/0,0 line-change statement is in error, since the -0 line-change statement does not actually exist. If relative changes of this type are anticipated, it is recommended that an explicit -0 line-change statement be supplied.*
- *If a temporary correction file stream contains -0/0 followed by images to be inserted at the start of the PCF stream, the PCF stream must contain a -0 as the first image. If the PCF stream does not contain the -0, SSG rejects the change and issues the message "`INVALID ATTEMPT TO MAKE RELATIVE CORRECTION:….`"*

Example 2

This example uses the following PCF and TCF input:

```
PCF                TCF                  Revised PCF

                   -2                   -2
                   MNO                  MNO
-4,6               -4,6/2,2             -4,6
X                  LJK                  X
YYY                                     LJK
                   -8                   -8
                   ZZZ                  ZZZ
-10                -10/0,0/12,12        12,12
B                  NEW                  NEW
                                        B
-25,25-            -25,25-/1,1          -25,25-
/ABC/DEF/          /ABC/EFJ/            /ABC/EFJ/
-32,33-            -32,33-/0,0/32,32-   -32,32
/L/S/              -32,33-/1/35,35-     /L/S/
/A0/A2/                                 -35,35-
                                        /A0/A2/
-43,44                                  -43,44
                   -54                  -54
                   QRS                  QRS
                   -72,76               -72,76
```

The changes in the TCF stream reference the PCF as follows:

```
-2
MNO
```

Since this is not a relative change to the PCF, it must be a change to be added to the PCF. Therefore, -2 and MNO are inserted at the beginning of the revised PCF.

```
-4,6/2,2
LDK
```

Deletes line 2 relative to the image -4,6 in the PCF, which is YYY, and inserts the line LJK at that position.

```
-8
ZZZ
```

Is a correction to be added to the PCF. Therefore, -8 and ZZZ are inserted between -4,b and -12,12 followed by the line NEW.

```
-10/0,0/12,12
NEW
```

Deletes line 0 relative to the image -10 in the PCF, which is -10, and inserts the image -12,12 followed by the line NEW.

```
-25,25/1,1
/ABC/EFJ/
```

Deletes line 1 relative to the image -25,25- in the PCF, which is /ABC/DEF/, and inserts the line /ABC/EFJ/ at that position.

```
-32,33-/0,0/32,32-
```

Deletes line 0 relative to the image -32,33- in the PCF, which is -32,33-, and inserts the image -32,32-.

```
-32,33-/1/35,35-
```

Inserts the image -35,35- following line 1 relative to the image -32,33- in the PCF, which is /L/S/.

```
-54
QRS
```

Is a correction to be added; -54 and QRS are inserted between -43,44 and -72,76 in the revised PCF.

6.9. Change Control Characters

The standard control character for the *CORRECT and *MERGE types of merges is the minus sign (-). You can change this standard to another control image (up to three characters), according to the following format:

```
-=new-control-characters
```

Note: *Control characters cannot be numeric. Bracketed references are not allowed. The image must appear as the first image in the stream to which it belongs.*

When you perform a *CORRECT merge, the change of control character image can appear as the first image of the PCF stream, as the first image of the TCF stream, or immediately after the *CORRECT directive in the skeleton. Any other images, in either stream or between the *CORRECT directive and its matching *ENDCORRECT directive, are in error. Images in the skeleton that look like change control character images and are not between a *CORRECT directive and its matching *ENDCORRECT directive are treated as nondirective images and do not affect the control character.

- If the change control character image is the first image in the PCF stream, it triggers the change and the image is passed to the revised PCF stream (if there is one). It does not appear in the generated output stream.
- If the change control character image is in the TCF stream, it triggers the change and the image is passed to the revised temporary correction (RTC) stream (if there is one), but is not passed to the revised PCF stream or the generated output stream.
- If the change control character image appears after the *CORRECT directive in the skeleton, it triggers the change but does not appear in the revised PCF, RTC, or generated output streams.

Note: *When listing the RTC stream (@SSG,H), SSG does not recognize a change of the change control characters (-=xxx) in the RTC stream. This causes the subsequent counter to be incorrect.*

When a change control character image for a *CORRECT merge comes from more than one source, the destination for each image is as described in the preceding paragraph. However, the *new-control-characters* for that merge are determined according to the following priorities:

1. TCF overrides PCF and skeleton.
2. PCF overrides skeleton.

After each *CORRECT merge is performed (the matching *ENDCORRECT directive is encountered), the control character is reset to -.

When you perform a *MERGE merge, the change control character image can appear as the first image of either input TCF stream. If a change control character image appears in both TCF streams, the one in TCF*xxxx* is used. This image triggers the change of control characters and is sent to TCF*zzzz* (TCF*yyyy*, if assumed) but does not appear in the generated output stream. When the merge is complete, the control character is reset to -.

6.10. Revised Temporary Correction (RTC) Stream

The revised temporary correction (RTC) stream is a collection of element entries created by SSG. The RTC stream is saved in *param-4* if it is specified on the @SSG call. The RTC stream is listed if the H option is specified on the @SSG call. If either *param-4* or the H option is specified on the @SSG call, the value of the reserved global variable, RTC$, is initially set to 1 by SSG; otherwise, it is initially set to 0.

An RTC entry is created by SSG when

- A permanent stream and a temporary stream are merged with the *CORRECT directive. If a temporary stream is not included in the merge (due to the absence of the input temporary stream or specification of PERM on the *CORRECT directive), an RTC entry is not created.
- Either the H option or *param-4*, or both, are specified on the @SSG call. If neither *param-4* nor the H option is specified on the @SSG call, no RTC images are created.

- The value of the reserved global variable RTC$ is nonzero when the *CORRECT directive is encountered. Creation of RTC images results only if the value of RTC$ is nonzero when the *CORRECT directive is encountered.

RTC$ can be used two ways:

- To alert the skeleton that an RTC stream is being maintained by SSG.
- To turn the generation of RTC images off or on when a *CORRECT operation is being performed.

An RTC entry is a copy of the temporary stream with line numbers revised to make it applicable to the updated symbolic element created by applying the permanent stream to the original base level symbolic element. If skeleton-generated corrections are used, the revised line numbers reflect skeleton-generated corrections. Skeleton-generated corrections are included in the output of the *CORRECT directive. They modify the new base level symbolic element.

Example 1

Base	PCF	TCF	PCF+TCF	Base +PCF+TCF
B image 1	-1,2	-6,9	-1,2	B image 3
B image 2	-4	-12,12	-4	B image 4
B image 3	P image a	T image a	P image a	P image a
B image 4	-10,10	T image b	-6,9	B image 5
B image 5			-10,10	B image 11
B image 6			-12,12	T image a
B image 7			T image a	T image b
B image 8			T image b	
B image 9				
B image 10				
B image 11				
B image 12				

Newbase	RTC	Newbase+RTC
B image 3	-5,8	B image 3
B image 4	-10,10	B image 4
P image a	T image a	P image a
B image 5	T image b	B image 5
B image 6		B image 11
B image 7		T image a
B image 8		T image b
B image 9		
B image 11		
B image 12		

If the temporary stream contains line-correction images that change line-correction images in the permanent stream (for example, -1,2/0,0/2,3), it is impossible to be sure that the RTC stream will be correct. It is also impossible to restore lines deleted by the original permanent stream. Examine the RTC stream before you apply it to the updated symbolic element. SSG places the following message around the line-correction images that may be in error:

```
*WARNING *.******UPDATE PERFORMED-RESULTS ARE UNPREDICTABLE******
```

Example 2

This example uses the following permanent and temporary input streams:

```
PCF            TCF

-1,2           -3,3
-6,8           -6,8/0,0/6
               -16
```

The RTC stream looks like the following:

```
-1,1
*WARNING *.****** UPDATE PERFORMED-RESULTS ARE UNPREDICTABLE ******
-3
*WARNING *.****** UPDATE PERFORMED-RESULTS ARE UNPREDICTABLE ******
-11
```

The TCF stream revises the PCF stream by deleting lines 6,7, and 8 in the original symbolic element and inserting images after line 6. After the PCF has been applied to the symbolic element, SSG cannot restore the deleted images, so the RTC stream needs to be modified to produce the desired results. The following message points to the images in the RTC stream that may need to be further revised (in this case, -3):

```
"UPDATE PERFORMED-RESULTS ARE UNPREDICTABLE"
```

When revised images are generated from input temporary images and those images contain images with slashes (/), the revised images cannot be assumed to be correct. If the value of IMGLEN$ is not sufficient to hold the revised line number, the result is meaningless. Setting the value of the IMGLEN$ to a large number, such as 132, should eliminate any problems. This is also true for virtual line numbers produced as a result of the *MERGE,R directive.

Appendix A
Diagnostic, Error, and Warning Messages

This appendix lists and explains diagnostic, error, and warning messages.

A.1. Diagnostic Messages

**** ABORT ENCOUNTERED AT LINE *n* OF *source*

This message is printed when an ABORT directive is encountered. The parameter *n* indicates the line where the abort was encountered and *source* indicates SKEL$ if the abort was in the main skeleton, or a define packet main if the abort was encountered in a define packet.

END SSG. ERRORS: *w* WARNING *s* SYNTAX *n* NO-FIND *m* MERGE/CORRECT
LINES: *i* INTERPRETED *o* OUTPUT TIME: *t* STORAGE: *a/b/c/d/e*

The SSG termination line displays when SSG terminates normally.

where:

w
is the warning.

s
is the value of ERRCNT$.

n
is the value of NOFND$.

m
is the value of CONFLC$.

i
is the value of SKELINECNT$.

o
is the value of GOSCNT$.

t

is the time in standard units of processing (SUP) in the format *hh:mm:ss.sss*.

a/b/c/d/e

describe memory allocation as follows:

- *a* is the number of times a memory packet was requested.
- *b* equals the number of errors encountered while returning packets back to the available lists (should be 0).
- *c* is the number of data banks acquired during processing.
- *d* is the upper address limit in the final data bank. Data bank (D-bank) addresses begin at the end of the shared I-bank.
- *e* is the maximum size of the data banks expressed as a decimal. Note that the maximum number of SGSs with the same label is *e* - 16.

For more information, refer to the *Exec System Software Administration Reference Manual* (7831 0323).

A.2. Error Messages

`*ERROR* - 1 LEFT PARENTHESIS IS UNMATCHED`

With NEW_BOOLEAN$ nonzero, there was a left parenthesis that did not have the required matching right parenthesis on an *IF directive.

`*ERROR* - AREAD$ IMAGE GREATER THAN 132 CHAR MAX ALLOWED - PART OF SSG D-BANK DESTROYED`

Part of SSG has been overwritten. This happens when an image in the runstream is bigger than 132 characters.

`*ERROR* - BANK$ TO ACQUIRE ADDITIONAL BANK FAILED - SNAP$ OF PKT FOLLOWS:`

A fatal error occurred within SSG while trying to get an extra data bank in order to continue processing. Contact your local Unisys Support Representative.

`*ERROR* - BRK SPEC --> message`

Result of a *COPY error. See the "`*ERROR* - SPEC nn -->`" message description.

```
*ERROR* - CONFLICT IN RELATIVE CORRECTIONS: *MERGE image
```

During the merging of temporary streams with *MERGE, internal conflicts in line corrections resulted. Some conflicts could not be resolved (PCF entry unknown). As a result, SSG displayed this message and discarded the correction from TCF*yyyy*. The parameter *image* is a copy of the *MERGE directive in error.

```
*ERROR* - *CORRECT DIRECTIVE MISSING MATCHING *ENDCORRECT
```

A *CORRECT directive was found but no *ENDCORRECT was found to match it.

```
*ERROR* - *CREATE IMAGE EXCEEDED CURRENT MAX ALLOWED
```

A *CREATE image exceeded the maximum number of characters for that image defined by the value of IMGLEN$.

```
*ERROR* - DATAFILE SPECIFIED ON DIRECTIVE LINE IS A PROGRAM FILE
```

A data file was specified on the *SAVE directive line; however, this file is a program file. The *SAVE directive will not be executed, to prevent destroying a program file by rewriting it as a data file.

```
*ERROR* - *DEFINE DIRECTIVE MISSING MATCHING *ENDDEFINE
```

No *ENDDEFINE was found for a *DEFINE directive.

```
*ERROR* - DEFINE PACKET IS TOO LARGE TO PROCESS
*ERROR* - SIZE CAN BE REDUCED BY MOVING PARTS OF DEFINE
*ERROR* - PACKET INTO SMALLER DEFINE PACKETS - SEE SSG PRM (7831 1784)
```

The define packet specified is too big to be read in by SSG. Split this define packet into two or more smaller packets.

```
*ERROR* - *DO DIRECTIVE MISSING MATCHING *ENDDO
```

No *ENDDO was found for a *DO directive.

```
*ERROR* - *DO DIRECTIVE MISSING MATCHING *ENDDO OR *INCREMENT MISSING
MATCHING *LOOP
```

No *ENDDO was found for a *DO directive or no *LOOP was found for an *INCREMENT directive.

```
*ERROR* - ENCOUNTERED INVALID DIRECTIVE CODE - INTERNAL ERROR
```

A fatal error occurred within SSG while processing directives. An internal directive code (DIRCOD) has been corrupted. Contact your local Unisys Support Representative.

`*ERROR* - ER PFI$ ERROR` *n* `ADDING ELT` *element-name/version-name* `TO` *file-id*

SSG calls ER PFI$ to create an output element. Octal error code *n* was returned on the call. Refer to the *Executive Requests Programming Reference Manual* (7830 7899) for a description of the ER PFI$ error codes. The *file-id* contains additional information about the file in error. See Table A-1.

Table A–1. *File-Id* and Descriptions

`*BRKPT FILE` The error occurred for the generated output stream file specified on the last interpreted *BRKPT directive. The error occurred at the end of skeleton execution.
`*BRKPT AT SKEL LINE sl` The error occurred for a generated output stream file specified on a previously interpreted *BRKPT directive. The error occurred on the *BRKPT directive at line sl.
`PCFxxxx AT SKEL LINE sl` The error occurred for the PCF set file specified. The error occurred on the *CORRECT or *ENDCORRECT directive at line sl.
`REVISED SKELETON FILE` The error occurred for the revised skeleton stream file specified by param-5 on the @SSG call. The error occurred before the start of skeleton execution.
`*SAVE FILE AT SKEL LINE sl` The error occurred for the file specified on the *SAVE directive at line sl.
`SGR$2 - SSG CANNOT PERFORM DYNAMIC @ADD` The error occurred for the default generated output stream in file SSG$*SGR$2, element SGR$2. The error occurred at the end of skeleton execution.
`SPEC 03 (GOS) FILE` The error occurred for the generated output stream file specified by *param-3* on the @SSG call. The error occurred at the end of skeleton execution.

Table A–1. *File-Id* and Descriptions

SPEC 03 (GOS) FILE AT SKEL LINE *sl* The error occurred for the generated output stream file specified by param-3 on the @SSG call. The error occurred on the *BRKPT directive at line sl.
SPEC 04 (RTC) FILE The error occurred for the revised temporary correction stream file specified by param-4 on the @SSG call. The error occurred at the end of skeleton execution.
TCFxxxx ST SKEL LINE sl The error occurred for the TCF set file specified. The error occurred on the *MERGE directive at line sl.

ERROR – ER PFWL$ ERROR *n* FOR *file-id*

SSG calls ER PFWL$ to determine where to begin writing the text for an output element. Octal error code *n* was returned on the call. Refer to the *Executive Requests Programming Reference Manual* (7830 7899) for a description of the ER PFWL$ error codes. The *file-id* contains additional information about the file in error. See Table A-1.

ERROR – ERROR DETECTED, ON BRKPT *n*, ON OR BEFORE COLUMN *c*, SKEL LINE *sl*

An error was detected on a *BRKPT directive. The *n* parameter is the part number of the generated output stream, *c* is the column position, and *sl* is the line number of the revised skeleton stream. The breakpoint is performed, but name is ignored.

ERROR – ERROR DETECTED ON LINE *sl* ON OR BEFORE COLUMN *c*

An error occurred in the revised skeleton stream at line number *sl* and position *c*. Another diagnostic that explains the error in more detail is also displayed.

ERROR – ERROR IN FORMATION OF OUTPUT STREAM – SSG WILL NOT PERFORM DYNAMIC @ADD

An error is detected during generation of the output stream and the B option has not been specified on the @SSG call. The generated output stream is not dynamically added (@ADD).

ERROR – ERROR IN PROCESSING *element-name/version-name*

An error occurred in the specified element while processing a *CORRECT or a *MERGE directive. Other clarifying diagnostics and images are displayed.

`*ERROR* - ERROR IN REVISED SKEL - SSG WILL NOT PROCESS IT`

This message is displayed at the end of the listing of the revised skeleton stream if an error has been detected. Another message explains the error where it occurs in the listing. Fix the skeleton.

`*ERROR* - FILE ID --> message`

Result of an error encountered on the specified file. For more information, refer to the "`*ERROR* - SPEC nn -->`" message description.

`*ERROR* - FILE IDENTIFICATION STATEMENT ERROR`

A file-id statement has a format error or specifies an illegal type of input stream.

`*ERROR* - FILE IDENTIFICATION STATEMENT ERROR - REENTER STATEMENT`

An erroneous, SSG recoverable file-id statement was submitted from demand that was not an @ADD image. Reenter the statement or an end-of-file image.

`*ERROR* - FILE SPECIFIED ON DIRECTIVE LINE NOT SECTOR FORMAT`

The file specified on the directive line, immediately following this error message is not a sector-formatted file. SSG processes only sector-formatted input and output files.

`*ERROR* - FOLLOWING DIRECTIVE IMPROPERLY NESTED`

A *CORRECT operation is performed and a *BRKPT or *MERGE directive is encountered. The *BRKPT or *MERGE directive is in error and is not processed.

`*ERROR* - FOLLOWING DIRECTIVE IN ERROR`

An unknown skeleton directive is encountered in the revised skeleton or the *define-name* on a *DEFINE directive is in error.

`*ERROR* - FOLLOWING DIRECTIVE OUT OF SEQUENCE`

The indicated directive is illegally nested. *DEFINE directives must precede the control section and may not be statically nested. *BRKPT and *MERGE directives cannot be embedded between a *CORRECT and its matching *ENDCORRECT.

`*ERROR* - FORMAT ERROR ON CHANGE CONTROL CHAR IMAGE`

The change control image is not correct. More than three characters were specified or at least one is numeric.

`*ERROR* - FORMAT ERROR ON FOLLOWING CONTROL STATEMENT IMAGE`

The first nonspace character on a *CSF directive is not @. The ER ACSF$ is not performed, and the CSFSTA$ variable and SGS remain unchanged.

```
*ERROR* - FORMAT ERROR ON FOLLOWING *CREATE IMAGE
```

Given the type of image created, the image was badly formatted.

```
*ERROR* - FORMAT ERROR ON INPUT
```

The specified image is in error and is displayed.

```
                                          ⎡SGS IGNORED                ⎤
*ERR0R* - FORMAT ERROR ON FOLLOWING SGS - ⎨                            ⎬
                                          ⎣SUBFIELD AND SGS TRUNCATED⎦
```

The SGS label field (SGS IGNORED) or some subfield (SUBFIELD AND SGS TRUNCATED) is in error. SGS subfields are truncated at 128 characters; this error occurs when that limit is exceeded.

```
*ERROR* - FORMAT ERROR ON INPUT - REENTER
```

The image input to SSG through an *ACCEPT directive is invalid and must be reentered.

```
*ERROR* - @FREE FAILED ON FILE ON DIRECTIVE LINE BELOW.  FI STATUS: x
```

The freeing (@FREE) of the file specified on the directive line immediately following this error message failed, resulting in Exec error status *x*. This message is intended for internal use only.

```
*ERROR* - *IF DIRECTIVE MISSING MATCHING *ENDIF
```

An *IF directive exists with no matching *ENDIF directive. This can occur after a LOOP, since an *IF directive cannot end outside a LOOP if the *IF was inside the loop. It can also occur at the end of a skeleton when there are no more directives left.

```
*ERROR* - ILLEGAL RESPONSE - CONTROL STATEMENT
```

An Exec control statement (@ image) was input in response to an *ACCEPT directive. This error causes termination of SSG through ER ERR$.

```
*ERROR* - IMAGE LENGTH TOO LONG - REENTER
```

The image input on an *ACCEPT directive exceeds 121 characters after all references have been satisfied.

```
*ERROR* - *INCREMENT DIRECTIVE MISSING MATCHING *LOOP
```

No *LOOP directive was found for an *INCREMENT directive.

```
*ERROR* - *INCREMENT OR *DO LOOP LOCAL VARIABLE CANNOT BE SET TO STRING
TYPE
```

A *SET variable-name TO 'string' directive specifies a local variable. The *SET directive is ignored.

```
*ERROR* - *INCREMENT OR *DO LOOP WHILE VARIABLE CANNOT BE SET TO STRING
TYPE
```

A *SET variable-name TO 'string' directive specifies a global variable that is currently in use on a WHILE phrase for an active *INCREMENT or *DO directive loop. The *SET directive is ignored.

```
*ERROR* - INTERNAL CONFLICT: *MERGE image
```

An internal conflict was found when the indicated temporary streams were merged. The parameter *image* is a facsimile of the offending *MERGE image (see also "CONFLICT IN RELATIVE").

```
*ERROR* - INTERNAL ERROR AT LINE sl, COLUMN c, NESTING LEVEL l,
                    ILLEGAL STACK INDEX i ON type CALL FROM adrs
```

A serious internal logic error was detected on an *IF directive at skeleton line number *sl* at or before column *c*.

l	is the number of unmatched left parentheses when the error was detected. The allowable range is 0 through 25.
i	is the stack index that is out of range. The allowable range is 1 through 5.
type	is the type of stack call. Allowable values are LOAD, STACK, STORE, and UNSTACK.
adrs	is the octal address from where the stack call was made.

SSG is terminated after this error occurs. To gather additional diagnostic information, insert a *SET NOPT$ directive immediately before the *IF directive and reexecute the skeleton. Then provide your local Unisys service representative with the original error message and the additional diagnostic information.

```
*ERROR* - INTERNAL SSG FILE COULD NOT BE ASSIGNED
```

An internal file that SSG uses for processing could not be assigned. The @ASG image is displayed.

ERROR - INVALID ATTEMPT TO MAKE RELATIVE CORRECTION: *CORRECT *image*

During the merging of the indicated permanent and temporary streams, an invalid modifying image in the temporary stream or an invalid *relative-line-nos* value occurred. The parameter *image* is a facsimile of the invalid *CORRECT image.

ERROR - INVALID DEFINE NAME ON FOLLOWING DIRECTIVE

The *DEFINE name is not a valid name for a define packet. Usually the problem is caused by a hyphen in the name, which is not allowed

ERROR - INVALID FILE SPECIFICATION - IGNORED

SSG was unable to assign a file properly for a file information SGS.

ERROR - INVALID ICW: *icw*, SDFI FCT ADDR: *p*, FILE: *f*

While reading symbolic streams, SSG received an image control word (*icw*), that may be incorrect.

icw is the image control word.

p is the File Control Table (FCT) used by system data format input (SDFI). For more information, refer to the *SYSLIB Programming Reference Manual* (7833 1733).

f is the internal file name where the error is detected. The external file name can be found by processing an @PRT,F *f* image. For more information, refer to the *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949).

ERROR - I/O ERROR *xx* AT ADDR *a*, PKT: *p*, FILE: *f*

This message signals an I/O error, which usually indicates data corruption. Check the file indicated.

xx specifies the type of I/O error.

a is the address of the I/O error.

p is the address of the I/O packet.

f is the external file name.

ERROR - *ITERATE OUTSIDE OF *INCREMENT OR *DO LOOP

An *ITERATE directive was encountered outside of some type of loop directive. *ITERATE can only function in a loop.

```
*ERROR* -  L OPTION SPECIFIED AND SGS LABEL IS IN ERROR
```

The L option is specified for an SGS file identification statement (SGS,L ...) or for an *ADD directive (*ADD SGS,L ...) and the required SGS label is in error. The label must be from 1 to 12 characters from the set of alphabetics (A through Z), numerics (0 through 9), dollar sign ($), and hyphen (-). For more information, refer to 2.2 and 5.2.3.

```
*ERROR* -  L OPTION SPECIFIED AND SGS LABEL IS MISSING
```

The L option is specified for an SGS file identification statement (SGS,L ...) or for an *ADD directive (*ADD SGS,L ...) and the required SGS label is not present.

```
*ERROR* - LEFT PARENTHESIS AT LINE sl, COLUMN c WITH NESTING LEVEL AT
25
```

With NEW_BOOLEAN$ nonzero, a left parenthesis was encountered on an *IF directive when there were already 25 unmatched left parentheses, the maximum number allowed. The skeleton line number *sl* and column *c* are provided to help locate the left parenthesis that is in error.

```
*ERROR* - MAIN SKELETON IS TOO LARGE TO PROCESS
*ERROR* - SIZE CAN BE REDUCED BY MOVING PARTS OF MAIN
*ERROR* - SKELETON INTO SMALLER DEFINE PACKETS - SEE SSG PRM
(7831 1784)
```

When this error occurs, the size of the main skeleton can be reduced by putting parts of it in *DEFINE packets. In the main skeleton a *PROCESS name is then substituted for the moved images.

```
*ERROR* - n LEFT PARENTHESES ARE UNMATCHED
```

With NEW_BOOLEAN$ nonzero, there were *n* more left than right parentheses at the end of an *IF directive. All left parentheses must have matching right parentheses.

```
*ERROR* - NO FIND BEFORE COLUMN c ON FOLLOWING IMAGE, SKEL LINE sl
```

A reference before column *c* on the following image could not be satisfied. The parameter *sl* is the corresponding line number of the revised skeleton stream.

```
*ERROR* - NO FIND, ON BRKPT n, BEFORE COLUMN c, SKEL LINE sl
```

This message occurs if a NO FIND condition exists while processing the *BRKPT directive. The breakpoint is performed, but the *name* is ignored.

n is the part number of the generated output stream.

c is the column position.

sl is the line number of the revised skeleton stream.

ERROR - NO FIND ON BRACKETED REFERENCE - REENTER

An attempt is made to satisfy any bracketed references from the *ACCEPT directive. One such reference could not be satisfied, and the image should be reentered.

ERROR - NO MATCH ABOVE FOR FOLLOWING *DIRECTIVE

An *ELSEIF, *ELSE, *END, *ENDDEFINE, *ENDIF, *ENDCORRECT, or *LOOP directive was encountered in the revised skeleton that did not have a matching *DEFINE, *IF, *CORRECT, or *INCREMENT directive.

ERROR - NON-NUMERIC CHARACTER SUBFIELD ENCOUNTERED WITH X-OPTION SORT SGS LABEL: *label* SUBFIELD VALUE: *non-numeric-character*

A *SORT,X directive was encountered in the revised skeleton, indicating that a numeric sort is to be executed. However, a nonnumeric character was present on the *label* SGS with subfield *non-numeric-character*, so the sort is not performed.

ERROR - PROGRAM FILE SPECIFIED ON DIRECTIVE LINE IS A DATAFILE

A program file was specified on the *SAVE directive line. Since this is a data file, the *SAVE directive will not be executed.

ERROR - RESTORE FILE/ELEMENT IS UNOBTAINABLE, SKELETON LINE *x*

The file/element specified on skeleton line *x* is unobtainable. The *RESTORE operation is not performed, and skeleton execution halts.

ERROR - RESTORE FILE/ELEMENT NOT CREATED BY SSG, SKELETON LINE *x*

The file/element specified on skeleton line *x* was not created by the SSG *SAVE directive. The RESTORE operation is not performed, and skeleton execution halts.

ERROR - RESTORE FILE/ELEMENT TEXT IS INCOMPLETE, SKELETON LINE *sl*

The file/element specified on skeleton line *sl* is not complete. The most likely cause is that the file/element contains a large number of SGSs and an SSG with level earlier than 24R2B was used to write it to a program file element.

ERROR - RESTORE FILE/ELEMENT VERSION ID *n* IS ILLEGAL, SKELETON LINE *sl*

The file/element specified on skeleton line *sl* has an illegal version id. Currently, version id 4 is supported.

`*ERROR* - RIGHT PARENTHESIS AT LINE sl, COLUMN c WITH NESTING LEVEL AT 0`

With NEW_BOOLEAN$ at nonzero, a right parenthesis was encountered on an *IF directive when there were no unmatched left parentheses. The skeleton line number *sl* and column *c* are provided to help locate the right parenthesis that is in error.

`*ERROR* - RQST SIZE OF n EXCEEDS MAX BANK SIZE OF m`
`*ERROR* - IMAGE LINE j FOLLOWS (RQST CALLED AT ADDRESS addr):`

A fatal error occurred within SSG where a routine was trying to acquire a memory packet larger than any single data bank can supply. *n* is the size of the requested memory packet, while *m* is the largest size available to any data bank. Contact your local Unisys Support Representative.

`*ERROR* - SEQUENCE OR FORMAT ERROR ON FOLLOWING IMAGE`

This message refers to either the permanent or temporary streams or both that are being merged at the time (*MERGE or *CORRECT). Examine the image and the correction stream to determine if the line is out of sequence or in the wrong format.

`*ERROR* - REACHED SSG MEMORY LIMIT OF 0777777 IN ALL n DATA BANKS`

The maximum address space of 262K has been reached in all available data banks, so SSG cannot continue. Either partition the application, *REMOVE entities no longer needed, or recompile SSG with more data banks by increasing MAXBNKS in SSG$DEF (see 1.3.2).

`*ERROR* - SPEC nn --> message`

The error has occurred for the reason specified in the *message*, where *nn* is the parameter field number on the @SSG call *param-nn* (the SPEC *nn* part of this message could be replaced by FILE ID -> or BRK SPEC ->), and *message* can be any of the following:

`BAD LABEL IMAGE`

The label image of a system data format (SDF) input source is invalid.

`FILE EMPTY`

SSG expected input from a file and it was empty.

`INVALID FORM ccccc`

The *param-7* input stream type is illegal. The first five characters (*ccccc*) of the illegal type are shown. The type is not OPT, PCF*xxxx*, SGS, or TCF*xxxx*.

`NOT FILE OR ELT`

A *param*, *specification*, or *name* was supplied that was not a file name or an element in a file. The specification is badly formatted.

`NOT PROGRAM FILE`

SSG expected the specified file to be a program file, and it was not.

`NOT SDF FORMAT`

SSG expected the specified file or element to be in SDF, and it was not.

`NOT SECTOR FORMAT`

SSG processes only sector-formatted input and output files.

`NUM P OPT PCFS > 1`

The P option was used on the @SSG call or a PCF*xxxx* file-id statement, and a program file was already attached to PCF*xxxx*.

`NUM T OPT TCFS > 1`

The T option was used on the @SSG call or a TCF*xxxx* file-id statement, and a program file was already attached to TCF*xxxx*.

`PF, AND NO P OPT`

A program file was specified for a permanent correction file (PCF) set, but the P option was not used.

`PF, AND NO T OPT`

A program file was specified for a temporary correction file (TCF) set, but the T option was not used.

`PF ELT CYCLE ERROR`

The cycle given for the specified program file element is in error.

`PF ELT DELETED`

The specified element has been deleted from the given program file.

`PF ELT NOT FOUND`

The specified element was not found in the program file.

```
P OPT AND ELT/VER
```

SSG expects a program file specification. A specification with an element name or version name, or both, was supplied.

```
P OPT AND NO PF
```

SSG expects a program file specification, and it was not supplied.

```
T OPT AND ELT/VER
```

SSG expects a program file specification. A specification with an element name or version name, or both, was supplied.

```
T OPT AND NO PF
```

SSG expects a program file specification, and it was not supplied.

```
USE OR ASG ERROR
```

An error occurred trying to perform @USE or @ASG operations for the indicated *param*, *specification*, or *name*.

```
/VER NOT NUMERIC
```

Following PCF*xxxx*, TCF*xxxx* or SGS, QGS, or QSGS, */number* was specified but was badly formatted.

```
THIS CREATED ENTRY DOES NOT EXIST IN THE NEW STREAM
```

The entry only exists as a result of a *CREATE image in the skeleton. There are no correction images associated with the entry.

```
THIS INTERNAL ENTRY DOES NOT EXIST IN THE NEW STREAM
```

The stream listed is internal to SSG. To save the stream, a *CORRECT,P or *MERGE,T operation must be performed.

```
*ERROR* - SSG MUST BE TERMINATED WITH AN @EOF
*ERROR* - WHEN *ACCEPT IS USED - SSG TERMINATED
```

In order to perform a read for the *ACCEPT directive, an @EOF must follow the SSG call. If any control image other than an @EOF follows the SSG call, input is not possible so SSG terminates with an ER ERR$.

ERROR - TEXT LENGTH (*n* SECTORS) EXCEEDS MAXIMUM (262143 SECTORS) FOR ELT *element/version* IN *file-id*. A DATA FILE MUST BE USED INSTEAD OF A PROGRAM FILE.

The symbolic or omnibus text to write to PF or LPF program file element *element/version* was *n* sectors in length, exceeding the maximum length for program file elements. The *file-id* contains additional information about the file in error, as shown in Table A-1. As indicated by the *file-id* the corresponding *param-n* on the @SSG call or the corresponding directive in the skeleton must be changed from a program file element (*filename.element*) to a data file (*filename*). Do not use a current program file as data file.

If it is necessary to save the symbolic or omnibus text in a program file, then the data files created by SSG can be copied to a large element program file (LEPF) with FURPUR commands.

- The @CHG,E or @CHG,ET commands can be used to create an LEPF.
- The @COPY,I command can be used to copy a symbolic data file to and from an LEPF symbolic element.
- The @COPY,IO command can be used to copy an omnibus data file to and from an LEPF omnibus element.

SSG does not support reading or writing elements in an LEPF, so the elements must be copied back to data files if SSG is to read them. For additional information about the FURPUR @CHG and @COPY commands, refer to the *Executive Control Language (ECL) and FURPUR Reference Manual*.

ERROR - THE FOLLOWING DIRECTIVE IS INVALID OR DEFINE NAME NOT FOUND

A directive image was encountered that was neither an SSG directive nor a *DEFINE packet. This message also occurs when a *PROCESS directive is encountered, but the *DEFINE packet was not found.

ERROR - THE FOLLOWING IMAGE IS OUT OF SEQUENCE

An image was encountered where it does not belong. For correction streams, this usually indicates an **element name/version name* image is missing; for skeleton corrections, a sequence error.

```
*ERROR* - THE MAXIMUM NUMBER OF SGSS WITH THE SAME LABEL
                  (nnnnnn) HAS BEEN EXCEEDED FOR LABEL sgs-label
```

The number of SGS *sgs-label* exceeded the maximum decimal number *nnnnnn*. The maximum number is not guaranteed to be the same for all SSG levels or when SSG corrections or local code are applied. The maximum number of SSGs with the same label can be computed from the SSG termination line STORAGE field, *e* subfield (see A.1).

Many techniques are available for reducing the number of SGSs with the same label. For example, you can

- Divide the SGSs into groups with two or more different labels.
- Process the SGSs in groups of *nnnnnn* or less. When processing of the group is complete, use the *REMOVE directive to remove them and the *ADD directive to add the next group.
- Reformat the SGSs to reduce the number of instances. For example, you can increase the number of fields and subfields, which are not limited.

$$*\ \text{ERROR}\ * \ -\ \text{UNABLE TO EXCLUSIVELY ASSIGN} \quad \left\{\begin{matrix}\text{PCF}xxxx \\ \\ \text{TCF}xxxx\end{matrix}\right\} \text{AT SKEL TIME } sl$$

The output file for the *CORRECT,P or *MERGE,T directive could not be exclusively assigned. For information about @ASG,AX, refer to the *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949). SSG does not require exclusive use to read a P option PCF or T option TCF. However, upon the first write attempt, exclusive use is required. The line number *sl* of the revised skeleton stream at requested output is displayed.

```
*ERROR* - USE OR ASG ERROR OCCURRED WITH FILE SPECIFIED ON DIRECTIVE
LINE
```

An error occurred while attempting to perform an @USE or an @ASG operation on the file specified on the directive line immediately following this error message.

```
*ERROR* - VARIABLE IS TYPE STRING - NUMERIC NEEDED
```

An attempt was made to use a string variable in a numeric expression such as *SET or *INCREMENT.

A.3. Warning Messages

```
*WARNING* - *COPY,A FILE/ELEMENT NOT FOUND - COPY IGNORED
```

The file or file.element for a *COPY directive with the A option was not found.

```
*WARNING* - ER CONFIG$ ERROR STATUS n - MCN UNAVAILABLE FOR SYSTEM$ SGS
```

Retrieval of the Manufacturing Control Number (MCN) failed with the octal status *n*. For interpretation of the error status refer to the *Exec System Software Executive Requests Programming Reference Manual* (7830 7899). The SYSTEM$ SGS *system-mcn* subfield is set to a null value.

```
*WARNING* - ER MSCON$ DIRID$ FUNCTION ERROR STATUS n - MHFS STATUS
UNAVAILABLE FOR SYSTEM$ SGS
```

The function used to retrieve the Multi-Host File Sharing (MHFS) status failed, with octal status *n* returned in register A0. For interpretation of the error status, refer to the *Exec System Software Administration Reference Manual* (7831 0323). The SYSTEM$ SGS *mhfs-status* is set to a value of NC and *mhfs-host-id* is set to a null value.

```
*WARNING* - ERROR ON USER MARGIN STATEMENT - SSG CONFIGURED (*,*,*)
MARGINS USED
```

Either SSG or the EXEC rejected the MARGIN statement page margin values. The margin values indicated are used, if needed.

```
*WARNING* - HAVE ENCOUNTERED 100 NO FINDS - NO FIND MESSAGES TO BE
SUPRESSED
```

Upon the occurrence of a NO FIND condition, the value of NOFND$ equals 100. This message appears once only; no further NO FIND diagnostics appear.

```
*WARNING* - ILLEGAL OPTION CHARACTER(S) IGNORED:  illegal-option-chars
```

Up to 20 *illegal-option-chars* are output. The characters are either not alphabetic option characters or are alphabetic option characters that are not supported for this statement or directive.

```
*WARNING* - IMAGE GREATER THAN 132 CHAR MAX ALLOWED - IMAGE TRUNCATED
```

An image longer than 132 characters was input. Everything beyond character 132 was discarded.

```
*WARNING* - INVALID SGS LABEL ON FOLLOWING STATEMENT - SGS IGNORED
```

An attempt was made to add or create an SGS with an invalid label name. Currently, the only invalid name is #.

```
*WARNING* - LOCAL VARIABLE TRUNCATED - EXCEEDED 8 CHAR MAX
```

The *variable-name* on an *INCREMENT directive was longer than 8 characters, so only the first 8 characters are used. Variable references ([**variable-name*]) assume the global variable chain if *variable-name* contains more than eight nonspace characters.

```
*WARNING* - NESTING OF *COPY TO FIVE LEVELS ONLY - COPY IGNORED
```

A nesting level of five is allowed for the *COPY directive. Anything deeper is ignored.

```
*WARNING* - NON-STANDARD VARIABLE 'var-name' USED AT LINE x
```

The global variable *var-name* on line *x* has been renamed. *var-name* can be CARD or CARD$, which has been changed to STMT$; FLD, which has been changed to FLD$; or SFLD, which has been changed to SFLD$. The value of the appropriate renamed variable is used for the *var-name* reference.

```
*WARNING* - OPTION(S) 'ignored-options' IGNORED -  OVERRIDDEN BY THE
'option' OPTION
```

The displayed *'option'* has precedence over the displayed *'ignored-options'* and causes them to be ignored.

```
*WARNING* - OPTION(S) 'ignored-options' IGNORED WITHOUT THE 'option'
OPTION
```

The displayed *'ignored-options'* require the displayed *'option'* and are ignored without it.

```
*WARNING* - PERM CORRECTIONS OVERRIDE SKEL CORRECTIONS IN ELEMENT
element-name/version-name
```

During the merging of permanent, temporary, and skeleton-generated corrections, conflicts in line corrections resulted. The conflicts are settled by the priority order: temporary overrides permanent; permanent overrides skeleton. The overridden correction images are also listed. Take note of any conflicts between permanent and skeleton-generated corrections.

```
*WARNING* - POSSIBLE ERROR ON FOLLOWING * IMAGE - ASSUMED TO BE DATA
```

During input of permanent stream and temporary stream element entries, if an **element-name/version-name* image has nonalphabetic characters, nonnumeric characters, a hyphen (-), a dollar sign ($), or is otherwise an invalid element name or version name, this message is displayed. The **element-name/version-name* image is then assumed to be part of the correction stream for the last element entry.

```
*WARNING* - RESULT OF DIVISION UNDEFINED, 0,0 USED
```

An attempt to divide by 0 has been detected. A value of 0 is returned for the quotient and the remainder.

$$*\,\text{WARNING}\,* \;-\text{RESULT OF MULTIPLICATION}\;\left\{\begin{matrix}>\text{ MAX ALLOWED 34359738367 USED}\\<\text{ MIN ALLWED} - 34359738367\text{ USED}\end{matrix}\right\}$$

The result of a multiplication exceeded 36 bits of precision (one single-precision computer word, including sign bit). The value indicated is used.

$$*\,\text{WARNING}\,* \;-\text{RESULT OF NUMWRIC EXPRESSION}\;\left\{\begin{matrix}>\text{ MAX ALLOWED 34359738367 USED}\\<\text{ MIN ALLWED} - 34359738367\text{ USED}\end{matrix}\right\}$$

The final result of a numeric expression exceeded 36 bits of precision (one single-precision computer word, including sign bit). The value indicated is used.

WARNING - SPEC *nn* --> ILLEGAL OPTION CHARACTERS IGNORED: *illegal-opt-chars*

Following the OPT specification at parameter field number *nn*, the "*/extra-options*" contain characters that are not legal additional SSG options (see Table 2-4).

```
*WARNING* - STRING VARIABLE VALUE TRUNCATED TO 132 CHARACTERS
```

After satisfying all references, the length of the string being set (*SET) was greater than 132 characters. All characters after character 132 were discarded.

$$*\,\text{WARNING}\,* -\text{TEMP CORRECTIONS OVERRIDE}\left\{\begin{matrix}\text{PERM}\\\text{SKEL}\end{matrix}\right\}\text{CORRECTIONS IN ELEMENT } element - name/version\ name$$

During the merging of permanent, temporary, and skeleton-generated corrections, conflicts in line corrections resulted. The conflicts are settled by the priority order: temporary overrides permanent; permanent overrides skeleton. The overridden correction images are also listed. Take note of conflicts between permanent, temporary, and skeleton-generated corrections.

```
*WARNING* - THE FOLLOWING DEFINE NAME IS NOT FOUND - *PROCESS,A IGNORED
```

A *PROCESS directive with an A option was interpreted, but the define packet was not found.

```
*WARNING* - UPDATE PERFORMED - RESULTS ARE UNPREDICTABLE
```

This message is placed around correction lines in the revised temporary correction (RTC) stream produced by correction lines in the TCF that changed correction lines in the PCF (for example, -1,2/0,0/2,3). It is not possible for SSG to be sure that these lines will produce the correct results when applied to the updated symbolic element. Therefore, the RTC stream should be examined before being applied.

`*WARNING* - WARNING DETECTED ON LINE sl, ON OR BEFORE COLUMN c`

A warning was detected in the revised skeleton stream at line number *sl* on or before column position *c*. The directive with the warning is displayed followed by a line containing a 'pointer' (^) to column *c*. These displays are followed by a detailed display of the warning condition that was detected.

`*WARNING* - 100 NO FINDS HAVE BEEN ENCOUNTERED - NO FIND MESSAGES TO BE SUPPRESSED`

The value of NOFND$ has reached 100. From this point, no more NO FIND messages will be printed unless the skeleton resets NOFND$.

Appendix B
Maintaining a Permanent TCF

B.1. Introduction

Most software supplied by Unisys is maintained with a set of base level symbolic elements and a permanent correction file (PCF). The PCF is a set of changes that must be applied to the current base level symbolic elements to build the software product. A change to a product is usually in the form of a temporary correction file (TCF). The TCF is applied to the PCF, resulting in a new PCF, which is then applied to the base level symbolic elements in an incremental update. When the PCF (containing fixes and enhancements) becomes too large or meets the specifications for a new release, the following steps are taken:

1. The PCF is applied to the base level symbolic elements.
2. New base level symbolic elements are created.
3. The PCF is erased.

This is called a symbolic update. Stability updates and most feature releases are incremental updates to base level symbolic elements.

B.2. Building a Software Product

The building of a software product is referred to as system generation. An SSG skeleton and a runstream are usually supplied to help generate a runstream that modifies the product. The System Processor SSG Skeleton (SYSKEL) is used for most system processors; the basic runstream is SSGCALL.

There are two types of changes that a user can apply during a build:

- Changes supplied by Unisys

  These include the PCF supplied on the release tape and any changes supplied by Unisys, such as User Communication Form (UCF) fixes.

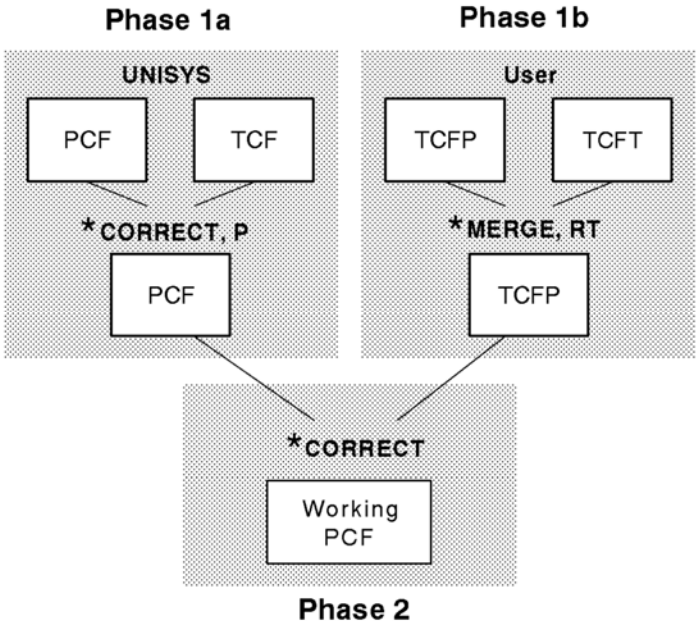
- Changes supplied locally

  These include such items as adjusting configuration tags, modifying algorithms, and adding new features.

The Unisys PCF should be permanently updated only with a Unisys TCF. If local changes are permanently merged into the Unisys PCF, the line numbers of changes subsequently supplied by Unisys could cause conflicts with line numbers in the PCF. This can be avoided by using a two-phase system generation process to maintain a permanent TCF containing local changes.

B.3. Correction Files (Definitions and Relationships)

The term two-phase system generation is used since two distinct @SSG calls may be required to properly maintain PCF, TCF, and the working PCF. This applies to most system generation runstreams. For SYSKEL users, however, the concept of a permanent TCF is assumed and only one @SSG call (SSGCALLT) is needed. The following illustration shows phases 1 and 2:



The process shown in the figure can be summarized as follows:

```
PCF   <-  PCF + TCF    (Phase 1a)
TCFP  <- TCFP + TCFT    (Phase 1b)
WORKING PCF   <- TCFP + PCF    (Phase 2)
```

where:

PCF

is the PCF created by Unisys and initially supplied on the release tape. It is updated locally by applying the TCF, resulting in a new PCF. This new PCF should probably be saved (before phase 2 if SYSKEL is not used) since it replaces the PCF originally supplied by Unisys.

TCF

is the TCF supplied by Unisys, typically in answer to a UCF. It is merged locally with the current PCF, resulting in a new PCF. The line numbers in the TCF are relative to the Unisys working PCF.

TCFP

is the permanent TCF. It contains local modifications to PCF and is modified by the TCFT. Initially, TCFP is empty or is an SSG revised temporary correction (RTC) stream based on the old TCFP and the old PCF (OLDPCF$ for SYSKEL users). The old PCF was used by Unisys to perform the last symbolic update.

TCFT

is the temporary TCF. It is merged locally with the current TCFP, resulting in a new TCFP. The line numbers in the TCFT are relative to the working PCF.

WORKING PCF

is the result of merging the TCFP with the PCF when the system generation is performed. The working PCF need not be saved, just listed. This can be done either by using the R or J options on the @SSG call (if the skeleton saves the working PCF) or by using the V or W options on the generated processor calls.

B.4. Local Use of a Permanent TCF

Although different software products use different system generation procedures, the following steps outline the basic procedure for a typical runstream:

1. Initialize all user files. For SYSKEL users, this is a call on the System Processor File Utility SSG Skeleton (INITIAL).
2. Create the new PCF and the new TCFP. This can be accomplished in the same SSG skeleton. New TCFT entries can modify new TCF entries. For SYSKEL users, this is automatically done by adding (@ADD) TPF$ element SSGCALLT (rather than SSGCALL) in step 4.
3. Save the new PCF. This can automatically be included in the skeleton from step 2 by initially testing for any (primary) TCF entries. Also, to allow phase 2 to correctly select elements (for example, to assemble and compile) you must define a scheme to remember which elements were modified by phase 1. For SYSKEL users, this is automatically done in step 4.
4. Perform the system generation. For SYSKEL users, this is done by adding (@ADD) TPF$ element SSGCALLT. The presence of SSGCALLT (rather than SSGCALL) informs SYSKEL that a permanent TCF exists.
5. Save the results of step 4, the updated version of the software product. If the new TCFP is not cataloged or was not saved in step 3, it should be saved now. For SYSKEL users, this is automatic if the appropriate SYSKEL directives have been supplied; if not, INITIAL can now be used to save any files. The revised PCF from step 4 need not be saved, since all that is needed is the listing of the revised (working) PCF.

The new software product is now built. The new Unisys PCF has been saved, along with the new TCFP. The working PCF is contained in the listings supplied by step 4.

B.5. A Sample Skeleton

The following example is a modified version of an SSG E-option listing of a skeleton capable of performing phase 1 of a two-phase system generation. Input to this skeleton consists of the PCF, TCF, TCFP, TCFT, and any number of secondary TCF sets TCF1,...,TCF*n* and TCFT1,...,TCFT*m*. (The numbers *n* and *m* both have a maximum value of 999 when using this skeleton.) Each TCF set input is merged with itself to remove duplicate element entries.

A UCF response usually consists of a sequence of changes. A change contains code to be introduced into the PCF. A change also contains control information to help in determining how to merge that code into the PCF. Attached to each change is a system number. Placing all changes with the lowest system number in TCF1, then all changes with the next highest system number in TCF2, and so on, causes changes to be properly merged into PCF (by this skeleton). Similar facilities are provided for large scale local modification by entering TCFT1,...,TCFT*m*.

Example

The following changes would be assigned TCF sets as follows:

Change Number	System Number	TCF Set
123 124	12.2.1 12.2.1	TCF1 TCF1
157	12.2.3	TCF2
234 235 236	12.2.5 12.2.5 12.2.5	TCF3 TCF3 TCF3
345	12.2.10	TCF4

Runstream

A possible runstream:

```
@SSG,PT SKEL,,,,,,PCF,PCF.,TCFP,TCFP.
   TCF1 chg123
   TCF1 chg124
   TCF2 chg157
   TCF3 chg234
   TCF3 chg235
   TCF3 chg236
   TCF4 chg345
@EOF
```

Skeleton

```
    SSG   REVISED SKELETON

0001  00  *. This skeleton maintains PCF and TCFP.
0002  00  *.
0003  00  *. PCF must be a P option PCF, and TCFP must be a T option TCF.
0004  00  *.
0005  00  *. PCF is modified by TCF1,...,TCF999 and TCF; TCFP
          by.TCFT1,...,TCFT999 and TCFT.
0006  00  *.
0007  00  *. Only PCF modification is described, since TCFP modification
          is analogous:
0008  00  *.
0009  00  *.   1. Transfer TCF to TCFU. This is accomplished by
                 Merge_duplicates.
0010  00  *. All duplicate element entries are "straight" merged (*MERGE, no
0011  00  *. R option, not a "relative" merge) into TCFU. TCFU then acts as
0012  00  *. a catch all in step 3.
0013  00  *.
0014  00  *.   2. Each TCFi is transferred (*MERGE) to TCFUi by
                 Merge_duplicates.
0015  00  *.
0016  00  *. The TCFU[i] sets and element entries now look something like:
0017  00  *.
0018  00  *.  (All elements in all TCFU[i] sets)
0019  00  *.      E1/V1 E2/V2 E3/V3 E4/V4 E5/V5 E6/V6 E7/V7 ...
0020  00  *. TCFU1  Yes   Yes         Yes
0021  00  *. TCFU2  Yes   Yes   Yes   Yes
0022  00  *. TCFU3        Yes         Yes   Yes
0023  00  *. TCFU4        Yes
0024  00  *.   .
0025  00  *.   .
0026  00  *.   .
0027  00  *. TCFU   Yes   Yes
0028  00  *.
0029  00  *. Where "Yes" indicates the element entry exists in the TCF set.
```

```
0030  00  *.
0031  00  *.   3. Each element entry in TCFU1 is "bubbled" (via a
                 relative merge,
0032  00  *. *MERGE,R) into the first "accepting" TCF set, the first TCF set
0033  00  *. that contains the element entry:  E1/V1 into TCFU2,  E3/V3  into
0034  00  *. TCFU4, E5/V5 into TCFU3.   Since element entries are removed as
0035  00  *. they are processed, the "Yes" line next to TCFU1 is a blank line
0036  00  *. after this  process is complete. Next, all element entries in
0037  00  *. TCFU2 are "bubbled" into "later" TCFU[i] sets, just as TCFU1 was
0038  00  *. handled.  If at any time no TCFUi can be found that contains the
0039  00  *. Ej/Vj that is being "bubbled", it is merged (*MERGE) into TCFU
0040  00  *. (this is why it acts as a "catch all"). All element entries in
0041  00  *. all TCFUi sets are similarly merged. At the end of this process,
0042  00  *. TCFU contains the final, merged TCF. Note that the last defined
0043  00  *. TCFUi is always merged directly into TCFU.
0044  00  *.
0045  00  *. 4. The final TCFU is used to "correct" PCF.
0046  00  *.
0047  00  *. At the end of this process TCF, TCFi, TCFU, TCFUi are all empty!
0048  00  *.
0049  00  *. Throughout this skeleton element entries are referenced using index 1.
0050  00  *. This is because, after each element entry is processed, it is
removed.
0051  00  *Eject,ed  SSG   REVISED SKELETON

SSG   REVISED SKELETON

0052  00  *. Define Find_E_in_T
0053  01  *.
0054  01  *. This define packet performs a "double" column search
0055  01  *. It is used to find an Ej/Vj when "bubbling" element entries into
            "later" TCF sets
0056  01  *.
0057  01  *. [#1] is the set name (label) for the search
0058  01  *. [#2] is the element name to search for in field 1, subfield 1
0059  01  *. [#3] is the version name to search for in field 2, subfield 1
0060  01  *.
0061  01  *. Find_E_in_T does not return STMT$, it returns variable E_is_in_T
0062  01  *. If the search was successful, E_is_in_T equals the entry (statement)
            number
0063  01  *.
0064  01  *. Initialize final answer, assume no match has been found
0065  01  *.
0066  01  *. Clear E_is_in_T
0067  01  *.
0068  01  *. The following increment loop stops when a match is found
0069  01  *. To find a possible match, a "normal" column search is performed
0070  01  *. On a match, field 2 is checked, and if it matches, E_is_in_T is set
0071  01  *. Otherwise, the "normal" column search is again performed
0072  01  *.
0073  01  *. The "normal" column search starts at entry number LOOP
0074  01  *. Note the value of LOOP is set in the body of the increment loop
```

```
0075  01  *.
0076  01  *. If the "normal" column search fails, the increment loop terminates
0077  01  *. This is accomplished by setting the value of LOOP to the "TO value"
0078  01  *.
0079  01  *. Increment LOOP to [[#1]] while E_is_in_T is clear
0080  02  *.
0081  02  *. Check to see if the element name can be found (LOOP initially
            equals 1)
0082  02  *.
0083  02  *If column search from [#1],LOOP for [#2]
0084  03  *.
0085  03  *. Element name found, check if version name matches
0086  03  *.
0087  03  *If [[#1],STMT$,2,1]=[#3]
0088  04  *.
0089  04  *. A match!  Set E_is_in_T to STMT$, and, thereby, terminate iteration
0090  04  *.
0091  04  *Set E_is_in_T to STMT$     0092  03*Else
0093  04  *.
0094  04  *. Version name different, might be more, though -- pickup search here
0095  04  *. (LOOP is incremented by 1 by *Increment, no infinite loop possible)
0096  04  *.
0097  04  *Set LOOP to STMT$
0098  03       *Endif
0099  02     *Else
0100  03       *.
0101  03       *. No match for element name, search has failed, terminate
                  iteration
0102  03       *.
0103  03       *Set LOOP to [[#1]]
0104  02    *Endif
0105  01   *Loop
0106  00 *Enddefine
0107  00 *Eject,ed  SSG   REVISED SKELETON

SSG   REVISED SKELETON

0108  00  *Define Merge_duplicates
0109  01    *.
0110  01    *. This define packet merges (*MERGE) element entries in [#1] into
[#2]
0111  01    *.
0112  01    *. [#2] should be empty prior to calling Merge_duplicates
0113  01    *.
0114  01    *. Called from Merge_P_with_next_T, it merges TCF[T][i] into TCFU[i]
0115  01    *.
0116  01    *. Don't do anything if there are no element entries in [#1]
0117  01    *.
0118  01    *If [[#1]]
0119  02      *.
0120  02      *. To avoid a "no find" referencing [#2], ensure it exists
0121  02      *.
```

```
0122  02      *Merge [[#1],1,1,1]/[[#1],1,2,1] [#1] giving [#2]
0123  02      *Remove temp [[#1],1,1,1]/[[#1],1,2,1] from [#1]
0124  02      *.
0125  02      *. For each remaining entry in [#1], transfer it, via *MERGE, to
[#2]
0126  02      *. (Note that "subsequent" images are treated as "temporary"
              images, i.e.,
              *. conflicts are resolved in favor of images in "later" element
                 entries.)
0127  02      *.
0128  02      *.
0129  02      *Increment T to [[#1]]
0130  03        *Merge [[#1],1,1,1]/[[#1],1,2,1] [#1] with [#2] giving [#2]
0131  03        *Remove temp [[#1],1,1,1]/[[#1],1,2,1] from [#1]
0132  02      *Loop
0133  01    *Endif
0134  00  *Enddefine
0135  00  *Eject,ed  SSG   REVISED SKELETON

SSG   REVISED SKELETON

0136  00  *Define Generate_TCFU
0137  01    *.
0138  01    *. This define packet controls generation of TCFU from the input
              TCF[T][i] sets
0139  01    *.
0140  01    *. [#1] is the "base" TCF set (TCF or TCFT)
0141  01    *.
0142  01    *. There is a max of 999 TCF sets per "base" TCF set
0143  01    *. This is because TCFU999 (7 characters) is the "largest" set name
              allowed
0144  01    *.
0145  01    *. First build the initial TCFU from the "base" TCF
0146  01    *.
0147  01    *Process Merge_duplicates [#1] TCFU
0148  01    *.
0149  01    *. To determine which TCF[T]i sets exist, an SGS with label SET is
              used
0150  01    *. SET contains TCF[T]i in field 1, subfield 1, and TCFUi in field 1,
              subfield 2
0151  01    *. The following increment loop iterates until GO is clear or 999 TCF
              sets are checked
0152  01    *. GO, set to the number of TCF[T]i entries, becomes zero when a
              TCF[T]i set is empty
0153  01    *. All SET SGSs are removed prior to the loop, and the last one is
              removed at the end
0154  01    *.
0155  01    *Set GO
0156  01    *Remove SGS SET,1,[SET]
0157  01    *Increment S to 999+1 while GO is set
0158  02      *Create SGS: SET [#1][*S],TCFU[*S]
0159  02      *Set GO to [[SET,S,1,1]]
```

```
0160  01    *Loop
0161  01    *Remove SGS SET,[SET]
0162  01    *.
0163  01    *. All TCF[T]i and TCFUi pairs are defined, saved in label SET
0164  01    *. SET can now be used to define set names for merge operations, as
                follows:
0165  01    *.
0166  01    *. First, transfer TCF[T]i into TCFUi, merging (*MERGE) duplicate
                element entries
0167  01    *.
0168  01    *Increment P to [SET]
0169  02      *Process Merge duplicates [SET,P,1,1] [SET,P,1,2]
0170  01    *Loop
0171  01    *.
0172  01    *. Second, bubble, via *MERGE,R, TCF[T]i into
            *. TCFUi+1,TCFUi+2,...,TCFU
0173  01    *.
0174  01    *Increment P to [Set]
0175  02      *Process Merge_P_with_next_T [Set,P,1,2]
0176  01    *Loop
0177  01    *.
0178  01    *. At this point, TCFU contains the final, merged TCF[T]
0179  01    *.
0180  00  *Enddefine
0181  00  *Eject,ed SSG  REVISED SKELETON

  SSG  REVISED SKELETON

  0182  00  *Define Merge_P_with_next_T
  0183  01    *.
  0184  01    *.  This define packet merges Ej/VJs in TCFUi into
              *.  TCFU+1,...,TCFU
  0185  01    *.
  0186  01    *.  [#1] is the name of the "permanent" stream, TCFUi
  0187  01    *.
  0188  01    *. It is assumed that TCFUi occupies the P'th slot of the SET SGSs
  0189  01    *. In this way, sets after the P'th are searched for a matching
                element entry
  0190  01    *. Merge_P_with_next_T needs to "know" P, in order to "know" what
                TCF sets are "later" TCF sets
  0191  01    *.
  0192  01    *. Now, for each element entry in TCFUi ([#1])
  0193  01    *.
  0194  01    *Increment E to [[#1]]
  0195  02      *.
  0196  02      *. Indicate we are looking for the next TCF set containing Ej/Vj
  0197  02      *.
  0198  02      *Set Looking_for_T
  0199  02      *.
  0200  02      *. Starting at TCFUi+1, look for a match for Ej/Vj, terminating
                  the loop when a match is found
  0201  02      *.
```

```
0202  02      *Increment T from P+1 to [SET] while Looking_for_T is set
0203  03        *.
0204  03        *. Check to see if Ej/Vj is in TCFUi+1 ([SET,P+1,1,2],...)
0205  03        *.
0206  03        *Process Find_E_in_T [SET,T,1,2] [[#1],1,1,1] [[#1],1,2,1]
0207  03        *.
0208  03        *. If it is, stop looking, do the transfer, and pick up next
                   element entry
0209  03        *.
0210  03        *If E_is_in_T is set
0211  04  *Clear Looking_for_T
0212  04  *Merge,r [[#1],1,1,1]/[[#1],1,2,1] [SET,T,1,2] with [#1] giving
           [SET,T,1,2]
0213  03        *End
0214  03        *.
0215  03        *. If no match was found, TCFUi+2 is searched (T becomes P+2),
                   and so on
0216  03        *.
0217  02      *Loop
0218  02      *.
0219  02      *. If the element entry still isn't found, transfer it to TCFU
0220  02      *. (Note that the last TCF set is always transferred to TCFU)
0221  02      *.
0222  02      *If Looking_for_T is set
0223  03        *Merge [[#1],1,1,1]/[[#1],1,2,1] TCFU with [#1] giving TCFU
0224  02      *Endif
0225  02      *.
0226  02      *. And, as usual, remove the entry no longer needed
0227  02      *.
0228  02      *Remove temp [[#1],1,1,1]/[[#1],1,2,1] from [#1]
0229  01    *Loop
0230  00  *Enddefine
0231  00  *Eject,ed  SSG   REVISED SKELETON

SSG   REVISED SKELETON

0232  00  *. Always set PILO$ because of the PLUS commenting convention
0233  00  *.
0234  00  *Set PILO$
0235  00  *.
0236  00  *. Ensure TCFU exists so "no finds" are no concern on *MERGE and
             *CORRECT directives
0237  00  *. (Although this causes I/O,  it is the only way to define TCFU
             from this skeleton)
0238  00  *.
0239  00  *Merge E/V giving TCFU
0240  00  *Remove temp E/V from TCFU
0241  00  *.
0242  00  *. Build TCFU from TCF1,...,TCF999,TCF and modify PCF with TCFU
0243  00  *.
0244  00  *Process Generate_TCFU TCF
0245  00  *If [TCFU]
```

```
0246  01    *If not [PCF,0,F]
0247  02      *Message,iet *ERROR* PCF has no program file attached!
0248  01    *Endif
0249  01    *Increment T to [TCFU]
0250  02      *Correct,p [TCFU,1,1,1]/[TCFU,1,2,1] with TCFU giving PCF
0251  02      *Endcorrect
0252  02      *Remove temp [TCFU,1,1,1]/[TCFU,1,2,1] from TCFU
0253  01    *Loop
0254  01    #PACK,IZ  PCF.
0255  00  *Endif
0256  00  *.
0257  00  *. Build TCFU from TCFT1,...,TCFT999,TCFT and modify TCFP with TCFU
0258  00  *.
0259  00  *Process Generate_TCFU TCFT
0260  00  *If [TCFU]
0261  01   *If not [TCFP,0,F]
0262  02      *Message,iet *ERROR* TCFP has no program file attached!
0263  01    *End
0264  01    *Increment T to [TCFU]
0265  02      *Merge,tr [TCFU,1,1,1]/[TCFU,1,2,1] TCFU with TCFP giving TCFP
0266  02      *Remove temp [TCFU,1,1,1]/[TCFU,1,2,1] from TCFU
0267  01    *Loop
0268  01    #PACK,IZ  TCFP.
0269  00  *Endif
```

Comments

1. The S option on the @SSG call is not always used. If used, it must be in conjunction with the X option. Otherwise, SET should be modified (for example, use subfields 2 and 3).
2. Some skeletons ignore version names on element entries (for example, SYSKEL level 5R1). This skeleton does not; for example, entry A is treated as an entry different from A/B.
3. This skeleton does not assume sorted TCF input. A sorted TCF is desirable. If sorted, the X option should be used for library elements. If TCFs are sorted, the user must merge changes with the same system number prior to using this skeleton. A TCF set is assigned to each system number. The skeleton first merges (*MERGE) changes with the same system number, (TCF1 with TCF1 giving TCFU1, TCF2 with TCF2 giving TCFU2, ...) then merges (*MERGE,R) system numbers (TCFU2 with TCFU1 giving TCFU2, TCFU3 with TCFU2 giving TCFU3, ...).

Note: *SKEL assumes FURPUR level 28R1 or later with the Z option on @PACK images.*

Appendix C
Examples

This appendix gives examples for the use of SSG.

C.1. Introduction

SSG is well suited to perform

- Complex, demanding tasks that require a significant design phase
- Quick, spur-of-the-moment data generation

Both types of applications use symbolic streams (SGS streams) and a skeleton to direct the generation of one or more symbolic streams.

An SGS stream and a skeleton stream are required to perform a particular task, so you must build a structure (made of SGSs) and an interpreter (skeleton).

To design the structure, you must be familiar with

- Allowable SGS formats
- Entities already in SGS format
- The ease with which a symbolic stream can be edited to make it look like an SGS stream
- Features built into SSG to ease referencing the structure

Typically, tasks involving program files are easy to implement with SSG since SSG communicates directly with the operating system. To build an SGS stream based on elements in a program file, place *filename./label* on the @SSG call or on an SGS file-id.

In other cases, symbolic streams are already in SGS format or require a minor change. The following sequence shows a trial attempt at making a symbolic stream appear to SSG as a format with the label *label*:

```
@ED,N current-symbolic-stream,SGS-formatted-symbolic-stream
Change /./''.''/ All
Change /;/'';''/ All
Change //label / Rep
@SSG  some skeleton stream,SGS-formatted-symbolic-stream
@EOF
```

Depending on how long you need it, skeleton design can be

- Ignored (use the skeleton once, possibly edit the generated output stream)
- Given careful attention (save the skeleton for reuse)
- Designed by task force (document, distribute to users)

Note: *Some of the examples shown in this appendix can be found in the utility file of the SSG release tape.*

C.2. Tic Tac Toe Example

The following Tic Tac Toe example allows interaction with the skeleton, with flow of control in the skeleton controlled by input. Error checking is done on the input. If the input is incorrect, it is resolicited. This skeleton is intended to work on UTS 200, UTS 400, and UTS 20 terminals. It makes extensive use of string variables, including nonprintable ASCII characters.

```
SSG    REVISED SKELETON

  1.   0   *define initialize
  2.   1   | *.
  3.   1   | *.   Set up cursor positions for each move
  4.   1   | *.
  5.   1   | *set pos11 =  c'033013040056'
  6.   1   | *set pos12 =  c'033013040101'
  7.   1   | *set pos13 =  c'033013040125'
  8.   1   | *.
  9.   1   | *set pos21 =  c'033013047056'
 10.   1   | *set pos22 =  c'033013047101'
 11.   1   | *set pos23 =  c'033013047125'
 12.   1   | *.
 13.   1   | *set pos31 =  c'033013056056'
 14.   1   | *set pos32 =  c'033013056101'
 15.   1   | *set pos33 =  c'033013056125'
 16.   1   | *.
 17.   1   | *set si =     c'017000000000'
 18.   1   | *set d = c'033151'
 19.   1   | *set u = c'033146'
 20.   1   | *set b = c'033147'
 21.   1   | *.
 22.   1   | *  Clear the position flags
 23.   1   | *.
 24.   1   | *increment x to 3
 25.   2   | | *increment y to 3
 26.   3   | | | *set board[*x][*y] = 0
 27.   2   | | *loop
 28.   1   | *loop
 29.   1   | *.
 30.   1   | *set stringchar$ = ','
 31.   1   | *set esc = c'033'
```

```
 32.   1   | *set down = '[*esc]i'
 33.   1   | *set left = '[*esc]g'
 34.   1   | *set clear = '[*esc]e[*esc]M'
 35.   1   | *set ers = c'033142'
 36.   1   | *set eod = c'033115'
 37.   1   | *set nul = c'000'
 38.   1   | *set rs = c'036'
 39.   1   | *.
 40.   1   | * Various cursor positions for drawing grids, etc.
 41.   1   | *.
 42.   1   | *set input_pos = c'033013067040017000000000'
 43.   1   | *set error_pos = c'033013066060017000000000'
 44.   1   | *set start_1   = c'033013040071017000000000'
 45.   1   | *set start_2   = c'033013040123017000000000'
 46.   1   | *set start_3   = c'033013045056017000000000'
 47.   1   | *set start_4   = c'033013055056017000000000'
 48.   1   | *display '[*clear]'
 49.   0   *enddefine
 50.   0   *.
 51.   0   *eject

SSG    REVISED SKELETON

 52.   0   *.
 53.   0   *define draw_grid
 54.   1   | *display '[*start_1]'
 55.   1   | *set one_v = '|[*down][*left][*nul][*nul][*nul]'
 56.   1   | *increment a to 2
 57.   2   | | *display one_v one_v one_v one_v one_v one_v one_v one_v
               one_v one_v
 58.   1   | *loop
 59.   1   | *display '[*start_2]'
 60.   1   | *increment a to 2
 61.   2   | | *display one_v one_v one_v one_v one_v one_v one_v one_v
               one_v one_v
 62.   1   | *loop
 63.   1   | *display '[*start_3]'
 64.   1   | *display '================================================'
 65.   1   | *display '[*start_4]'
 66.   1   | *display '================================================'
 67.   0   *enddefine
 68.   0   *.
 69.   0   *define cleanup
 70.   1   | *set ans = 'NO'                    . in case eof has occurred
 71.   1   | *accept,e ans '[*input_pos] Play again? [*rs][*ers]'
 72.   1   | *if [*ans,upper_case_string$]=YES or [*ans,upper_case_string$]=Y
 73.   2   | | *process skel$
 74.   1   | *endif
 75.   1   | *pcontrol 'd,@@end esc'
 76.   1   | *display '[*eod]'
 77.   0   *enddefine
 78.   0   *.
```

```
 79.   0   *define draw_x
 80.   1   | *display pos[#1][#2] si 'x [*d]x [*d]x [*d]x [*d]x
           [*nul][*nul]'
 81.   1   | *display pos[#1][#2] si '[*d][*d][*d][*d]x [*u]x [*u]x
           [*u]x [*u]x[*nul][*nul][*nul]'
 82.   1   | *set board[#1][#2]=1
 83.   0   *enddefine
 84.   0   *.
 85.   0   *define draw_o
 86.   1   | *display  pos[#1][#2] si '       O[*d][*b][*b][*b]O
           O[*nul][*nul]'
 87.   1   | *display '[*d][*b][*b][*b][*b][*b]O
           O[*d][*b][*b][*b]O[*nul][*nul]'
 88.   1   | *set board[#1][#2]=2
 89.   0   *enddefine
 90.   0   *.
 91.   0   *define opponent_move
 92.   1   | *.
 93.   1   | *increment lp to 1000
 94.   2   | | *accept,e move '[*input_pos]  Your move  [*rs][*ers]'
 95.   2   | | *if eof$ is set
 96.   3   | | | *return
 97.   2   | | *endif
 98.   2   | | *display '[*error_pos]         '
 99.   2   | | *set x_string = '[*move,before_string$]'
100.   2   | | *set y_string = '[*move,after_string$]'
101.   2   | | *.
102.   2   | | *set xl=[*x_string,numeric_length$]
103.   2   | | *set yl=[*y_string,numeric_length$]
104.   2   | | *if not +xl=1 or not +yl=1
105.   3   | | | *display error_pos move ' is not a valid move'
106.   3   | | | *iterate
107.   2   | | *endif
108.   2   | | *.
109.   2   | | *set x_pos = [*move,before_string$]
110.   2   | | *set y_pos = [*move,after_string$]
111.   2   | | *.
112.   2   | | *if +x_pos>3 or +x_pos<1 or +y_pos>3 or +y_pos<1
113.   3   | | | *display error_pos  'Coordinates must be between 1 and 3'
114.   3   | | | *iterate
115.   2   | | *endif
116.   2   | | *.
117.   2   | | *set x = board[*x_pos][*y_pos]
118.   2   | | *if +x=0
119.   3   | | | *draw_x  [*x_pos],[*y_pos]
120.   3   | | | *.
121.   2   | | *else
122.   3   | | | *display error_pos 'There is already a mark
                 at[*x_pos],[*y_pos]'
123.   3   | | | *iterate
124.   2   | | *endif
125.   2   | | *return
```

```
126.   1   | *loop
127.   1   | *.
128.   0   *enddefine
129.   0   *.
130.   0   *eject

SSG    REVISED SKELETON

131.   0   *.
132.   0   *             MAIN SKELETON ROUTINE
133.   0   *.
134.   0   *initialize
135.   0   *pcontrol 'd,@@esc o'
136.   0   *draw_grid
137.   0   *.
138.   0   *draw_o 3,1
139.   0   *opponent_move
140.   0   *if eof$ is set
141.   1   | *cleanup
142.   1   | *return
143.   0   *endif
144.   0   *.
145.   0   *eject

SSG    REVISED SKELETON

146.   0   *.
147.   0   *if +board11=1
148.   1   | *draw_o 1,3
149.   1   | *opponent_move
150.   1   | *if eof$ is set
151.   2   | | *cleanup
152.   2   | | *return
153.   1   | *endif
154.   1   | *if +board22=0
155.   2   | | *draw_o 2,2
156.   1   | *else
157.   2   | | *draw_o 3,3
158.   2   | | *opponent_move
159.   2   | | *if eof$ is set
160.   3   | | | *cleanup
161.   3   | | | *return
162.   2   | | *endif
163.   2   | | *if +board32=1
164.   3   | | | *draw_o 2,3
165.   2   | | *elseif +board23 = 1
166.   3   | | | *draw_o 3,2
167.   2   | | *else
168.   3   | | | *draw_o 2,3
169.   2   | | *endif
170.   1   | *endif
171.   1   | *display error_pos 'I win, thanks for the game'
```

```
172.   1   | *cleanup
173.   1   | *return
174.   1   | *.
175.   1   | *eject

SSG    REVISED SKELETON

176.   1   | *.
177.   0   *elseif +board21=1
178.   1   | *draw_o 3,3
179.   1   | *opponent_move
180.   1   | *if eof$ is set
181.   2   | | *cleanup
182.   2   | | *return
183.   1   | *endif
184.   1   | *if +board32 = 0
185.   2   | | *draw_o 3,2
186.   1   | *else
187.   2   | | *draw_o 2,2
188.   2   | | *opponent_move
189.   2   | | *if eof$ is set
190.   3   | | | *cleanup
191.   3   | | | *return
192.   2   | | *endif
193.   2   | | *if +board11=1
194.   3   | | | *draw_o 1,3
195.   2   | | *elseif +board13=1
196.   3   | | | *draw_o 1,1
197.   2   | | *else
198.   3   | | | *draw_o 1,1
199.   2   | | *endif
200.   1   | *endif
201.   1   | *display error_pos 'I win, I''ve had a lot of fun'
202.   1   | *cleanup
203.   1   | *return
204.   1   | *.
205.   1   | *eject

SSG    REVISED SKELETON

206.   1   | *.
207.   0   *elseif +board12=1
208.   1   | *draw_o 1,1
209.   1   | *opponent_move
210.   1   | *if eof$ is set
211.   2   | | *cleanup
212.   2   | | *return
213.   1   | *endif
214.   1   | *if +board12 = 0
215.   2   | | *draw_o 1,2
216.   1   | *else
217.   2   | | *draw_o 2,2
```

```
218.   2   | | *if +board13=1
219.   3   | | | *draw_o 3,3
220.   2   | | *elseif +board33=1
221.   3   | | | *draw_o 1,3
222.   2   | | *else
223.   3   | | | *draw_o 1,3
224.   2   | | *endif
225.   1   | *endif
226.   1   | *display error_pos 'Thanks for the game, it was a real challenge'
227.   1   | *cleanup
228.   1   | *return
229.   1   | *.
230.   1   | *eject

SSG    REVISED SKELETON

231.   1   | *.
232.   0   *elseif +board32=1
233.   1   | *draw_o 1,1
234.   1   | *opponent_move
235.   1   | *if eof$ is set
236.   2   | | *cleanup
237.   2   | | *return
238.   1   | *endif
239.   1   | *if +board21 = 0
240.   2   | | *draw_o 2,1
241.   1   | *else
242.   2   | | *draw_o 2,2
243.   2   | | *opponent_move
244.   2   | | *if eof$ is set
245.   3   | | | *cleanup
246.   3   | | | *return
247.   2   | | *endif
248.   2   | | *if +board13=1
249.   3   | | | *draw_o 3,3
250.   2   | | *elseif +board33=1
251.   3   | | | *draw_o 1,3
252.   2   | | *else
253.   3   | | | *draw_o 1,3
254.   2   | | *endif
255.   1   | *endif
256.   1   | *display error_pos 'I just got lucky, you''re a good player'
257.   1   | *cleanup
258.   1   | *return
259.   1   | *.
260.   1   | *eject

SSG    REVISED SKELETON

261.   1   | *.
262.   0   *elseif +board13=1
263.   1   | *draw_o 3,3
```

```
264.   1   | *opponent_move
265.   1   | *if eof$ is set
266.   2   | | *cleanup
267.   2   | | *return
268.   1   | *endif
269.   1   | *if +board32 = 0
270.   2   | | *draw_o 3,2
271.   1   | *else
272.   2   | | *draw_o 1,1
273.   2   | | *opponent_move
274.   2   | | *if eof$ is set
275.   3   | | | *cleanup
276.   3   | | | *return
277.   2   | | *endif
278.   2   | | *if +board22=1
279.   3   | | | *draw_o 2,1
280.   2   | | *elseif +board12=1
281.   3   | | | *draw_o 2,2
282.   2   | | *else
283.   3   | | | *draw_o 2,1
284.   2   | | *endif
285.   1   | *endif
286.   1   | *display error_pos 'I sure like this game, thanks'
287.   1   | *cleanup
288.   1   | *return
289.   1   | *.
290.   1   | *eject

SSG    REVISED SKELETON

291.   1   | *.
292.   0   *elseif +board23=1
293.   1   | *draw_o 1,1
294.   1   | *opponent_move
295.   1   | *if eof$ is set
296.   2   | | *cleanup
297.   2   | | *return
298.   1   | *endif
299.   1   | *if +board21 = 0
300.   2   | | *draw_o 2,1
301.   1   | *else
302.   2   | | *draw_o 2,2
303.   2   | | *opponent_move
304.   2   | | *if eof$ is set
305.   3   | | | *cleanup
306.   3   | | | *return
307.   2   | | *endif
308.   2   | | *if +board13=1
309.   3   | | | *draw_o 3,3
310.   2   | | *elseif +board33=1
311.   3   | | | *draw_o 1,3
312.   2   | | *else
```

```
313.   3   | | | *draw_o 1,3
314.   2   | | *endif
315.   1   | *endif
316.   1   | *display error_pos 'Another win for the champ'
317.   1   | *cleanup
318.   1   | *return
319.   1   | *.
320.   1   | *eject

SSG    REVISED SKELETON

321.   1   | *.
322.   0   *elseif +board33=1
323.   1   | *draw_o 1,1
324.   1   | *opponent_move
325.   1   | *if eof$ is set
326.   2   | | *cleanup
327.   2   | | *return
328.   1   | *endif
329.   1   | *if +board21 = 0
330.   2   | | *draw_o 2,1
331.   1   | *else
332.   2   | | *draw_o 1,3
333.   2   | | *opponent_move
334.   2   | | *if eof$ is set
335.   3   | | | *cleanup
336.   3   | | | *return
337.   2   | | *endif
338.   2   | | *if +board22=1
339.   3   | | | *draw_o 1,2
340.   2   | | *elseif +board12=1
341.   3   | | | *draw_o 2,2
342.   2   | | *else
343.   3   | | | *draw_o 1,3
344.   2   | | *endif
345.   1   | *endif
346.   1   | *display error_pos 'You''re a tough player, but I''m tough too'
347.   1   | *cleanup
348.   1   | *return
349.   1   | *.
350.   1   | *eject

SSG    REVISED SKELETON

351.   1   | *.
352.   0   *else
353.   1   | *clear win
354.   1   | *draw_o 1,1
355.   1   | *opponent_move
356.   1   | *if eof$ is set
357.   2   | | *cleanup
358.   2   | | *return
```

```
359.   1   | *endif
360.   1   | *if +board21=0
361.   2   | | *draw_o 2,1
362.   2   | | *set win
363.   1   | *else
364.   2   | | *draw_o 2,3
365.   2   | | *opponent_move
366.   2   | | *if eof$ is set
367.   3   | | | *cleanup
368.   3   | | | *return
369.   2   | | *endif
370.   2   | | *if +board12=1
371.   3   | | | *draw_o 3,2
372.   3   | | | *opponent_move
373.   3   | | | *if eof$ is set
374.   4   | | | | *cleanup
375.   4   | | | | *return
376.   3   | | | *endif
377.   3   | | | *if +board33=0
378.   4   | | | | *draw_o 3,3
379.   4   | | | | *set win
380.   3   | | | *endif
381.   2   | | *elseif +board32=1
382.   3   | | | *draw_o 1,2
383.   3   | | | *opponent_move
384.   3   | | | *if eof$ is set
385.   4   | | | | *cleanup
386.   4   | | | | *return
387.   3   | | | *endif
388.   3   | | | *if +board31=0
389.   4   | | | | *draw_o 3,1
390.   4   | | | | *set win
391.   3   | | | *endif
392.   2   | | *elseif +board13=1
393.   3   | | | *draw_o 3,3
394.   3   | | | *opponent_move
395.   3   | | | *if eof$ is set
396.   4   | | | | *cleanup
397.   4   | | | | *return
398.   3   | | | *endif
399.   3   | | | *if +board32=0
400.   4   | | | | *draw_o 3,2
401.   4   | | | | *set win
402.   3   | | | *endif
403.   2   | | *else
404.   3   | | | *draw_o 1,3
405.   3   | | | *opponent_move
406.   3   | | | *if eof$ is set
407.   4   | | | | *cleanup
408.   4   | | | | *return
409.   3   | | | *endif
410.   3   | | | *if +board12=0
```

```
411.   4   | | | | *draw_o 1,2
412.   4   | | | | *set win
413.   3   | | | *endif
414.   2   | | *endif
415.   1   | *endif
416.   1   | *if win is set
417.   2   | | *display error_pos 'You almost got me that time'
418.   1   | *else
419.   2   | | *display error_pos 'Darn CAT won again, you sure are
               good though'
420.   1   | *endif
421.   1   | *.
422.   0   *endif
423.   0   *cleanup
```

Skeleton Analysis by Line Number

5.

The variable pos11 is set to a string of nonprintable ASCII characters. These are the cursor positioning sequence (when combined with the value of SI) for move 1,1. The cursor positioning sequence is ESC (033) UT (013) row (040) column (056) and SI (017).

30.

STRINGCHAR$ is set to a comma. This will be used later by the functions BEFORE_STRING$ and AFTER_STRING$.

54.

The cursor is positioned at row 1, column 9 to start printing the first vertical grid line.

93.

LP is incremented to 1000. This allows control to be transferred to the top of the loop if the input is not valid.

94.

The next move is solicited. The expected format is R,C.

96.

If a control statement was input, control is returned to the statement after the call to this define packet.

106.

If the move was not a valid move, the next iteration of the loop takes place. The move is requested again.

135.

Escape mode on output is entered to allow the program to perform all screen handling functions.

149.

The define packet 'OPPONENT_MOVE' is called. This line has the same affect as '*PROCESS opponent_move'.

171.

The cursor is positioned to line 23, column 18, and the message 'I win, thanks for the game' is printed.

370.

Board position 1,2 is checked to see if the user put a mark there. If so, a 'O' is drawn at position 3,2.

C.3. File Lister Example

The following example is an interactive skeleton that allows all the symbolic elements in a program file to be listed with an @PRT,S directive. The skeleton asks for the file names to be listed, asks for the number of copies to be printed, and asks where the listing is to be printed.

```
SSG    REVISED SKELETON

 1.   0   *display .
 2.   0   *display '     Program file lister'
 3.   0   *display .
 4.   0   *set stringchar$ = '.'
 5.   0   *clear iopt$
 6.   0   #cat,p brk$pr$$(+1).,///3000
 7.   0   #brkpt print$/brk$pr$$
 8.   0   *.
 9.   0   *increment a$$ to 1000
10.   1   | *.
11.   1   | *remove sgs element$
12.   1   | *accept,e ans 'Program file to list <end>'
13.   1   | *.
14.   1   | *if +eof$ <> 0
15.   2   | | *display 'Lister cancelled'
16.   2   | | *display .
17.   2   | | *display 'End file lister'
18.   2   | | *clear add_gos$
19.   2   | | *return
20.   1   | *endif
21.   1   | *.
22.   1   | *if +[*ans,length$]=0
23.   2   | | *exit
24.   1   | *endif
25.   1   | *.
26.   1   | *set ans = '[*ans,before_string$]'       . make sure there
```

```
            is no period
27.   1   | *csf @asg,az [*ans]                . does the file exist?
28.   1   | *if +csfsta$<0
29.   2   | | *display 'Unable to assign file ''[*ans].'' - listing not
              done'
30.   2   | | *iterate a$$
31.   1   | *endif
32.   1   | *.
33.   1   | *add sgs [*ans]./element$        . add the TOC as SGSs
34.   1   | *sort,l sgs element$          . sort, including version
35.   1   | *increment elt to [element$]
36.   2   | | *if [element$,elt,3,1] <= 4
37.   3   | | | #hdg,p      [element$,elt,7,1].[element$,elt,6,1]
38.   3   | | | #prt,s [element$,elt,7,1].[element$,elt,6,1]
39.   2   | | *endif
40.   1   | *loop
41.   1   | *.
42.   0   *loop
43.   0   *.
44.   0   #brkpt print$
45.   0   *accept,e copies 'Number of copies to print <none> ? '
46.   0   *if eof$ is set
47.   1   | *display 'Lister cancelled'
48.   1   | *clear add_gos$
49.   0   *else
50.   1   | *.
51.   1   | *if +[*copies,numeric_length$]>0
52.   2   | | *accept,e device 'Printer <PR> ? '
53.   1   | *endif
54.   1   | *if eof$ is set
55.   2   | | *display 'Lister cancelled'
56.   2   | | *clear add_gos$
57.   2   | | *return
58.   1   | *else
59.   2   | | *.
60.   2   | | *if +[*copies,numeric_length$]>0
61.   3   | | | #sym brk$pr$$.,[*copies],[*device]
62.   3   | | | *display 'Print will be sym''ed to [*device] by
                  [info$,1,1,2]'
63.   2   | | *endif
64.   1   | *endif
65.   0   *endif
66.   0   *.
67.   0   *display .
68.   0   *display 'End file lister'
```

Skeleton Analysis by Line Number

5.

Suppress printing of SGSs that are added.

9.

A$$ is incremented to 1000. This permits the question to be asked repeatedly.

18.

If a control statement is input, ADD_GOS$ is set to 0. This causes the generated output stream not to be added (@ADD).

19.

If a control statement is input, the skeleton is terminated at this point.

33.

The TOC of the file input is passed to SSG as SGSs with the label ELEMENT$.

34.

The SGSs just added are sorted across all fields so the listed elements are alphabetically ordered by element and version.

C.4. A Correcting Skeleton (CORRECTSKEL)

CORRECTSKEL was written for ease of correcting (*CORRECT) multiple PCF sets.

Features

- The value of PCFNAME$ is irrelevant.
- By varying [#6], the input permanent streams can be left unchanged. This could prove to be useful in conjunction with *CORRECT,P.
- The keyword ALL directs the *CORRECT directive to revise all permanent streams for which there is a corresponding temporary stream and allows for extensions using *ELSF.

Runstream

```
@SSG,P  saved skeleton  ,,,,,,PCFxxxx  ,filename  .
TCFxxxx  SDF stream
@EOF
```

Skeleton

```
SSG   REVISED SKELETON

0001  00   *.This skeleton assumes no duplicate element entries exist
0002  00   *.
0003  00   *Define Correct
0004  01     *.
0005  01     *. *Process CORRECT <keyword> [#2] with [#4] giving [#6]
0006  01     *.
0007  01     *.  where:  <keyword>  ALL  All elements in [#2] are
                "corrected" by [#4], or
0008  01     *          Anything besides ALL, by adding *ELSF
directives
0009  01     *      [#2]    Any PCF set
0010  01     *      [#4]    Any TCF set
0011  01     *      [#6]    Any PCF set
0012  01     *.
0013  01     *. Verify result of using Correct, that modification is
                remembered
0014  01     *.
0015  01     *If not [[#6],0,F]
0016  02       *Message,iet Output PCF set [#6] has no program file
                attached!
0017  01     *End
0018  01     *.
0019  01     *. Process as directed by <keyword>
0020  01     *.
0021  01     *If [#1] = ALL
0022  02       *.
0023  02       *. For each element entry in the temporary stream
0024  02       *.
0025  02       *Increment E to [[#4]]
0026  03         *Correct,p [[#4],E,1,1]/[[#4],E,2,1] [#2] with [#4]
                  giving [#6]
0027  03         *Endcorrect
0028  02       *Loop
0029  01     *Endif
0030  00   *Enddefine
0031  00   *Eject,ed

0032  00   *. For each TCF set, "correct" the corresponding PCF set
0033  00   *.
0034  00   *Increment T to [$TCF]
0035  01     *.
0036  01     *. Only process those sets with element entries (could be
                TCF, ...)
0037  01     *.
0038  01     *If [[$TCF,T,1,1]]
0039  02       *.
0040  02       *. Create the corresponding PCF set (TCFxxxx -> PCFxxxx)
0041  02       *.
```

```
0042  02       *Create SGS: PSET PCF[$TCF,T,1,1,0,4,[$TCF,T,1,1,5]-3]
0043  02       *.
0044  02       *. Carry out action assumed ("correct all")
0045  02       *.
0046  02       *Process Correct ALL [PSET,1,1,1] with [$TCF,T,1,1]
                giving [PSET,1,1,1]
0047  02       *.
0048  02       *. In case the sort is on, remove PSET SGSs (always use
index 1)
0049  02       *.
0050  02       *Remove SGS PSET
0051  01     *Endif
0052  00   *Loop
```

C.5. UTS Calculator Skeleton

The following skeleton uses the ability of SSG to perform decimal, hexadecimal, octal, and binary arithmetic like a calculator. The answer appears in decimal, hexadecimal (preceded by a $ sign), octal (preceded by a 0), and binary forms. This skeleton is intended for use on a UTS terminal, but can be adapted to any demand terminal.

```
SSG    REVISED SKELETON

 1.   0   *set clear = c'033145033115'
 2.   0   *display clear
 3.   0   *display    '          UTS calculator'
 4.   0   *display .
 5.   0   *display    '   Enter a number or numeric expression,
parentheses'
 6.   0   *display    '  are not recognized, only *, /, +, and - are
recognized'
 7.   0   *display .
 8.   0   *display    '  To enter a hexadecimal, octal, or binary
number, '
 9.   0   *display    '  use one of the following formats: '
10.   0   *display .
11.   0   *display '   hexadecimal -  h''hex-number'''
12.   0   *display '   decimal -  decimal-number'
13.   0   *display '   octal -  o''octal-number'''
14.   0   *display '   binary-  b''binary-number'''
15.   0   *display .
16.   0   *display 'The answer is displayed in decimal, hex, octal, and
binary'
17.   0   *display .
18.   0   *.
19.   0   *pcontrol 'd,@@esc o'
20.   0   *pcontrol 'd,@@ins 23'
21.   0   *pcontrol 'd,@@rlu'
22.   0   *.
23.   0   *set rs = c'036'
24.   0   *set next_line = c'033145033153033013066040017000000000'
```

```
25.   0   *set clear_line = c'033013066040017033115000'
26.   0   *.
27.   0   *increment l$1$ to 9000
28.   1   | *accept,e expression 'Expression [*rs]'
29.   1   | *set nofnd$ = 101
30.   1   | *if eof$ is set
31.   2   | | *display next_line
32.   2   | | *pcontrol 'd,@@end'
33.   2   | | *display 'End calculator'
34.   2   | | *set nofnd$ = 0
35.   2   | | *return
36.   1   | *endif
37.   1   | *set errors = errcnt$
38.   1   | *set ans = [*expression]
39.   1   | *if +nofnd$ > 101  or +errors <> +errcnt$
40.   2   | | *display clear_line '''' expression ''' is an
                invalid expression - reenter'
41.   2   | | *display next_line
42.   2   | | *iterate
43.   1   | *endif
44.   1   | *display ' = [*ans] / $[*ans,h] / [*ans,o] / [*ans,b] '
45.   1   | *display next_line
46.   0   *loop
```

Skeleton Analysis by Line Number

1.

The variable CLEAR is set to the ASCII control sequence to clear a UTS screen.

19.

An 'ER APRTCN$' is performed to cause escape mode on output.

24.

The variable NEXT_LINE is set to the ASCII control sequence to cause a <*cursor-to-home*> <*delete-line*> and then position the cursor to line 23, column 1. This gives the screen scrolling.

29.

NOFND$ is set to 101 to suppress NO FIND message from being printed.

38.

The expression entered is substituted into the *SET expression for evaluation.

39.

If a NO FIND or a SYNTAX ERROR occurred on the *SET directive, any error message printed by SSG is erased and replaced by the user's message.

44.

The result of the expression is printed in decimal, hexadecimal, octal, and binary.

C.6. Define Packet Examples

The following examples illustrate a set of define packets that can be used by any skeleton.

C.6.1. To Find a Day (Find_the_Day)

This code converts the month/day/year part of a TDATE$ word to days from January 1, 1964 [relative day 1, a Wednesday], including leap years, etc. It is assumed that the TDATE$ word [and DAYTAB are] in [main storage and can be referenced by placing TDATE and DAYTAB in the u-field of a referencing instruction].

The day of the week can be found from a date using the following algorithm:

```
            L,S3        A0,TDATE              . YEAR
            L,S1        A1,TDATE              . MONTH
            MSI,U       A0,365*4+1            .
            A           A0,DAYTAB-1,A1        .
            SSL         A0,2                  .
            A,S2        A0,TDATE              . DAY

DAYTAB      +           0*4+3                 . JAN
            +           31*4+3                . FEB
            +           59*4+4                .
            +           90*4+4                .
            +           120*4+4               .
            +           151*4+4               .
            +           181*4+4               .
            +           212*4+4               .
            +           243*4+4               .
            +           273*4+4               .
            +           304*4+4               .
            +           334*4+4               . DEC
```

Using this algorithm, a define packet such as the following can be produced that converts a MMDDYY-formatted date to the day of the week.

Features

- The algorithm requires the use of MMDDYY_Tab (DAYTAB) and MMDDYY_Day. If these SGSs are not defined when MMDDYY_to_Day is called, they are created.
- The algorithm is a faithful reproduction of the assembly language code with the following change:

  All multiplications and additions have been evaluated.

- Rather than day being placed seven times in MMDDYY_Day, it is appended to the created Day SGS. Although this saves only a small amount of main storage, it is, nevertheless, an efficient use of SSG.
- Process parameter reference method 2 is used in this define packet. This requires the global variable NEW_PROCESS$ to be set before calling Find_the_day.

- Since YY does not contain the 64 offset, the 64 offset is accounted for in the define packet. Note that, instead of subtracting 64 from YY at skeleton line 23, since 64*1461 produces a remainder of 3 at the end of the computation, MMDDYY_Day has been shifted.

Call

To compute the day on which SSG is called, enter

```
*PROCESS find_the_day [DATIME$,1,3,1]
```

Define Packet

```
SSG   REVISED SKELETON

   1.   0   *define find_the_day
   2.   1   | *.
   3.   1   | *.         Call:   *PROCESS FIND_THE_DAY [datime$,1,3,1]
   4.   1   | *.
   5.   1   | *.  The answer is returned in the string variable
                'DAY_OF_THE_WEEK'
   6.   1   | *.
   7.   1   | *.         [#,process_level$,1,1] is YYMMDD
   8.   1   | *.         Check that the parameter is 6 numeric digits
   9.   1   | *if +[#,process_level$,1,1,numeric_length$] <> 6
  10.   2   | | *display '[#,process_level$,1,1] is not a valid date'
  11.   2   | | *return
  12.   1   | *endif
  13.   1   | *.
  14.   1   | *.       Create the mmddyy_table SGS if there isn't one
  15.   1   | *if not [mmddyy_table]
  16.   2   | | *create sgs: mmddyy_table
                 3,127,240,364,484,608,728,852,976,1096,1220,1340
  17.   1   | *endif
  18.   1   | *.
  19.   1   | *.       Create the mmddyy_day SGS if there isn't one
  20.   1   | *if not [mmddyy_day]
  21.   2   | | *create sgs: mmddyy_day
Satur,Sun,Mon,Tues,Wednes,Thurs,Fri
  22.   1   | *endif
  23.   1   | *.
  24.   1   | *.      Pick the month, day, and year out of the string
  25.   1   | *set year = [#,process_level$,1,1,sub_string$,1,2]
  26.   1   | *set month = [#,process_level$,1,1,sub_string$,3,2]
  27.   1   | *set day = [#,process_level$,1,1,sub_string$,5,2]
  28.   1   | *.
  29.   1   | *.   Perform the computations for finding the day of the
                 week
  30.   1   | *if +year >= 64
  31.   2   | | *set year = year - 64
  32.   1   | *else
  33.   2   | | *set year = year + 100 - 64
  34.   1   | *endif
```

```
35.   1   | *set hold = year * 1461 + [mmddyy_table,1,1,month]
36.   1   | *set hold = hold / 4 + day + 3
37.   1   | *set temp = hold / 7
38.   1   | *set hold = hold - temp * 7
39.   1   | *set day_of_the_week = '[mmddyy_day,1,1,hold+1]day'
40.   1   | *.
41.   0   *enddefine
```

C.6.2. A Debug Aid (Echo_Pars)

When debugging a skeleton or a define packet in a skeleton, it is useful to dump process parameters.

Call

```
*Process Echo_Pars define name
```

where *define name* is the name of some define packet.

Define Packet

```
SSG   REVISED SKELETON

0001  00    *Define Echo_Pars
0002  01    *.
0003  01    *. [#1] is the name of some define packet
0004  01    *.
0005  01    *. Check each process parameter defined
0006  01    *.
0007  01    *Increment P to [#,[#1]]
0008  02      *.
0009  02      *. This process parameter exist (or is it a "no find"
                 process parameter)?
0010  02      *.
0011  02      *If [#P,[#1]]
0012  03        *Message,ti Par [*P] in define packet [#1] is [#P,[#1]]
0013  02      *Else
0014  03        *Message,ti Par [*P] in define packet [#1] is a "no
find"
0015  02      *Endif
0016  02      *.
0017  02      *. Pickup the next process parameter defined for [#1]
0018  02      *.
0019  01    *Loop
0020  00  *Enddefine
```

C.6.3. A Recursive Define Packet (Count_Nested_Calls)

This example illustrates a recursive define packet. Note the way the images are interpreted if you set the M and N options on the @SSG call. You should perform a breakpoint (@BRKPT).

Skeleton

```
SSG   REVISED SKELETON

0001  00  *Define Count_Nested_Calls
0002  01    *.
0003  01    *. This define packet illustrates use of recursion
0004  01    *.
0005  01    *. [#1] is the number of times to nest calls to
               Count_Nested_Calls
0006  01    *.
0007  01    *. This define packet counts down from [#1] to 1 by -1
0008  01    *. This is accomplished by calling Count_Nested_Calls
               with a value 1 less than [#1]
0009  01    *. When a value of 1 is reached, no more nested calls
               are requested
0010  01    *. This results in the most recent call to
               Count_Nested_Calls
               to complete processing
0011  01    *. This will set the value of COUNT to 1, since it is
               initially clear
0012  01    *. The next most recent call completes, setting COUNT
               to 2, and so on
0013  01    *. The final value of COUNT equals the first value of [#1]
0014  01    *.
0015  01    *. The variable COUNT is global (*SET creates a global
               variable)
0016  01    *. Therefore, when all calls are complete, COUNT equals
               the first [#1]
0017  01    *. Had an *INCREMENT loop been used, each call would have
               its own local variable
0018  01    *.
0019  01    *. Check to see if [#1] equals 1 yet
0020  01    *.
0021  01    *If value of [#1] > 1
0022  02      *.
0023  02      *. No, nest use of Count_Nested_Calls
0024  02      *.
0025  02      *Process Count_Nested_Calls [#1]-1
0026  02      *.
0027  02      *. This line isn't reached until the last call, when
                 [#1] equals 1
0028  02      *. (Actually, it is skipped on the last call, reached
                 on second to last, ...)
0029  02      *.
0030  01    *Endif
```

```
0031  01    *.
0032  01    *. Begin incrementing the value of COUNT
0033  01    *.
0034  01    *Set COUNT to COUNT+1
0035  01    *.
0036  01    *. All calls to Count_Nested_Calls ultimately reach here
0037  01    *.
0038  00  *Enddefine
0039  00  *.
0040  00  *. Clear COUNT so the first-executed *SET directive doesn't
            produce a "no find"
0041  00  *.
0042  00  *Clear COUNT
0043  00  *.
0044  00  *. Start the process, recurse 25 times
0045  00  *.
0046  00  *Process Count_Nested_Calls 25
0047  00  *.
0048  00  *. Verify the value of COUNT is now 25!
0049  00  *.
0050  00  *Message,ti Recursive depth is [*COUNT] (should be 25)!
```

Output

The following message should appear in the SSG run log:

```
Recursive depth is 25 (should be 25)!
```

C.6.4. Processing Unsolicited Input

The following example illustrates processing unsolicited console operator input, described in detail in 3.2.3.4. This skeleton executes for approximately 3 minutes and is designed to simulate a skeleton than runs for hours and processes hundreds of thousands of data records.

Skeleton

```
 1.   0   *. ----------------------------------------
 2.   0   *.        Check for unsolicited input
 3.   0   *. ----------------------------------------
 4.   0   *DEFINE check_unsolicited_input
 5.   1   | *.
 6.   1   | *.        Repeat while there is new unsolicited input
 7.   1   | *.
 8.   1   | *INCREMENT uns TO 9999999999 WHILE UNS_CONTROL$ IS SET
 9.   2   | | *SET save_input = UNS_INPUT$
10.   2   | | *CLEAR UNS_CONTROL$ . Enable new unsolicited input
11.   2   | | *.
12.   2   | | *.        Process the entered unsolicited input
13.   2   | | *.
14.   2   | | *IF [*save_input,UCSTR$] = HELP
15.   3   | | | *.
16.   3   | | | *.        This is a request for the valid unsolicited
keyins
17.   3   | | | *.
```

```
  18.   3   | | | *DISPLAY,O '-----------------------------------------
_'
  19.   3   | | | *DISPLAY,O 'Valid unsolicited keyins are:'
  20.   3   | | | *DISPLAY,O '  HELP - Display this HELP text'
  21.   3   | | | *DISPLAY,O '  STAT - Display the current record
summary'
  22.   3   | | | *DISPLAY,O '  TERM - Request early termination'
  23.   3   | | | *DISPLAY,O '-----------------------------------------
_'
  24.   3   | | | *.
  25.   2   | | *ELSEIF [*save_input,UCSTR$] = STAT
  26.   3   | | | *.
  27.   3   | | | *.        This is a request for the current skeleton
status
  28.   3   | | | *.
  29.   3   | | | *PROCESS record_summary
  30.   3   | | | *.
  31.   2   | | *ELSEIF [*save_input,UCSTR$] = TERM
  32.   3   | | | *.
  33.   3   | | | *.    This is a request for early termination of the
skeleton
  34.   3   | | | *.
  35.   3   | | | *CLEAR continue
  36.   3   | | | *.
  37.   2   | | *ELSE
  38.   3   | | | *.
  39.   3   | | | *.        This is not a recognized request
  40.   3   | | | *.
  41.   3   | | | *DISPLAY,O 'Unsolicited input request ''[*save_input]''
                  Not recognized'
  42.   3   | | | *DISPLAY,O 'Enter ''II [INFO$,1,1,2] HELP'' for valid
                  unsolicited requests'
  43.   3   | | | *.
  44.   2   | | *ENDIF
  45.   2   | | *.
  46.   1   | *LOOP . uns
  47.   1   | *.
  48.   0   *ENDDEFINE . check_unsolicited_input
  49.   0   *.
  50.   0   *. ----------------------------------------
  51.   0   *.        Output the current status
  52.   0   *. ----------------------------------------
  53.   0   *DEFINE record_summary
  54.   1   | *DISPLAY,O '-------------------------------------------'
  55.   1   | *DISPLAY,O 'Record Summary at [TIME$,1,1,1]'
  56.   1   | *DISPLAY,O '  Records processed: [*records_processed] of
                [*records_to_process]'
  57.   1   | *DISPLAY,O '  Records updated:   [*records_updated]'
  58.   1   | *DISPLAY,O '  Errors detected:   [*errors_detected]'
  59.   1   | *DISPLAY,O '  Serious errors:    [*serious_errors]'
  60.   1   | *DISPLAY,O '-------------------------------------------'
  61.   0   *ENDDEFINE . record_summary
  62.   0   *.
  63.   0   *. ----------------------------------------
  64.   0   *.        Start of the Main Control Section
  65.   0   *. ----------------------------------------
  66.   0   *.
  67.   0   *.        Skeleton initialization actions here
  68.   0   *.
  69.   0   *DISPLAY,O 'Record Update Utility (RUU) - version 1.0.3 -
            [DATIME$,1,1,1] [DATIME$,1,2,1]'
```

```
 70.   0   *.
 71.   0   *IF +UNS_CONTROL$ = -1
 72.   1   | *DISPLAY,O 'Enter ''II [INFO$,1,1,2] HELP'' for valid
             Unsolicited requests'
 73.   1   | *CLEAR UNS_CONTROL$ . Enable unsolicited input
 74.   0   *ELSE
 75.   1   | *DISPLAY,O 'This level of SSG does not support unsolicited
             requests'
 76.   1   | *CLEAR UNS_CONTROL$ . Define variable with permanent zero
             value
 77.   0   *ENDIF
 78.   0   *.
 79.   0   *SET records_to_process = 213869 . Simulated records to
           process
 80.   0   *CLEAR records_processed
 81.   0   *CLEAR records_updated
 82.   0   *CLEAR errors_detected
 83.   0   *CLEAR serious_errors
 84.   0   *SET continue
 85.   0   *.
 86.   0   *.        Main skeleton processing loop here
 87.   0   *.
 88.   0   *INCREMENT i TO 9999999999 WHILE continue IS SET
 89.   1   | *.
 90.   1   | *.        Check for unsolicited input during the previous
                       loop iteration.
 91.   1   | *.
 92.   1   | *PROCESS check_unsolicited_input
 93.   1   | *.
 94.   1   | *.        Check if an unsolicited termination request was
                       received.
 95.   1   | *.
 96.   1   | *IF continue IS CLEAR
 97.   2   | | *DISPLAY,O '* * * Early termination requested * * *'
 98.   2   | | *ITERATE
 99.   1   | *ENDIF
100.   1   | *.
101.   1   | *.        Proceed with (simulated) main loop activities
102.   1   | *.
103.   1   | *SET records_processed = records_processed + 5941
104.   1   | *SET records_updated = records_updated + 1269
105.   1   | *SET errors_detected = errors_detected + 3
106.   1   | *SET serious_errors = i / 12
107.   1   | *IF +records_processed >= +records_to_process
108.   2   | | *SET records_processed = records_to_process
109.   2   | | *CLEAR continue
110.   1   | *ENDIF
111.   1   | *INCREMENT w TO 5
112.   2   | | *WAIT 1 . Only 1 unsolicited input possible per *WAIT
               directive
113.   1   | *LOOP . w
114.   1   | *.
115.   1   | *.        End of (simulated) main loop activities
116.   1   | *.
117.   0   *LOOP . i
118.   0   *.
119.   0   *.        Skeleton termination actions here
120.   0   *.
121.   0   *PROCESS record_summary . Display the final summary.
122.   0   *.
123.   0   *DISPLAY,O 'END RUU - Errors: [*errors_detected], Serious
```

```
            Errors: [*serious_errors]'
 124.   0   *.
```

Skeleton Analysis by Line Number

4 – 48

Define packet that checks for and processes unsolicited console operator input.

8 – 10, 46

Directives that implement the second method of unsolicited input processing recommended in 3.2.3.4.

53 – 61

Define packet that outputs the record summary on an unsolicited input entry of STAT and on termination of the main processing loop.

71 – 73

Skeleton initialization directives that enable unsolicited input. This also illustrates how a skeleton can determine if the installed SSG supports unsolicited input.

74 – 77

Directives executed if the installed SSG does not support unsolicited input. This illustrates that a skeleton designed to process unsolicited input can still be executed with SSG 24R1 and earlier levels.

88 – 117

The main processing loop, with the recommended call to the unsolicited input check subroutine at line 92.

96 – 99

Main processing loop directives that handle an unsolicited input request for early termination.

121 – 123

Skeleton termination directives.

Console Output

The following shows the execution of the skeleton and the entered unsolicited input, as seen from the system console:

```
  RECUPD*Record Update Utility (RUU) - version 1.0.3 - 08/13/08 14:09:37
  RECUPD*Enter 'II RECUPD HELP' for valid unsolicited requests
ii recupd sta
  RECUPD*Unsolicited input request 'STA' not recognized
  RECUPD*Enter 'II RECUPD HELP' for valid unsolicited requests
ii recupd help
  RECUPD*-----------------------------------------
  RECUPD*Valid unsolicited keyins are:
```

```
  RECUPD*  HELP - Display this HELP text
  RECUPD*  STAT - Display the current record summary
  RECUPD*  TERM - Request early termination
  RECUPD*-----------------------------------------
ii recupd stat
  RECUPD*-----------------------------------------
  RECUPD*Record Summary at 14:10:17
  RECUPD*  Records processed: 47528 of 213869
  RECUPD*  Records updated:   10152
  RECUPD*  Errors detected:   24
  RECUPD*  Serious errors:    0
  RECUPD*-----------------------------------------
ii recupd stat
ii recupd stat
  RECUPD*PREVIOUS UNSOLICITED INPUT NOT PROCESSED - THIS UNSOLICITED
INPUT
  IGNORED
  RECUPD*-----------------------------------------
  RECUPD*Record Summary at 14:10:57
  RECUPD*  Records processed: 95056 of 213869
  RECUPD*  Records updated:   20304
  RECUPD*  Errors detected:   48
  RECUPD*  Serious errors:    1
  RECUPD*-----------------------------------------
ii recupd term
  RECUPD** * * Early termination requested * * *
  RECUPD*-----------------------------------------
  RECUPD*Record Summary at 14:11:07
  RECUPD*  Records processed: 106938 of 213869
  RECUPD*  Records updated:   22842
  RECUPD*  Errors detected:   54
  RECUPD*  Serious errors:    1
  RECUPD*-----------------------------------------
  RECUPD*END RUU - Errors: 54, Serious Errors: 1
```

Index

A

ACCSGS$, 5-5
ADD_GOS$, 5-8
ALEN$, 3-8
arithmetic overflow, 5-62
asterisk
  SGS label, 3-1
  use in defining directives, 4-5
automatically defined SGSs
  CSFSTA$, 3-20
  DATE$, 3-26
  DATIME$, 3-19
  INFO$, 3-21
  MARGIN$, 3-20
  SOURCE$, 3-24
  SUPS$, 3-25
  SYSTEM$, 3-22
  TIME$, 3-24

B

binary number, 4-2
binary numeric strings, 3-30
Boolean operations
  creating expressions, 5-32
  examples, 5-39, 5-40
bracketed references
  nested, 3-49
  process parameter, 5-64
  restrictions, 3-47
  SGS, 3-30

C

change control character, 6-11
changing
  character signifying a directive, 4-5
  variable values, 5-2
character directive, changing, 4-5
characters
  asterisk (to define a directive), 4-5
  change control), 6-11
  continuation, 3-2
  control (minus sign), 2-12
  dollar sign ($), 3-33, 5-66
  equals sign instead of TO, 5-69
  illegal # character, 3-4
  illegal in element names, 4-3
COLED$, 5-24
compound statements, 5-32
conditional test
  for element entry, 5-50
  for existence, 5-43
  for relation, 5-44
  for variable value, 5-42
  for zero, 5-43
  keyword search, 5-46
  using Boolean operators, 5-32
CONFLC$, 5-13, 5-57, A-1
continuation character, 3-2
continuing SGSs, 3-2
control character, 2-12
conventions, 1-1
correction set references, 6-5
correction, revised temporary (RTC), 6-12
CORRECTSKEL, C-14
creating variable values, 5-2
CSFSTA$, 3-20, 5-17

D

DATE$, 3-26
DATIME$, 3-19
define packet, 4-8
descriptor numbers, 3-7
destination
  revised skeleton stream, 2-4
  revised temporary correction (RIC) stream, 2-4
diagnostic messages, A-1
directives
  *ABORT, 5-4
  *ACCEPT, 3-35, 5-4
  *ADD, 3-35, 5-6
  *BRKPT, 2-13, 3-34, 5-7, 5-25

*CLEAR, 5-8
*COPY, 5-9
*CORRECT, 4-3, 5-10, 5-25, 5-57, 6-2, 6-11
*CREATE, 3-35, 4-3, 5-14
*CSF, 3-34, 3-35, 5-16
*DEFINE, 4-7, 5-18, 5-29
*DISPLAY, 5-19
*DIVIDE, 5-20
*DLOC, 5-21
*DO, 3-29, 5-22
*DUMP, 5-22
*EDIT, 5-23
*EJECT, 4-7, 5-27
*ELSE, 5-28
*ELSEIF, 5-28, 5-29, 5-30
*END, 5-30
*ENDCORRECT, 5-11, 5-28
*ENDDEFINE, 4-7, 5-18, 5-28
*ENDDO, 5-28
*ENDIF, 5-28, 5-29
*ENDO, 5-22
*EXIT, 5-28
*IF, 4-3, 5-29
*INCREMENT, 3-29, 5-22, 5-51
*ITERATE, 5-54
*LIST, 5-54
*LOOP, 5-51
*LOOP(,E), 5-54
*MERGE, 3-34, 4-3, 5-25, 5-55, 6-2, 6-11
*MESSAGE, 3-35, 5-60
*MULTIPLY, 5-62
*PCONTROL, 5-62
*PROCESS, 3-36, 4-8, 5-18, 5-63
*REMOVE, 4-3, 5-20, 5-65
*RESTORE, 5-67
*RETURN, 5-68
*SAVE, 5-68
*SET, 5-69
*SORT, 5-72
*UNLIST, 5-74
*WAIT, 5-74
types
miscellaneous, 5-3
PCF/TCF modification, 5-3
SGS modification, 5-3
skeleton communication, 5-2
skeleton creation, 5-1
skeleton listing, 5-1
skeleton logic control, 5-2
skeleton output control, 5-3
variable modification, 5-2
displaying
global variables, 5-23
local variables, 5-23
documentation updates, 1-1

E

EDITTAB$, 5-23
element entry defined, 6-2
ERRCNT$, 2-13, 5-8, 5-17, A-1
errors, 2-13
examples
correcting skeleton, C-14
correction image format, 6-1
Count_Nested_Calls, C-21
day of the year finder, C-18
debug aid, C-20
define packets, C-18
File Lister, C-12
RTC stream, 6-13
sample skeleton, B-4
search table, 5-46
set references, 6-6
setting a variable value, 5-70
SGS minor change format, C-1
Tic Tac Toe, C-2
UTS calculator, C-16
variable multiplication, 5-62

F

FEATURE_LEVEL$, 1-4, 3-43
field name references, 3-16
fields, SGS, 3-3
file identification statements
OPT, 2-6
OPT options, 2-9
options, 2-8
PCF, 2-6
SGS, 2-6
SKEL, 2-6
SKEL COR, 2-6
TCF, 2-6
file-id statement, 2-5, 6-3, 6-4
files
program files, 2-4
system data format (SDF), 2-3
format
SGS input, 3-1

G

global variables
  creation, 3-29
  diplay values of, 5-23
  list, 3-34
GOSCNT$, 4-4, 5-8, A-1

H

hexadecimal number, 4-2
hexadecimal numeric strings, 3-30

I

image
  nondirective, 4-4
  SGS format, 3-1
IMGLEN$, 3-9, 4-4, 5-25, 6-14
INFO$, 3-21
input
  SGS format, 3-1
  SSG, 2-11
IOPT$, 5-61

K

keyword search, 5-46

L

label
  access, 3-17
  legal characters, 2-4, 3-4
  SGS format, 3-4
large program files, 1-3
LBRK$, 4-1, 4-4, 5-6
LCSTR$, 3-34
LEPFs, *3-15*
limits, read, 4-8
line number conflicts, 5-13
line-change format, 6-8
local variables, 3-29, 5-23, 5-51
loops
  examples, 5-52
  nested, 5-51

M

MARGIN statement, 2-13
MARGIN$, 3-20
margins, 2-13
maximum number of words in read, 4-8
messages
  diagnostic, A-1
  warning, A-17
Modified-SWTIME, 3-6

N

nested loops, 5-51
NEW_BOOLEAN$, 5-35
NOFND$, A-1
nondirective images, 4-4
notation conventions, 1-1
NSIGN$, 4-4
NSIGNCHAR$, 4-4
null subfield, 3-4
NUMERIC relational tests, 5-44
numeric variables, 3-30

O

octal number, 4-2
octal numeric strings, 3-30
OPT, 2-6
options, SSG, 2-1

P

page eject, 5-27
PCF, 2-6, 5-47
PCF set access, 6-5
PCFNAME$ variable, 6-5
permanent correction file (PCF), 5-11
permanent correction file (PCF) sets
  defining, 6-3
  described, 6-1
permanent corrections (PCF streams), 1-3
permanent stream, 5-56
permanent TCF
  building a software product, B-1
  correction files, B-2
  general, B-1
  local use, B-3

sample skeleton, B-4
PILO$, 4-4
PILOCHAR$, 4-4
process parameters, 5-63
program
skeleton, 1-2
PSGS_STRING$, 3-31

Q

QLEN$, 3-8

R

referencing SGSs, 3-7
reserved global variables
*CORRECT, 3-34
ACCSGS$, 3-34
ADD_GOS$, 3-34
CASE$, 3-34
CASECHAR$, 3-34
COLED$, 3-34
CONFLC$, 3-34
CSFSTA$, 3-34
EDITAB$, 3-34
EOF$, 3-35
ERRCNT$, 3-35
FEATURE_LEVEL$, 3-35
FLD$, 3-35
GOSCNT$, 3-35
IMGLEN$, 3-35
IOPT$, 3-35
LBRK$, 3-35
MOPT$, 3-35
NEW_BOOLEAN$, 3-36
NEW_PROCESS$, 3-36
NOFND$, 3-36
NOPT$, 3-36
NSIGN$, 3-36
NSIGNCHAR$, 3-36
PCFNAME$, 3-36
PILO$, 3-36
PILOCHAR$, 3-36
PROCESS_LEVEL$, 3-36
PRTOFF$, 3-37
RTC$, 3-37
SFLD$, 3-37
SGSADDCNT$, 3-37
SGSCNT$, 3-37
SKELINE$, 3-37
SKELINECNT$, 3-37
SOPT$, 3-37
SSGSGS$, 3-37
STARTCHAR$, 3-37
STMT$, 3-37
STOPCHAR$, 3-37
STRINGCHAR$, 3-37
TCFNAME$, 3-37
UNS_CONTROL$, 3-38
UNS_INPUT$, 3-38
UTC$, 3-38
WARNINGCNT$, 3-38
WARNINGFLG$, 3-38
ZOPT$, 3-38
reserved label
$PCF, 6-5
$SGS, 3-9, 3-17, 5-64, 5-66
$TCF, 6-5
revised temporary correction (RTC)
stream, 6-12
RTC$ variable, 6-12
rules for composing directives, 4-6
rules for continuing input images, 2-11

S

scanning user-defined SGS labels, 5-53
searches, conditional, 5-46
semicolon continuation character, 3-2
SET, 3-29
SGS
examples, 3-26
field references, 3-16
label access, 3-17
source, 2-3
SGS file identification statement, 2-6
SGS_STRING$, 3-31
SGSs automatically defined, 3-19
SGSSTR$, 3-7
SKEL, 2-6
SKEL COR, 2-6
SKEL$, 5-63, A-1
skeleton (SKEL) stream, 2-12
skeleton correction (SKEL COR) stream, 2-12
skeleton samples, B-4
SKELINECNT$, A-1
source
corrections to skeleton stream, 2-4
SGSs, 2-3
skeleton stream, 2-3
SOURCE$, 3-24

SSG directives, 5-1
SSG$PF, 5-9
SSGSGS$ variable, 3-19
STARTCHAR$, 3-11
statements, file identification, 2-5
STMT$, 5-49
STOPCHAR$, 3-11
Stream Generation Statements (SGS)
  continuation, 3-2
  fields, 3-3
  format, 3-1
  references, 3-7
  types, 3-1
streams
  permanent, 6-3
  revised temporary correction (RTC), 6-12
  skeleton, 2-12
  skeleton correction, 2-12
  temporary, 6-4
string descriptor references, 3-8
string variables, 3-30
STRINGCHAR$, 3-10
SUBSTR$, 3-8
SUPS$, 3-25
SYMBOLIC relational tests, 5-44
Symbolic Stream Generator (SSG)
  @SSG control statement
    options, 2-1
    parameters, 2-3
  applications, 1-3
  general description, 1-2
  output, 2-13
Symstream language, 1-2
  definitions, 4-1
  directives list, 4-5
  sample definitions, 4-3
system data format (SDF) files, 2-3
SYSTEM$, 3-22

T

TCF, 2-6, 5-47
TCF set access, 6-5
TCFNAME$ variable, 6-5
TDATE$, 3-6
temporary correction file (TCF), 5-11
temporary correction file (TCF) sets
  defining, 6-4
  described, 6-1
temporary corrections (TCF streams), 1-3
test
  element entry conditional, 5-50
  existence conditional, 5-43
  relational conditional, 5-44
  variable value, 5-42
  zero conditional, 5-43
tests, relational, 5-44
TIME$, 3-24
TOC SGS date and time, 3-6
TOC SGS, long version, 3-5
TOC SGSs, 1-3

U

UCSTR$, 3-34
UNS_CONTROL$, 3-41
UNS_INPUT$, 3-41
User Communication Form (UCF), B-1

V

variable referencing, 3-29
variables
  creating and changing values, 5-69
  global, 3-29
  list of reserved global, 3-34
  list of string descriptor, 3-39
  local, 3-29, 5-51
  modifying, 5-2
  multiplication, 5-62
  referencing values, 3-29
  removal of global, 5-62
  SGS, 3-29

W

warning messages, A-17
WARNINGCNT$, 5-8

Special Characters

*ACCEPT directive, 5-4
*ADD directive, 5-6
*BRKPT directive, 5-7
*CLEAR directive, 5-8
*COPY directive, 5-9
*CORRECT directive, 5-10
*CREATE directive, 5-14
*CSF directive, 5-16

*DEFINE directive, 5-18
*DISPLAY directive, 5-19
*DIVIDE directive, 5-20
*DLOC directive, 5-21
*DO directive, 5-22
*DUMP directive, 5-22
*EDIT directive, 5-23
*EJECT directive, 5-27
*ELSE directive, 5-28, 5-29
*ELSEIF directive, 5-28, 5-29
*ENDCORRECT directive, 5-28
*ENDDEFINE directive, 5-18, 5-28
*ENDDO directive, 5-28
*ENDDO directives, 5-22
*ENDIF directive, 5-28, 5-29
*EXIT directive, 5-28
*IF directive, 5-29
*INCREMENT directive, 5-51
*ITERATE directive, 5-54
*LIST directive, 5-54
*LOOP directive, 5-51
*LOOP(,E) directive, 5-54
*MERGE directive, 5-55
*MESSAGE directive, 5-60
*MULTIPLY directive, 5-62
*PCONTROL directive, 5-62
*PROCESS directive, 5-63
*REMOVE directive, 5-65
*RESTORE directive, 5-67
*RETURN directive, 5-68
*SAVE directive, 5-68
*SET directive, 5-69
*SORT directive, 5-72
*UNLIST directive, 5-74
*WAIT directive, 5-74

© 2023 Unisys Corporation.
All rights reserved.

7831 1784–010